



CSC4602 Machine Learning

Campus Hazard Detection Mobile App Using YOLO Ensemble and Meta-Classfier

Group Members:

Name	Matric No
Liu Enzhuo	218472
Li Hongpu	218445
Wang Qihua	218576
He Jun	218406

Lecturer:
DR. NG KENG YAP

1. Introduction	7
1.1 Project Background	7
1.2 Problem Statement	8
1.3 Project Objectives	8
1.4 Project Scope	9
1.5 Expected Outcome	9
2. Campus Zones and Hazard Classes	9
2.1 Overview of Campus Hazard Zones	10
2.2 Member Model Categories and Hazard Classes	10
2.3 Hazard Class Descriptions	11
2.4 Overlapping and Related Hazard Classes	13
2.5 Global Label Mapping Purpose	14
2.6 Handling Rules for Overlap and Conflicting Predictions	15
2.7 Summary	16
3. System Architecture	16
3.1 Overview of the Proposed System Architecture	17
3.2 Main System Components	17
3.3 Mobile Application Interface	18
3.4 YOLO Detection Model Layer	19
3.5 Meta-Classifer Layer	20
3.6 LLM-Based Recommendation Module	21
3.7 Data Flow of the System	22
3.8 Backend and Model Integration Design	23
3.9 Output Display Design	24
3.10 Summary	24
4. Data Collection and Preparation	25
4.1 Overview of Data Collection	25
4.2 Data Collection by Member	25
4.3 Dataset Sources	26
4.4 Annotation Format	27
4.5 Dataset Folder Structure	28
4.6 Data Cleaning and Label Mapping	29
4.7 Dataset Splitting	29
4.8 Dataset Balancing	30
4.9 Pre-processing	31
4.10 Dataset Evidence and Report Images	31
4.11 Testing Image Preparation	32
4.12 Summary	32
5. Individual YOLO Model Training	33
5.1 Member 1: Walkway Surface Hazard Detection Model	33
5.1.1: Member 1 Model Overview	33

5.1.2: Dataset Preparation	34
Dataset Sources and Class Definition	35
5.1.3: Training Configuration	37
Training Configuration	37
Training Iterations and Improvement Attempts	37
5.1.4: Training Results	39
Training Results	39
5.1.5: Confusion Matrix and Error Analysis	40
Confusion Matrix and Error Analysis	41
5.1.6: Prediction Testing	42
5.1.7: Challenges and Limitations	47
Challenges and Limitations	47
5.1.8: Conclusion for Member 1	47
Conclusion	47
5.2 Member 2: Walkway Obstruction Hazard Detection Model	48
5.2.1: Member 2 Model Overview	48
5.2.2: Dataset Preparation	49
Dataset Sources and Class Definition	50
5.2.3: Training Configuration	52
Training Configuration	52
Training Iterations and Improvement Attempts	53
5.2.4: Training Results	53
Training Results	53
5.2.5: Confusion Matrix and Error Analysis	55
Confusion Matrix and Error Analysis	55
5.2.6: Prediction Testing	57
Prediction Testing	57
5.2.7: Challenges and Limitations	58
5.2.8: Conclusion for Member 2	59
5.3 Member 3: Drainage and Ground Surface Hazard Detection Model	59
5.3.1: Member 3 Model Overview	60
5.3.2: Dataset Preparation	61
Dataset Sources and Class Definition	62
5.3.3: Training Configuration	64
Training Iterations and Improvement Attempts	65
5.3.4: Training Results	66
5.3.5: Confusion Matrix and Error Analysis	67
5.3.6: Prediction Testing	68
5.3.7: Challenges and Limitations	71
5.3.8: Conclusion for Member 3	72
5.4 Member 4: Access, Boundary and Safety Facility Hazard Detection Model	73

5.4.1: Member 4 Model Overview	73
5.4.2: Dataset Preparation	75
Dataset Class Mapping	76
Dataset Distribution	77
Global Label Mapping for Member 4	78
5.4.3: Training Configuration	81
Training Iterations and Improvement Attempts	82
5.4.4: Training Results	84
Per-class Performance Results	85
5.4.5: Confusion Matrix and Error Analysis	86
5.4.6: Prediction Testing	88
5.4.7: Challenges and Limitations	89
5.4.8: Conclusion for Member 4	90
6. Meta-Classifier and Conflict Resolution	90
6.1 Introduction	90
6.2 Label Harmonisation and IoU Grouping	91
6.3 Meta-Classifier Feature Extraction	93
6.4 Meta-Classifier Architecture	94
6.5 Meta-Classifier Dataset	95
6.6 Training and Evaluation Results	96
6.7 Conflict Resolution Example	97
6.8 Detection Status and Reliability Handling	100
6.9 Final Inference Workflow	100
6.10 Limitation and Future Improvement	101
7. Mobile App Implementation	101
7. Mobile App Implementation	101
7.1 Development Framework and Tools	102
7.2 System Architecture of the Mobile App	102
7.3 User Workflow	103
7.4 Backend API Integration	103
7.5 YOLO Ensemble and Meta-Classifier Display	104
7.6 Bounding Box Visualisation	104
7.7 Gemini API Recommendation	105
7.8 Evidence Capture Feature	105
7.9 User Interface Design	106
7.10 Testing on Android Emulator and Web	106
7.11 Example Test Result	107
7.12 Limitations and Future Improvements	107
8. LLM-Based Recommended Action	108
8.1 Purpose	108
8.2 Recommendation Workflow	108

8.3 Input Data	109
8.4 LLM Implementation	110
8.5 Prompt Design	110
8.6 Safety Controls	111
8.7 Rule-Based Fallback Mechanism	112
8.8 Example Outputs	113
Example 1: Uncovered Manhole	113
Example 2: Wet and Slippery Floor	113
Example 3: Broken Handrail	113
8.9 Integration with the Mobile Application	114
8.10 Privacy and Security Considerations	114
8.11 Limitations	115
8.12 Summary	115
9. Mobile App Testing	116
9.1 Overview of Mobile App Testing	116
9.2 Testing Environment	117
9.3 Test Images	117
9.4 Functional Test Cases	118
9.5 Detection Testing Scenarios	120
9.6 Example App Output	121
9.7 Evidence Saving Test	121
9.8 Testing Observations	122
9.9 Summary	123
10. Results and Discussion	123
10.1 Overview of Results	123
10.2 Individual YOLO Model Results	124
10.3 Discussion of YOLO Model Performance	124
10.4 Meta-Classifer Results	125
10.5 Conflict Resolution Discussion	126
10.6 Mobile App Testing Results	127
10.7 LLM Recommendation Results	128
10.8 Overall System Discussion	128
10.9 Summary	129
11. Challenges and Limitations	129
11.1 Overview	129
11.2 Dataset Collection Challenges	129
11.3 Annotation and Label Mapping Limitations	130
11.4 Visual Similarity Between Hazard Classes	130
11.5 Individual YOLO Model Limitations	131
11.6 Meta-Classifer Limitations	131
11.7 Mobile App Limitations	132

11.8 LLM Recommendation Limitations	132
11.9 Testing Limitations	133
11.10 Summary of Challenges and Limitations	133
11.11 Future Improvements	134
11.12 Summary	135
12. Conclusion	135
12.1 Project Summary	135
12.2 Achievement of Project Objectives	136
12.3 Overall Findings	137
12.4 System Contribution	138
12.5 Limitations	138
12.6 Future Work	139
12.7 Final Conclusion	139
Appendix	139
Proposed Group Work Distribution	140
Development Logbook	140

1. Introduction

1.1 Project Background

Campus environments contain many public areas such as walkways, corridors, roads, parking areas, lecture blocks, student colleges, laboratories, and outdoor facilities. These areas are used daily by students, staff, visitors, and maintenance workers. However, different types of maintenance and safety hazards may appear in these areas, such as potholes, wet floors, cracked pavement, open drains, uncovered manholes, blocked walkways, damaged warning signs, broken handrails, and missing barricades.

If these hazards are not detected and reported early, they may increase the risk of slipping, tripping, falling, injury, blocked access, or delayed maintenance response. Traditional hazard reporting usually depends on manual observation, user complaints, or routine inspection. This process may be slow because some hazards are not reported immediately or may be missed during daily maintenance checks.

Therefore, this project proposes a **Campus Hazard Detection Mobile App Using YOLO Ensemble and Meta-Classifier**. The system uses multiple YOLO object detection models trained by different group members to detect different types of campus hazards. Each member model focuses on a specific hazard category, such as walkway surface hazards, obstruction hazards, drainage and ground hazards, and access or safety facility hazards.

To improve the final detection decision, the system also applies a neural-network-based meta-classifier. The meta-classifier combines the outputs from the individual YOLO models and resolves overlapping or conflicting predictions. After the final hazard class is produced, the mobile application generates a recommended maintenance action using an LLM-based recommendation module. This allows the system not only to detect hazards, but also to provide useful action suggestions for campus maintenance.

1.2 Problem Statement

Campus maintenance and safety hazards may occur in many different locations and forms. Some hazards are easy to notice, such as traffic cones or large obstacles on walkways, while others may be more difficult to detect, such as cracked pavement, uneven surfaces, damaged sidewalks, or uncovered manholes. If these hazards are not identified quickly, they may affect pedestrian safety and reduce the overall safety of the campus environment.

A single object detection model may not be sufficient to cover all types of campus hazards because the hazard classes are visually different and may come from different campus contexts. For example, surface hazards such as potholes and cracked pavement are different from safety facility hazards such as damaged warning signs or broken handrails. In addition, some classes may overlap between member models, such as wet/slippery floor, pothole,

cracked pavement, and damaged sidewalk. These overlapping classes may produce similar or conflicting predictions.

Therefore, the problem addressed in this project is the need for a simple mobile-based hazard detection system that can detect multiple types of campus hazards, combine predictions from several YOLO models, resolve overlapping labels, and provide practical maintenance recommendations.

1.3 Project Objectives

The main objective of this project is to develop a mobile application that can detect campus maintenance and safety hazards in real time using YOLO object detection models, a neural-network meta-classifier, and an LLM-based recommendation module.

The specific objectives are:

1. To collect and prepare annotated datasets for different campus hazard categories using YOLO format.
2. To train individual YOLO object detection models, where each group member is responsible for five hazard classes.
3. To design a global label mapping structure for handling exact overlaps, similar labels, generalised classes, and contextual overlaps between member models.
4. To develop a neural-network-based meta-classifier that combines outputs from multiple YOLO models and produces the final hazard class.
5. To integrate the trained models and meta-classifier into a mobile application for real-time or near real-time hazard detection.
6. To generate short and practical maintenance recommendations using an LLM-based recommendation module.
7. To evaluate the individual YOLO models, meta-classifier, and mobile application through performance metrics, prediction examples, and testing scenarios.

1.4 Project Scope

The scope of this project focuses on detecting selected campus maintenance and safety hazards using image-based object detection. The system covers four main model areas developed by different group members:

- Walkway surface hazards
- Walkway obstruction hazards
- Drainage and ground surface hazards
- Access, boundary, and safety facility hazards

Each member trained one YOLO model with five hazard classes. The final system is designed to combine the outputs of these models using a meta-classifier. The mobile application is

expected to display the final hazard class, confidence score, severity level, bounding box, and recommended maintenance action.

The project does not aim to replace professional campus safety inspection. Instead, it serves as a prototype system to support faster hazard identification and reporting. The dataset is mainly collected from public datasets, safe examples, and selected campus-related images. No real hazards were created, and no unsafe actions were performed during data collection.

1.5 Expected Outcome

By the end of the project, the group is expected to produce a working prototype of a campus hazard detection mobile application. The system should be able to detect selected hazard classes, combine multiple YOLO model outputs using a meta-classifier, and display recommended maintenance actions. The final outcome also includes trained YOLO model files, meta-classifier files, dataset evidence, source code, testing results, screencast video, and a technical report documenting the full development process.

2. Campus Zones and Hazard Classes

2.1 Overview of Campus Hazard Zones

The campus hazard detection system is designed to identify different types of maintenance and safety hazards that may appear in common university areas. These areas include walkways, corridors, roads, parking areas, staircases, ramps, drainage areas, outdoor facilities, and areas near construction or maintenance work.

In this project, the hazard classes were divided into four main model focuses. Each group member trained one YOLO object detection model with five hazard classes. This division allows each model to focus on a specific type of hazard instead of training one large model with too many visually different classes.

The four model focuses are:

Member	Model Focus	Main Campus Zone
Member 1	Walkway Surface Hazards	Walkways, pavements, sidewalks, corridors, and pedestrian paths

Member 2	Walkway Obstruction Hazards	Walkways, pedestrian routes, campus corridors, and access paths
Member 3	Drainage and Ground Surface Hazards	Drainage areas, road edges, pavement areas, and ground surfaces
Member 4	Access, Boundary and Safety Facility Hazards	Walkways, staircases, ramps, restricted areas, safety facilities, and boundary areas

This structure helps the system detect a wider range of campus hazards while keeping each individual model focused and manageable.

2.2 Member Model Categories and Hazard Classes

Each member was responsible for training one YOLO model with five hazard classes. The selected classes represent different hazard types that may affect pedestrian safety, access, visibility, or maintenance response on campus.

Member	Model Focus	Hazard Classes
Member 1	Walkway Surface Hazards	pothole, cracked_pavement, wet_slippery_floor, surface_damage, damaged_sidewalk
Member 2	Walkway Obstruction Hazards	obstacle_on_walkway, construction_debris, fallen_branch, traffic_cone, wet_slippery_floor
Member 3	Drainage and Ground Surface Hazards	open_drain, uncovered_manhole, pothole, uneven_pavement, cracked_pavement
Member 4	Access, Boundary and Safety Facility Hazards	missing_barricade, damaged_warning_sign, broken_handrail, blocked_walkway, damaged_sidewalk

These classes were selected because they are common or important hazards in campus environments. Some hazards affect the walking surface directly, such as potholes and cracked pavement. Some hazards block pedestrian movement, such as fallen branches and construction debris. Other hazards are related to drainage, ground openings, access control, or

safety facilities, such as open drains, uncovered manholes, damaged warning signs, and broken handrails.

2.3 Hazard Class Descriptions

The following table summarises the main meaning and safety concern of each hazard class used in the project.

Hazard Class	Description	Possible Safety Risk
pothole	A hole or depression on a road, walkway, or pavement surface	May cause tripping, falling, or vehicle damage
cracked_pavement	Visible cracks on pavement, road, or walkway surface	May become worse over time and create walking difficulty
wet_slippery_floor	Wet or slippery walking surface	May cause slipping accidents
surface_damage	General damage on walkway or pavement surface	May reduce walking safety and comfort
damaged_sidewalk	Broken or damaged sidewalk area	May cause tripping or falling accidents
obstacle_on_walkway	Object placed on or blocking a pedestrian route	May obstruct safe pedestrian movement
construction_debris	Construction materials or debris left on pedestrian or access areas	May block access or create tripping hazards
fallen_branch	Tree branch fallen on a walkway or pedestrian area	May block movement or cause injury

traffic_cone	Traffic cone placed on or near a pedestrian route	May indicate temporary warning, obstruction, or maintenance area
open_drain	Drainage opening or exposed drain area	May cause falling or foot injury
uncovered_manhole	Manhole without proper cover or protection	High-risk hazard that may cause serious falls or injury
uneven_pavement	Uneven ground or pavement surface	May cause tripping or balance problems
missing_barricade	Area that lacks proper barricade or boundary protection	May allow people to enter unsafe or restricted areas
damaged_warning_sign	Warning sign that is damaged, blocked, unclear, or unreliable	May reduce hazard awareness
broken_handrail	Damaged, loose, or unsafe handrail or railing	May increase falling risk on stairs, ramps, or elevated walkways
blocked_walkway	Walkway or access route blocked by objects or materials	May reduce accessibility and create obstruction hazards

2.4 Overlapping and Related Hazard Classes

Since each member trained a separate YOLO model, some hazard classes are shared or related across different models. These overlapping classes are important because they allow the final system to compare outputs from multiple YOLO models and improve the final prediction through the meta-classifier.

The major overlapping or related classes are shown below.

Overlap / Related Class	Related Members	Relationship Type	Explanation
wet_slippery_floor	Member 1 and Member 2	Exact overlap	Both models include wet_slippery_floor because it can be considered both a surface hazard and a walkway safety hazard.
pothole	Member 1 and Member 3	Exact overlap	Potholes are related to both walkway surface hazards and drainage or ground surface hazards.
cracked_pavement	Member 1 and Member 3	Exact overlap	Cracked pavement appears in both surface hazard and ground surface hazard contexts.
damaged_sidewalk	Member 1 and Member 4	Exact overlap	Damaged sidewalk is related to both surface condition and access or safety facility hazards.
obstacle_on_walkway / blocked_walkway	Member 2 and Member 4	Contextual overlap	Both classes describe objects or conditions that block pedestrian movement, but the class names and contexts are slightly different.
construction_debris / blocked_walkway	Member 2 and Member 4	Contextual overlap	Construction debris may become a blocked walkway hazard if it appears on a pedestrian path.

traffic_cone / missing_barricade	Member 2 and Member 4	Contextual relationship	Traffic cones and barricades may both be used to mark unsafe areas or guide pedestrians away from hazards.
-------------------------------------	-----------------------------	----------------------------	--

These overlaps are useful for ensemble learning because more than one model may detect the same or related hazard. However, they may also create conflicts if different models produce different class names or confidence scores for the same object. Therefore, a global label mapping and meta-classifier are needed.

2.5 Global Label Mapping Purpose

The purpose of global label mapping is to organise all member model labels into a consistent structure. Since the four YOLO models were trained separately, each model has its own class ID and class name. For example, Member 2 may detect an object as `obstacle_on_walkway`, while Member 4 may detect a similar situation as `blocked_walkway`. Without global label mapping, the system may treat them as completely different hazards even though they are related.

Global label mapping helps the system to:

1. Identify exact overlap classes between different models.
2. Group similar or related hazard labels.
3. Resolve conflicts between different model predictions.
4. Support the neural-network meta-classifier.
5. Produce one final hazard class for the mobile application.

The meta-classifier uses this mapping to compare model outputs based on confidence score, bounding-box location, IoU agreement, model reliability, and hazard relationship type. This allows the final system to make a more stable decision when multiple models detect the same or related hazard.

2.6 Handling Rules for Overlap and Conflicting Predictions

When two or more YOLO models detect the same or related hazard, the system applies handling rules before producing the final output. These rules help the meta-classifier decide which prediction should be selected as the final hazard result.

Situation	Example	Handling Rule
Exact overlap with same class name	Member 1 and Member 3 both detect pothole	Compare confidence score, IoU agreement, and model reliability. If both agree, keep the same final class.
Exact overlap with different confidence values	Member 1 detects damaged_sidewalk with 0.72 confidence, Member 4 detects damaged_sidewalk with 0.81 confidence	Select the prediction with stronger confidence if bounding boxes match.
Similar or related labels	Member 2 detects obstacle_on_walkway, Member 4 detects blocked_walkway	Treat both as related obstruction hazards and allow the meta-classifier to decide the final global label.
Contextual relationship	Member 2 detects traffic_cone, Member 4 detects missing_barricade	Use location and context to decide whether it is a warning object, boundary protection issue, or access hazard.
Low confidence prediction	Any model detects a hazard with very low confidence	The meta-classifier may reduce the final confidence or mark the result as uncertain.
Multiple hazards in one image	One image contains traffic_cone and construction_debris	Keep multiple detections if they are separate objects and display them individually in the mobile application.

2.7 Summary

In summary, the project covers a wide range of campus hazards across walkway surfaces, obstructions, drainage areas, ground surfaces, access areas, boundaries, and safety facilities. Each member model contributes to a different part of the overall hazard detection system. Since several classes overlap or are contextually related, global label mapping is required to standardise the class structure and support the meta-classifier. This ensures that the final mobile application can display a clearer and more reliable hazard detection result.

3. System Architecture

3.1 Overview of the Proposed System Architecture

The proposed system architecture is designed as a mobile-based campus hazard detection system that combines multiple YOLO object detection models, a neural-network meta-classifier, and an LLM-based recommendation module. The system allows users to capture or upload an image of a campus area, detect possible hazards, resolve overlapping predictions, and display a recommended maintenance action.

The overall system follows a multi-stage detection pipeline. First, an image is captured or selected through the mobile application. Then, the image is sent to the detection backend, where the four trained YOLO models analyse the image. Each YOLO model produces detection results such as bounding boxes, predicted class labels, confidence scores, and model IDs. After that, these outputs are processed by the meta-classifier to determine the final global hazard class. Finally, the final detection result is passed to the LLM recommendation module to generate a short and practical maintenance recommendation.

This architecture was selected because the project contains multiple hazard categories that are visually different. Instead of training only one large YOLO model for all hazard classes, each member trained a specialised YOLO model for a smaller group of related hazards. The meta-classifier is then used to combine these individual model outputs and produce a more consistent final result.

Figure 3.1: Overall system architecture of the campus hazard detection mobile application

[Insert system architecture diagram here]

3.2 Main System Components

The system consists of five main components: mobile application interface, YOLO detection models, meta-classifier, LLM recommendation module, and data/result storage.

Component	Description
Mobile Application Interface	Allows the user to capture or upload an image and view the final hazard detection result.
YOLO Detection Models	Four trained YOLO models detect different categories of campus hazards.
Meta-Classfier	Combines and resolves outputs from the YOLO models to produce the final global hazard class.
LLM Recommendation Module	Generates a short maintenance recommendation based on the final hazard class, severity, confidence score, and location zone.
Data and Result Storage	Stores model files, testing images, detection records, result screenshots, and report evidence.

3.3 Mobile Application Interface

The mobile application acts as the user-facing part of the system. It allows users to provide image input and view the final detection result. The app is designed to support either image upload or camera-based input, depending on the final implementation.

The main functions of the mobile interface include:

Function	Description
Image Input	Allows the user to capture a campus hazard image or upload an existing image.
Detection Request	Sends the image to the backend for YOLO model inference and meta-classifier processing.

Result Display	Displays the final hazard class, bounding box, confidence score, severity level, and recommended action.
Recommendation Display	Shows the LLM-generated or fallback maintenance recommendation.
Testing Support	Allows the group to test the system using selected walkway-related hazard images.

The mobile application does not directly decide the final hazard class. Instead, it sends the image to the detection pipeline and displays the processed result returned by the backend. This design keeps the app simpler and allows heavier processing, such as running multiple YOLO models, to be handled by the backend.

Figure 3.2: Mobile application interface prototype
[Insert mobile app screenshot here after integration is completed]

3.4 YOLO Detection Model Layer

The YOLO detection model layer consists of four individual YOLO models trained by the group members. Each model focuses on a different campus hazard category. When an input image is received, the image can be passed to the relevant YOLO models or all four models depending on the final integration design.

The models produce the following outputs:

Output	Description
Model ID	Identifies which member model produced the detection.
Predicted Class	The hazard class predicted by the YOLO model.
Confidence Score	The probability score of the prediction.
Bounding Box	The location of the detected hazard in the image.
Class ID	The local class ID used by the individual YOLO model.

The four YOLO models are summarised below.

Model	Model Focus	Example Classes
Member 1 YOLO Model	Walkway Surface Hazards	pothole, cracked_pavement, wet_slippery_floor, surface_damage, damaged_sidewalk
Member 2 YOLO Model	Walkway Obstruction Hazards	obstacle_on_walkway, construction_debris, fallen_branch, traffic_cone, wet_slippery_floor
Member 3 YOLO Model	Drainage and Ground Surface Hazards	open_drain, uncovered_manhole, pothole, uneven_pavement, cracked_pavement
Member 4 YOLO Model	Access, Boundary and Safety Facility Hazards	missing_barricade, damaged_warning_sign, broken_handrail, blocked_walkway, damaged_sidewalk

This model layer allows the system to detect a wider range of campus hazards compared with a single specialised model.

3.5 Meta-Classifier Layer

The meta-classifier is responsible for combining the outputs from the individual YOLO models. Since some classes overlap across different models, the system requires a decision layer to produce one final global hazard result.

For example, both Member 1 and Member 3 models may detect pothole or cracked_pavement. Member 1 and Member 4 may both detect damaged_sidewalk. Member 2 may detect obstacle_on_walkway, while Member 4 may detect blocked_walkway. These predictions may be exact overlaps, similar labels, or contextual overlaps. Therefore be exact overlaps, similar labels, or contextual overlaps. Therefore, the meta-classifier uses structured features from YOLO outputs to decide the final class.

The possible input features for the meta-classifier include:

Feature	Description
Model ID	The YOLO model that produced the prediction

Local Class ID	The class ID inside the individual YOLO model
Global Label ID	The mapped global hazard class
Confidence Score	YOLO confidence value
Bounding Box Coordinates	x, y, width, and height of the detected object
Bounding Box Area	Size of the detected hazard region
IoU Agreement	Overlap between bounding boxes from different models
Relationship Type	Exact overlap, similar label, generalisation, or contextual overlap
Location Zone	Walkway, corridor, drainage area, staircase, or other campus zone

The meta-classifier output includes:

Output	Description
Final Hazard Class	The final global hazard class selected by the meta-classifier
Final Confidence	The final confidence score after combining model outputs
Severity Level	Low, Medium, or High severity based on the hazard type and confidence
Handling Decision	Explanation of how overlapping or conflicting predictions were resolved

The meta-classifier improves the system by reducing confusion between related classes and helping the application display a single final result to the user.

3.6 LLM-Based Recommendation Module

After the meta-classifier produces the final hazard result, the system sends structured hazard information to the LLM-based recommendation module. The purpose of this module is to generate a short, practical, and safety-oriented maintenance recommendation.

The input to the LLM recommendation module includes:

Input	Example
Final Hazard Class	uncovered_manhole
Hazard Category	drainage and ground surface hazard
Location Zone	campus walkway
Severity Level	High
Confidence Score	0.91

Based on this information, the LLM generates a recommended maintenance action. For example, if the detected hazard is uncovered_manhole with high severity, the recommendation may be:

“Place a temporary barricade around the uncovered manhole, keep pedestrians away from the area, and report it to the campus maintenance unit for urgent repair.”

A fallback recommendation dictionary can also be prepared. This ensures that the mobile application can still display a suitable maintenance action if the LLM API is unavailable or the internet connection fails.

3.7 Data Flow of the System

The system data flow starts from image input and ends with a final hazard result and maintenance recommendation.

The overall data flow is as follows:

Step	Process	Description
1	Image Input	The user captures or uploads a campus hazard image through the mobile app.

2	Image Pre-processing	The image is resized or prepared for YOLO model inference.
3	YOLO Detection	The trained YOLO models detect possible hazards and generate bounding boxes, class labels, and confidence scores.
4	Global Label Mapping	Local class labels from different YOLO models are mapped into global hazard labels.
5	Meta-Classifier Processing	The meta-classifier combines YOLO outputs and resolves overlapping or conflicting predictions.
6	Final Hazard Output	The system produces the final hazard class, confidence score, severity level, and bounding box.
7	LLM Recommendation	The final hazard information is sent to the LLM recommendation module.
8	Result Display	The mobile app displays the detection result and recommended maintenance action.

The simplified process can be represented as:

Mobile Image Input

→ YOLO Detection Models

→ Global Label Mapping

→ Meta-Classifier

→ Final Hazard Result

→ LLM Recommendation

→ Mobile App Display

3.8 Backend and Model Integration Design

The backend is designed to handle the model inference and recommendation process. This is useful because running four YOLO models and a meta-classifier directly on a mobile device may require higher processing power. By using a backend-based approach, the mobile application can remain lightweight while the backend handles detection and decision-making.

The backend integration is expected to include the following functions:

Backend Function	Description
Receive Image	Accepts an image sent from the mobile application.
Run YOLO Models	Runs the trained YOLO model files on the input image.
Collect Predictions	Stores predicted class names, confidence scores, and bounding boxes.
Apply Global Label Mapping	Converts local model labels into global hazard labels.
Run Meta-Classifer	Produces the final hazard class and severity level.
Generate Recommendation	Calls the LLM module or fallback dictionary to generate maintenance advice.
Return Result	Sends the final detection result back to the mobile application.

The backend can be implemented using a lightweight API framework such as Flask or FastAPI. The trained YOLO model files, including each member's best.pt file, can be loaded by the backend for inference.

3.9 Output Display Design

The final output displayed in the mobile application should be simple and easy to understand. The system should show the detected hazard clearly and provide a practical recommendation.

The expected output includes:

Output Item	Example
Hazard Detected	damaged_warning_sign
Confidence Score	0.82
Severity Level	Medium

Location Zone	campus walkway
Bounding Box	Displayed around the detected hazard
Recommended Action	Repair or replace the damaged warning sign and ensure the safety message is clearly visible.

A possible app result display is shown below:

Hazard Detected: damaged_warning_sign

Confidence: 0.82

Severity: Medium

Location: Campus Walkway

Recommended Action:

Repair or replace the damaged warning sign and ensure the safety message is clearly visible to pedestrians.

3.10 Summary

In summary, the proposed system architecture combines mobile application input, four YOLO object detection models, global label mapping, a neural-network meta-classifier, and an LLM-based recommendation module. The YOLO models detect different campus hazard categories, while the meta-classifier resolves overlapping or conflicting predictions. The LLM recommendation module then generates a practical maintenance action based on the final hazard result. This architecture supports the development of a campus hazard detection prototype that can assist in faster hazard identification and reporting.

4. Data Collection and Preparation

4.1 Overview of Data Collection

Data collection is an important stage in this project because the YOLO object detection models require annotated image datasets for training and validation. Since the project covers different types of campus hazards, each group member collected and prepared a separate dataset based on their assigned model focus and hazard classes.

The datasets were mainly collected from public object detection datasets, online image sources, and safe campus-related examples. The images were selected to represent real or realistic campus safety and maintenance hazards such as damaged pavement, wet floors, walkway obstruction, construction debris, fallen branches, traffic cones, open drains, uncovered manholes, broken handrails, damaged warning signs, and missing barricades.

All images were prepared in YOLO object detection format. This means that each image has a corresponding text annotation file containing the class ID and bounding-box coordinates. The bounding boxes identify the location of the hazard object or hazard area in the image.

During data collection, the group followed safety rules. No real hazard was created for photography, no campus property was damaged, no restricted area was entered, and no dangerous object was moved or modified. The images were used only for academic model training, testing, and demonstration purposes.

4.2 Data Collection by Member

Each member collected data based on a specific hazard category. This division allowed each YOLO model to focus on a smaller and more meaningful set of hazard classes.

Member	Model Focus	Hazard Classes	Dataset Purpose
Member 1	Walkway Surface Hazards	pothole, cracked_pavement, wet_slippery_floor, surface_damage, damaged_sidewalk	To detect surface-related hazards on campus walkways, pavements, and pedestrian areas
Member 2	Walkway Obstruction Hazards	obstacle_on_walkway, construction_debris, fallen_branch, traffic_cone, wet_slippery_floor	To detect objects or materials that may block or interrupt safe pedestrian movement
Member 3	Drainage and Ground Surface Hazards	open_drain, uncovered_manhole, pothole, uneven_pavement, cracked_pavement	To detect drainage openings, manhole hazards, and ground surface problems

Member 4	Access, Boundary and Safety Facility Hazards	missing_barricade, damaged_warning_sign, broken_handrail, blocked_walkway, damaged_sidewalk	To detect access, boundary, and safety facility hazards such as damaged signs, handrails, barricades, and sidewalk damage
----------	--	---	---

4.3 Dataset Sources

The datasets were prepared from different public sources and manually organised according to the project class structure. Since different datasets used different original labels, label mapping was required before training.

For example, some datasets used labels such as “crack”, “damaged-sidewalk”, “obstacles”, “worksite_materials”, “handrail”, and “Road Barriers”. These labels were reviewed and mapped into the final class names used by the group models.

Member	Dataset Type	Example Original Labels	Final Labels
Member 1	Walkway and pavement surface images	pothole, crack, wet floor, damaged pavement, damaged sidewalk	pothole, cracked_pavement, wet_slippery_floor, surface_damage, damaged_sidewalk
Member 2	Walkway obstruction and object images	obstacles, debris, fallen branch, traffic cone, wet floor	obstacle_on_walkway, construction_debris, fallen_branch, traffic_cone, wet_slippery_floor
Member 3	Drainage and road surface images	drain, manhole, pothole, uneven pavement, crack	open_drain, uncovered_manhole, pothole, uneven_pavement, cracked_pavement

Member 4	Barrier, warning sign, handrail, obstruction, and sidewalk images	Road Barriers, damaged sign labels, handrail, obstacles, damaged-sidewalk	missing_barricade, damaged_warning_sign, broken_handrail, blocked_walkway, damaged_sidewalk
-------------	---	---	---

The final report appendix can include dataset links, downloaded dataset folders, annotation examples, and screenshots from Roboflow, Kaggle, or other public dataset sources.

4.4 Annotation Format

All datasets were prepared using YOLO annotation format. In YOLO format, each image has a corresponding `.txt` label file with the same file name. Each line in the label file represents one annotated object.

The YOLO annotation format is:

Class ID, x_center, y_center, width, height

The values of x_center, y_center, width, and height are normalised between 0 and 1. This means that the values are relative to the image size instead of using absolute pixel coordinates.

Example YOLO annotation:

0 0.456 0.417 0.258 0.283

This means that the object belongs to class ID 0 and has a bounding box located at the given normalised centre position, width, and height.

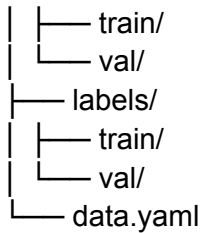
Using YOLO format allowed the group to train the models directly using the Ultralytics YOLO framework.

4.5 Dataset Folder Structure

Each member organised their dataset using a standard YOLO dataset folder structure. The dataset was divided into training and validation folders.

The general dataset structure is shown below:

```
dataset/
├── images/
```



The `images/train` folder contains training images, while the `images/val` folder contains validation images. The `labels/train` and `labels/val` folders contain the corresponding YOLO annotation files. The `data.yaml` file defines the dataset path, training folder, validation folder, number of classes, and class names.

A sample `data.yaml` file is shown below:

```
path: dataset_path
train: images/train
val: images/val
```

```
nc: 5
names:
0: class_1
1: class_2
2: class_3
3: class_4
4: class_5
```

4.6 Data Cleaning and Label Mapping

Before training, the datasets were cleaned and reorganised. This step was necessary because the original datasets were collected from different sources and did not always use the same class names or label meanings.

The data cleaning process included:

1. Removing unsuitable or unrelated labels.
2. Remapping original labels into the final project class names.
3. Checking that each image had a matching label file.
4. Removing images without useful hazard objects.
5. Ensuring that the class IDs matched the `data.yaml` file.
6. Separating images and labels into training and validation folders.

For example, in Member 4's dataset, the normal warning sign class "ok" was excluded because it did not represent a hazard. Obstacle-related labels such as "obstacles", "stalls", and "worksite_materials" were mapped into `blocked_walkway`. The original label "handrail" was

mapped into `broken_handrail` because public damaged handrail datasets were limited. The original labels “crack” and “damaged-sidewalk” were mapped into `damaged_sidewalk`.

This label mapping process helped standardise the datasets and ensured that each YOLO model trained on the correct final class structure.

4.7 Dataset Splitting

The datasets were split into training and validation sets. The training set was used to train the YOLO models, while the validation set was used to evaluate the model performance during and after training.

The purpose of dataset splitting is to test whether the model can detect hazards on images that were not used during training. This helps evaluate the generalisation ability of the model.

Member	Training Images	Validation Images	Notes
Member 1	1500	400	Walkway surface hazard dataset
Member 2	Approximately 1000	246	Walkway obstruction hazard dataset
Member 3	1922 total images	To be updated	Drainage and ground surface hazard dataset
Member 4	320	80	Balanced access, boundary, and safety facility hazard dataset
Total	To be updated	To be updated	Final total depends on all member dataset folders

For Member 4, the dataset was balanced so that each of the five classes contained 80 images. Each class had 64 training images and 16 validation images, giving a total of 320 training images and 80 validation images.

4.8 Dataset Balancing

Dataset balancing was applied when needed to reduce the effect of class imbalance. Class imbalance occurs when some hazard classes have many more images than others. If the

dataset is highly imbalanced, the YOLO model may learn to detect the larger classes better while performing poorly on smaller classes.

For Member 4, the original datasets had very different numbers of images. For example, the damaged warning sign dataset contained many more images than the missing barricade and broken handrail datasets. Therefore, a balanced dataset was created by selecting the same number of images for each class.

Member 4 Class	Training Images	Validation Images	Total Images
missing_barricade	64	16	80
damaged_warning_sign	64	16	80
broken_handrail	64	16	80
blocked_walkway	64	16	80
damaged_sidewalk	64	16	80
Total	320	80	400

This balanced dataset helped ensure that all five Member 4 classes had equal representation during model training and validation.

4.9 Pre-processing

Before training, the images were automatically processed by the YOLO training pipeline. The main pre-processing steps included image resizing, normalisation, and data augmentation. The images were resized to the image size used during training, such as 640×640 for several member models.

YOLO training may also apply augmentation techniques such as scaling, flipping, colour adjustment, and mosaic augmentation. These techniques help the model learn more general visual patterns and improve detection under different lighting, angles, and background conditions.

Pre-processing is important because campus hazard images may vary in image size, lighting, camera angle, object distance, and background complexity. By using standardised YOLO pre-processing, the model can learn more stable features from the training data.

4.10 Dataset Evidence and Report Images

To provide evidence of dataset preparation and training, each member included dataset and training output images in the report or appendix. These images help show that the dataset was correctly prepared and that the model was successfully trained.

The recommended evidence images include:

Evidence Image	Purpose
labels.jpg	Shows label distribution and bounding-box annotation overview
train_batch0.jpg	Shows sample training images and annotations
val_batch0_labels.jpg	Shows validation images with ground-truth labels
val_batch0_pred.jpg	Shows validation prediction examples
results.png	Shows training and validation performance curves
confusion_matrix.png	Shows class-level detection performance and errors

For the final technical report, only the most important images should be placed in the main body. Additional evidence images can be placed in the appendix to avoid making the report too long.

4.11 Testing Image Preparation

Besides the training and validation datasets, the group also prepared separate testing images for mobile application and integration testing. These testing images were collected separately from the training datasets to check whether the system could detect unseen examples.

The testing images focused on walkway-related hazard scenarios, including uneven pavement, obstacle on walkway, fallen branch, construction debris, traffic cone, broken handrail, damaged warning sign, and uncovered manhole.

The testing folder can be organised as follows:

```
test_images_walkway/  
├── uneven_pavement/  
├── obstacle_on_walkway/  
├── fallen_branch/  
├── construction_debris/  
├── traffic_cone/  
└── broken_handrail/
```

└─ damaged_warning_sign/
└─ uncovered_manhole/

Each class can contain three to five testing images. These images are used for app demonstration, meta-classifier testing, LLM recommendation testing, and final screencast recording.

4.12 Summary

In summary, the group prepared separate YOLO-format datasets for different campus hazard categories. The datasets were collected from public sources and safe campus-related examples, cleaned, remapped, and organised into standard YOLO folder structures. Each member trained one YOLO model using their prepared dataset. The dataset preparation stage also included label harmonisation, data splitting, dataset balancing, and testing image preparation. These steps provided the foundation for individual YOLO training, meta-classifier integration, mobile application testing, and LLM-based recommendation generation.

5. Individual YOLO Model Training

5.1 Member 1: Walkway Surface Hazard Detection Model

5.1.1: Member 1 Model Overview

Member 1 was responsible for developing a YOLO object detection model focusing on walkway and pedestrian surface hazards. This model was designed to detect hazards that may cause students, staff, or visitors to slip, trip, or fall when using campus pedestrian areas. The selected hazard classes were pothole, cracked pavement, wet/slippery floor, surface damage, and damaged sidewalk.

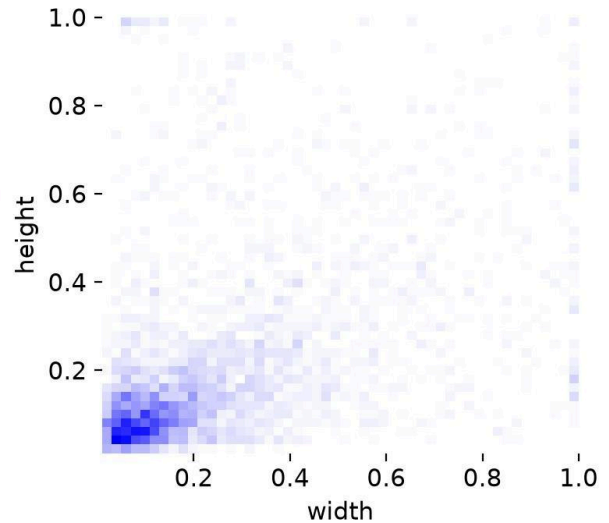
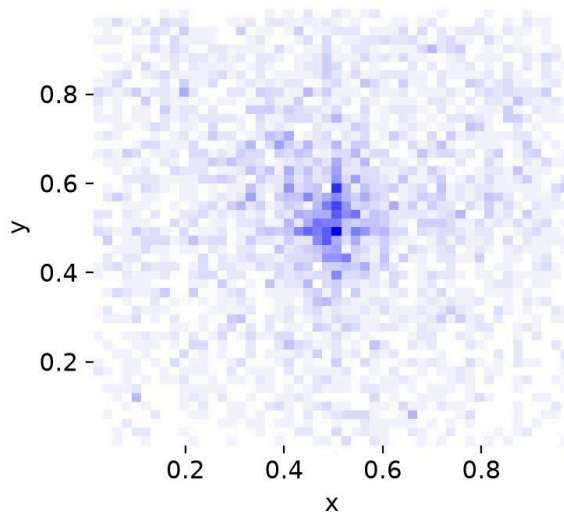
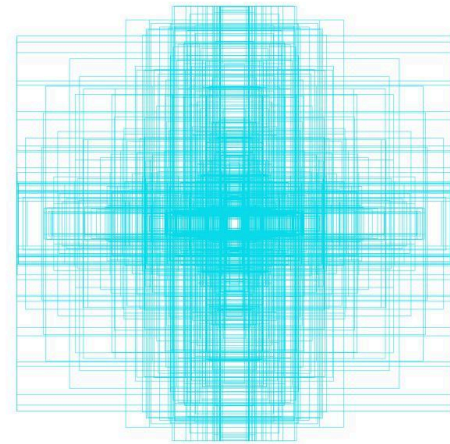
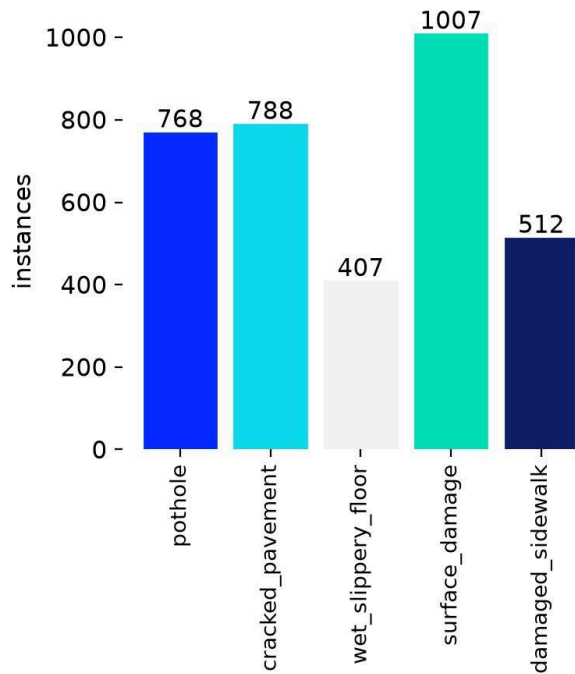
These classes were chosen because they are commonly related to walkway safety and campus maintenance issues. Potholes and cracked pavement may cause tripping accidents, while wet or slippery floors may increase the risk of slipping. Surface damage and damaged sidewalk represent general pedestrian surface defects that require maintenance attention. Therefore, this model contributes to the overall campus hazard detection system by identifying surface-related risks in pedestrian areas.

Item	Description
------	-------------

Member	LIU ENZHUO
Model Focus	Walkway Surface Hazards
YOLO Version	YOLOv8n
Number of Classes	5
Classes	pothole, cracked_pavement, wet_slippery_floor, surface_damage, damaged_sidewalk
Training Images	1500
Validation Images	400
Epochs	20
Image Size	640
Batch Size	8

5.1.2: Dataset Preparation

Figure 1: Label distribution and annotation overview for Member 1 dataset



The dataset for Member 1 was prepared by combining five separate public object detection datasets related to walkway surface hazards. Each original dataset was first checked to ensure that it contained image files, YOLO annotation labels, and a data configuration file. Since each downloaded dataset originally used its own class index, the labels were reorganised and mapped into a unified five-class structure.

The final class mapping used for Member 1 was as follows: pothole was assigned as class 0, cracked pavement as class 1, wet/slippy floor as class 2, surface damage as class 3, and damaged sidewalk as class 4. After merging the datasets, a balanced dataset was created to reduce class imbalance. For each class, 300 images were used for training and 80 images were

used for validation. As a result, the final dataset contained 1500 training images and 400 validation images.

The final dataset was saved in YOLO format with separate image and label folders for training and validation. This structure allowed the YOLOv8n model to read the images and corresponding bounding box annotations correctly during training.

Dataset Sources and Class Definition

The Member 1 dataset was collected from public object detection datasets related to walkway and pedestrian surface hazards. Since the original datasets came from different sources, their labels were reviewed and renamed into a consistent five-class structure before training. This step was necessary because each dataset originally used its own class label and class ID.

The five final classes were selected based on their relevance to campus walkway safety. Pothole refers to holes or depressions on road or walkway surfaces. Cracked pavement refers to visible cracks on the pedestrian surface. Wet/slippery floor refers to water, puddles, or slippery walking surfaces. Surface damage refers to general visible surface defects, such as broken surface areas or damaged paving blocks. Damaged sidewalk refers to larger sidewalk defects that may affect safe pedestrian movement.

This class design allows the model to detect different types of walkway surface hazards that may cause slipping, tripping, falling, or maintenance issues in campus pedestrian areas.

Class ID	Class Name	Train Images	Validation Images
0	pothole	300	80
1	cracked_pavement	300	80
2	wet_slippery_floor	300	80
3	surface_damage	300	80
4	damaged_sidewalk	300	80
Total		1500	400

Table: Public Dataset Sources Used for Member 1 Model

Final Class	Dataset Used	Source	License	Original Label
pothole	Pothole dataset	Roboflow Universe	CC BY 4.0 / Public	pothole
cracked_pavement	Road crack dataset	Roboflow Universe	CC BY 4.0 / Public	crack / road crack
wet_slippery_floor	Wet floor / puddle dataset	Roboflow Universe	CC BY 4.0 / Public	wet floor / puddle
surface_damage	Sidewalk damage dataset	Roboflow Universe	Public Domain / CC BY 4.0	sidewalk damage
damaged_sidewalk	Damaged sidewalk dataset	Roboflow Universe	CC BY 4.0	damaged sidewalk

Class	Definition	Safety Risk
pothole	A hole or depression on the walkway or road surface	May cause tripping or falling
cracked_pavement	Visible cracks on pavement or pedestrian surface	May develop into larger surface damage
wet_slippery_floor	Water, puddle, or slippery surface on walkway	May cause slipping
surface_damage	General visible surface defects, damaged blocks, or broken surface areas	May affect walking safety

damaged_sidewalk	Broken or damaged sidewalk structure	May cause pedestrian injury or block safe movement
------------------	--------------------------------------	--

5.1.3: Training Configuration

Training Configuration

The YOLOv8n model was selected because it is a lightweight YOLO model suitable for faster training and potential mobile or real-time deployment. The model was trained using the balanced Member 1 dataset for 20 epochs. The image size was set to 640, and the batch size was set to 8. Since GPU acceleration was not available on the training device, the training was conducted using CPU.

The training output was saved in the project directory under the folder `E:\member1_surface_hazards\runs\detect\member1_v2_final_20epochs`. The best-performing model was saved as `best.pt`, while the final epoch model was saved as `last.pt`. These trained model files can be used later for prediction testing, mobile app integration, or ensemble processing with other group members' models.

Training Iterations and Improvement Attempts

Several preparation and training steps were completed before the final model was produced. First, five separate YOLO datasets were merged into one dataset. During this process, all class IDs were remapped into a unified five-class structure. Second, a balanced dataset was created because the original datasets had different numbers of images. For each class, 300 images were selected for training and 80 images were selected for validation. This helped reduce class imbalance during training.

The final training iteration used YOLOv8n with 20 epochs, image size 640, and batch size 8. The model was trained using CPU because GPU acceleration was not available. The final training output produced the best model file, `best.pt`, and the last epoch model file, `last.pt`. The best model was then used for validation and prediction testing.

Parameter	Value
YOLO Version	YOLOv8n

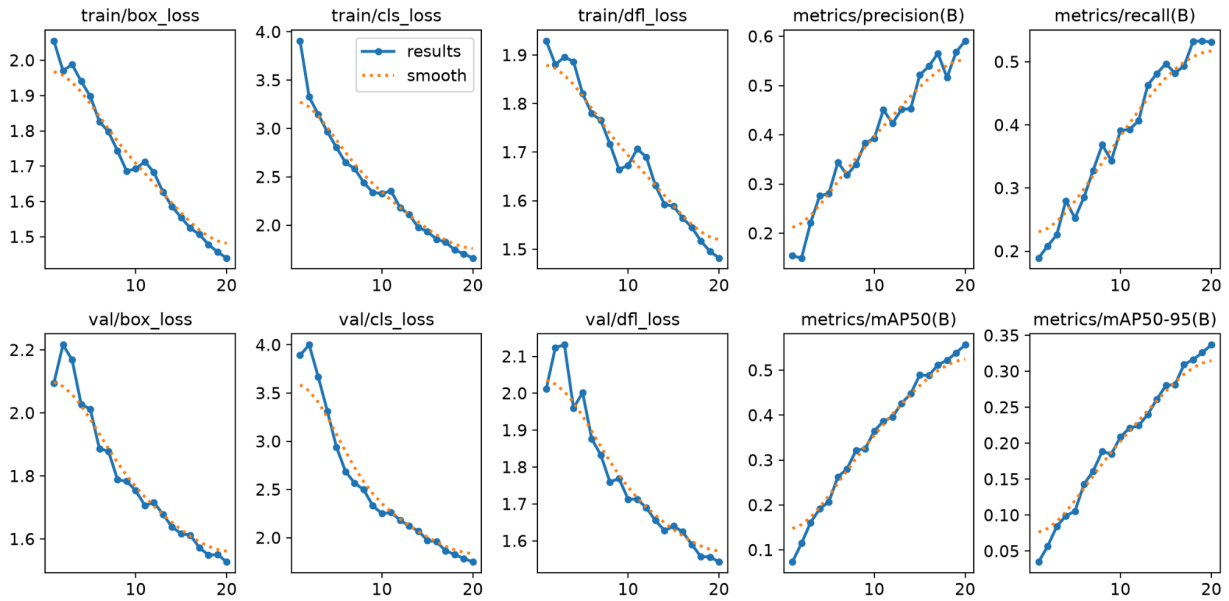
Model File	yolov8n.pt
Epochs	20
Image Size	640
Batch Size	8
Device	CPU
Optimizer	AdamW
Training Time	1.761 hours
Output Folder	member1_v2_final_2 0epochs
Best Model	weights/best.pt

Training Iteration

Iteration	Purpose	Action	Outcome
Initial dataset merge	Combine five datasets	Unified class IDs from 0 to 4	Created final_merged_dataset
Dataset balancing	Reduce class imbalance	Selected 300 train and 80 validation images per class	Created final_balanced_dataset
Final training	Train final model	YOLOv8n, 20 epochs, image size 640, batch 8	Generated best.pt and evaluation results

5.1.4: Training Results

Figure 2: Training performance curves for Member 1 YOLOv8n model



Training Results

After 20 epochs of training, the Member 1 YOLOv8n model achieved an overall precision of 0.588, recall of 0.530, mAP@0.5 of 0.556, and mAP@0.5:0.95 of 0.337 on the validation dataset. These results show that the model was able to learn the main visual patterns of the selected walkway hazard classes, although some classes still showed lower performance due to visual similarity and dataset variation. Among the five classes, wet/slippery floor achieved the strongest performance, with a mAP@0.5 of 0.886 and mAP@0.5:0.95 of 0.743. This indicates that the model was able to detect wet or slippery floor conditions more effectively than the other classes. In contrast, surface damage achieved the lowest mAP@0.5 of 0.321, which suggests that this class was more difficult for the model to distinguish from other surface-related classes such as damaged sidewalk and cracked pavement.

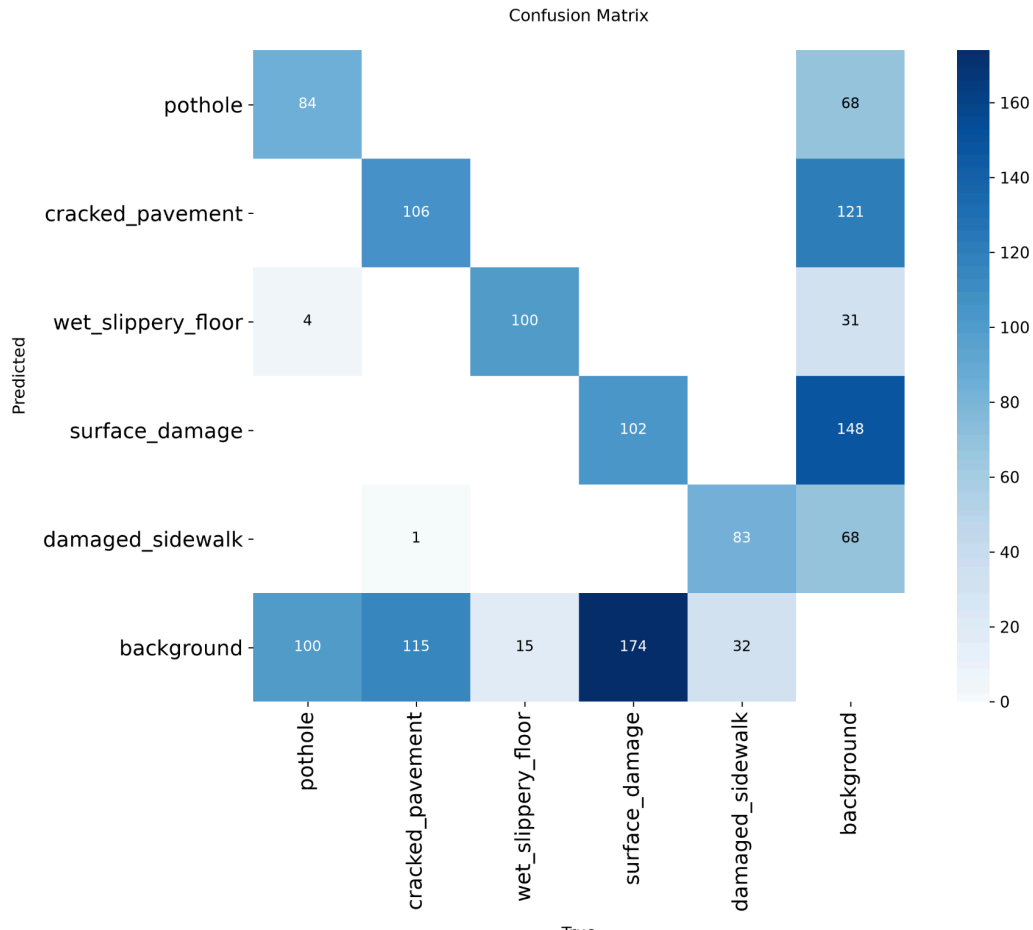
Table: Per-class performance results of Member 1 YOLOv8n model

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
pothole	0.563	0.399	0.446	0.215
cracked_pavement	0.552	0.422	0.428	0.187
wet_slippery_floor	0.773	0.843	0.886	0.743
surface_damage	0.450	0.301	0.321	0.154
damaged_sidewalk	0.603	0.687	0.701	0.384

Metric	Result
Precision	0.588
Recall	0.530
mAP@0.5	0.556
mAP@0.5:0.95	0.337

5.1.5: Confusion Matrix and Error Analysis

Figure 3: Confusion matrix of Member 1 YOLOv8n model



Confusion Matrix and Error Analysis

The confusion matrix was used to analyse how well the model classified each hazard class. The results show that the model performed better on wet/slippery floor and damaged sidewalk compared to the other classes. However, some confusion occurred between visually similar surface hazard classes, especially cracked pavement, surface damage, and damaged sidewalk.

One possible reason for this confusion is that these classes share similar visual features, such as irregular ground texture, cracks, broken surface patterns, and damaged walkway areas. In addition, some images may contain more than one surface defect, which makes class separation more difficult. For example, a damaged sidewalk may also contain cracked pavement, and surface damage may appear visually similar to uneven or broken walkway areas.

Another limitation is that the model was trained using public datasets, so the lighting conditions, camera angles, pavement materials, and environmental backgrounds may not fully match the actual campus environment. Therefore, future improvement should include more real campus

walkway images collected from the selected campus zones. This would help the model adapt better to local walkway conditions.

5.1.6: Prediction Testing

Figure 4 : Overall validation prediction examples generated by Member 1 YOLOv8n model



Figure 4 shows the overall prediction examples generated by the trained Member 1 YOLOv8n model on validation images. The model successfully detected multiple cracked pavement areas and drew bounding boxes around visible pavement cracks. This result shows that the trained model can perform object detection inference and generate visual outputs with predicted class labels. However, most examples in this batch belong to the cracked_pavement class, so additional single-image prediction examples are included below to show the other hazard classes such as wet/slippery floor, surface damage, damaged sidewalk, and pothole.

Figures: Prediction examples generated using the trained Member 1 YOLOv8n model

pothole



cracked_pavement



damaged_sidewalk



wet_slippery_floor



surface_damage



After training, the best model file `best.pt` was used to perform prediction testing on validation images. The prediction results show that the model was able to draw bounding boxes around detected walkway hazards and display the predicted class name with a confidence score. This confirms that the trained model can be used for inference and can provide visual detection output for later integration into the mobile application.

Figure 4 shows the overall validation prediction examples generated by the trained Member 1 YOLOv8n model. Most examples in this validation batch belong to the `cracked_pavement` class, and the model successfully detected multiple visible pavement crack areas. Therefore, additional single-image prediction examples are included to show the other hazard classes, including pothole, wet/slippery floor, surface damage, and damaged sidewalk.

The prediction examples show that wet/slippery floor was detected with high confidence, while surface damage and cracked pavement sometimes had lower confidence. This is consistent with the evaluation results, where wet/slippery floor achieved the highest mAP, while surface damage had the lowest performance. Some bounding boxes were not perfectly aligned, which indicates that the model can still be improved with more real campus images and clearer class definitions.

5.1.7: Challenges and Limitations

Challenges and Limitations

Several challenges were encountered during the development of the Member 1 YOLO model. First, suitable public datasets for some walkway hazard classes were difficult to find. For example, broken tile and uneven floor datasets were not easily available, so the class design was adjusted to include surface damage and damaged sidewalk, which are more suitable and available for walkway hazard detection.

Second, the original datasets had different class labels and different annotation formats. Therefore, the labels had to be reorganised into a unified five-class format before training. Some datasets also contained a much larger number of images than others, so a balanced dataset was created to reduce class imbalance.

Third, the model was trained using CPU because GPU acceleration was not available. This increased the training time and limited the number of training experiments that could be performed. Lastly, the model still had difficulty distinguishing between similar surface hazard classes such as cracked pavement, surface damage, and damaged sidewalk. Future work should include more real campus images, better class definitions, and additional training iterations to improve performance.

5.1.8: Conclusion for Member 1

Conclusion

In conclusion, Member 1 successfully trained a YOLOv8n model for detecting five walkway surface hazard classes. The final model was trained using 1500 training images and validated using 400 images. The model achieved an overall mAP@0.5 of 0.556 and was able to detect several important walkway hazards, including potholes, cracked pavement, wet/slippery floors, surface damage, and damaged sidewalks.

Although the model showed acceptable detection capability, further improvement is still needed, especially for visually similar classes such as surface damage and cracked pavement. The trained `best.pt` model will be used as Member 1's YOLO model output and can be integrated with other group members' models through the meta-classifier in the final campus hazard detection mobile application.

5.2 Member 2: Walkway Obstruction Hazard Detection Model

5.2.1: Member 2 Model Overview

Member 2 was responsible for developing a YOLO object detection model focusing on walkway obstruction hazards. The purpose of this model is to identify hazards that may block pedestrian movement, create tripping risks, or reduce safe access along campus walkways, corridors, and outdoor pedestrian routes.

The selected hazard classes were `obstacle_on_walkway`, `construction_debris`, `fallen_branch`, `traffic_cone`, and `wet_slippery_floor`.

These classes were selected because they represent common hazards that may affect pedestrian safety on campus. Obstacles on walkways may interrupt normal movement and create tripping risks. Construction debris such as bricks, metal bars, wood, or damaged materials may obstruct pedestrian paths and cause injuries. Fallen branches may block outdoor walkways, especially after rain or strong wind. Traffic cones may temporarily reduce walkway access when placed near maintenance areas or pedestrian routes. Wet or slippery floors may increase the risk of slipping accidents in corridors, public facilities, and outdoor walkways.

The class `wet_slippery_floor` overlaps with Member 1's Walkway Surface Hazard Detection Model. This overlap was intentionally included to support the ensemble learning and meta-classifier design. During final inference, outputs from both models can be compared using confidence scores, bounding-box IoU agreement, model reliability, and location context before determining the final hazard result.

Therefore, Member 2's model contributes to the overall campus hazard detection system by identifying obstruction-related and access-related hazards that may affect pedestrian safety.

Item	Description
Member	Li Hongpu
Model Focus	Walkway Obstruction Hazards
YOLO Version	YOLOv8n
Number of Classes	5
Classes	<code>obstacle_on_walkway</code> , <code>construction_debris</code> , <code>fallen_branch</code> , <code>traffic_cone</code> , <code>wet_slippery_floor</code>

Training Images	1,000 images
Validation Images	246 images
Epochs	20
Image Size	640x640
Batch Size	8
Training Device	GPU
Model Output	best.pt

5.2.2: Dataset Preparation

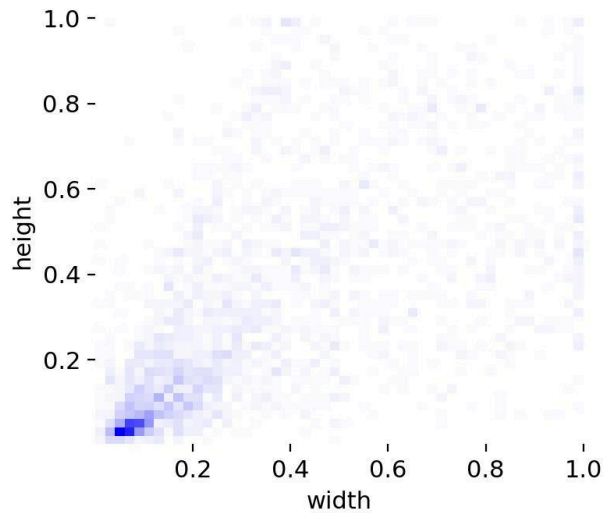
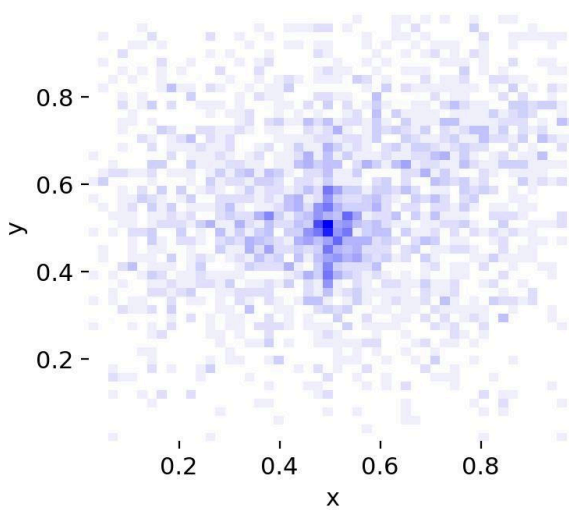
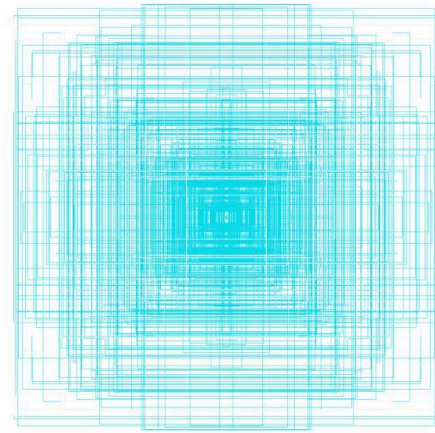
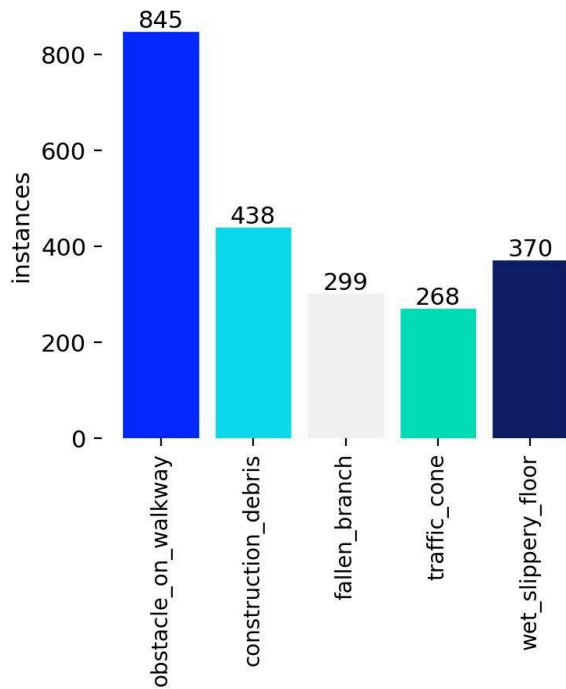
The dataset for Member 2 was prepared by combining five separate public object detection datasets related to walkway obstruction hazards. Each downloaded dataset was checked to ensure that it contained image files, YOLO annotation labels, and a dataset configuration file. Since the original datasets used different class names and class IDs, the labels were reorganised and mapped into one unified five-class structure.

The final class mapping used for Member 2 was as follows: `obstacle_on_walkway` was assigned as class 0, `construction_debris` as class 1, `fallen_branch` as class 2, `traffic_cone` as class 3, and `wet_slippery_floor` as class 4.

The datasets were merged into one final YOLO-format dataset. To reduce training time while keeping sufficient examples for each hazard class, a smaller balanced dataset was created for the initial training experiment. The final dataset contained approximately 1,000 training images and 246 validation images.

The final dataset was saved in YOLO format with separate image and label folders for training and validation. This structure allowed the YOLOv8n model to correctly load the images and corresponding bounding-box annotations during training.

Figure 1: Label distribution and annotation overview for Member 2 dataset



Dataset Sources and Class Definition

The Member 2 dataset was collected from public object detection datasets related to pedestrian walkway hazards, construction materials, fallen tree parts, traffic cones, and wet floor conditions. The labels from the original datasets were reviewed and renamed into a consistent five-class structure before training.

Class ID	Class Name	Description	Safety Risk
----------	------------	-------------	-------------

0	obstacle_on_walkway	Objects located on pedestrian routes, such as bags, boxes, rubbish, bicycles, or other items blocking a walkway	May interrupt movement and cause tripping
1	construction_debris	Construction-related materials or waste such as bricks, metal bars, wood, pipes, and damaged materials	May cause injuries or block pedestrian access
2	fallen_branch	Fallen tree branches or tree parts located on outdoor paths or walkways	May block the walkway and create trip hazards
3	traffic_cone	Traffic cones placed near maintenance zones or pedestrian routes	May reduce available walkway space or indicate temporary hazards
4	wet_slippery_floor	Visible water, puddles, or wet/slippery surfaces on indoor corridors or outdoor walkways	May cause slipping accidents

The `wet_slippery_floor` class overlaps with Member 1's model. The same class name and definition were used to support the ensemble and meta-classifier design. During final inference, the system can compare the predictions from both models using confidence score, bounding-box agreement, and model reliability.

Dataset Distribution

Class Name	Training Instances	Validation Instances
obstacle_on_walkway	845	745
construction_debris	438	147
fallen_branch	299	265
traffic_cone	268	223
wet_slippery_floor	370	96

The label distribution shows that `obstacle_on_walkway` contained the highest number of annotations, while `traffic_cone` contained the lowest number. Although the dataset was not perfectly balanced, each class contained sufficient annotated examples for initial YOLO model training.

5.2.3: Training Configuration

Training Configuration

The YOLOv8n model was selected because it is a lightweight object detection model suitable for faster training and future real-time or mobile application deployment. The model was trained using the prepared Member 2 walkway obstruction hazard dataset.

The initial training experiment used an image size of 416×416 , batch size 8, and 5 epochs. This lower image size was selected because training was conducted using CPU, and the original combined dataset contained a large number of images. The configuration reduced training time while still allowing the model to produce detection outputs for evaluation.

The training output was saved in the following directory:

`/content/runs/detect/member2_obstruction_hazards_small`

The best-performing model was saved as `best.pt`, while the final epoch model was saved as `last.pt`. These model files can later be used for prediction testing, mobile app integration, and ensemble processing with the models trained by other group members.

Parameter	Value
YOLO Version	YOLOv8n
Model File	yolov8n.pt
Epochs	5
Image Size	416x416
Batch Size	8
Device	CPU
Optimizer	AdamW
Training Dataset	Member 2 reduced final dataset
Training Images	Approximately 1,000
Validation Images	246
Output Folder	member2_obstruction_hazards_small
Best Model	weights/best.pt

Training Iterations and Improvement Attempts

Several preparation and training steps were completed before producing the initial Member 2 model. First, five public datasets were downloaded and merged into one unified dataset. During this process, the original class labels were remapped into the final five-class structure.

Second, a reduced dataset was created from the larger merged dataset. This was necessary because CPU training with the full dataset at a higher image resolution required a very long training time. The reduced dataset retained examples from all five hazard classes and was used to perform the initial YOLOv8n training experiment.

The final initial training iteration used YOLOv8n with 5 epochs, image size 416×416 , and batch size 8. The model was trained using CPU and produced the `best.pt` model file together with training curves, confusion matrix, and prediction examples.

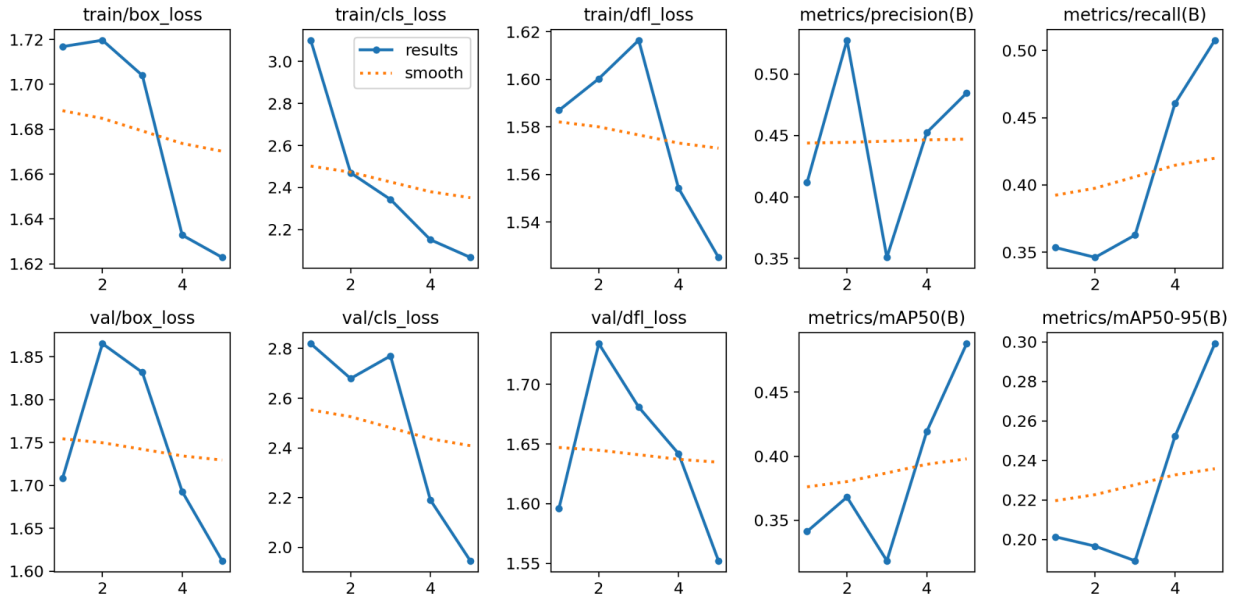
Training Iteration	Purpose	Action	Outcome
Initial dataset merge	Combine five hazard datasets	Unified the class IDs from 0 to 4	Created final merged dataset
Dataset reduction	Reduce CPU training time	Selected a smaller dataset while keeping all five classes	Created <code>final_dataset_small</code>
Initial training	Train baseline model	YOLOv8n, 5 epochs, image size 416, batch size 8	Generated <code>best.pt</code> and evaluation results

5.2.4: Training Results

Training Results

The Member 2 YOLOv8n model was trained for 5 epochs using an image size of 416×416 and a batch size of 8. The training was performed using CPU on the reduced five-class walkway obstruction hazard dataset.

Figure 2: Training performance curves for Member 2 YOLOv8n model



The training curves show that the training losses generally decreased over the five epochs. The box loss, classification loss, and distribution focal loss showed an overall downward trend, indicating that the model was learning useful visual patterns from the dataset.

The precision, recall, mAP@0.5, and mAP@0.5:0.95 values also improved during the later epochs. However, some fluctuations were observed because the model was trained for only five epochs and the selected hazard classes had different visual characteristics and dataset distributions.

After training, the model achieved an overall precision of 0.485, recall of 0.508, mAP@0.5 of 0.488, and mAP@0.5:0.95 of 0.299 on the validation dataset.

Metric	Result
Precision	0.485
Recall	0.508
mAP@0.5	0.488
mAP@0.5:0.95	0.299

The per-class performance results are shown below.

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
obstacle_on_walkway	0.197	0.149	0.122	0.041
construction_debris	0.479	0.578	0.538	0.410
fallen_branch	0.391	0.419	0.371	0.158
traffic_cone	0.860	0.975	0.970	0.660
wet_slippery_floor	0.500	0.417	0.439	0.222

Among the five classes, **traffic_cone** achieved the strongest performance, with an mAP@0.5 of 0.970. This is likely because traffic cones have a clear and consistent shape, colour, and visual appearance.

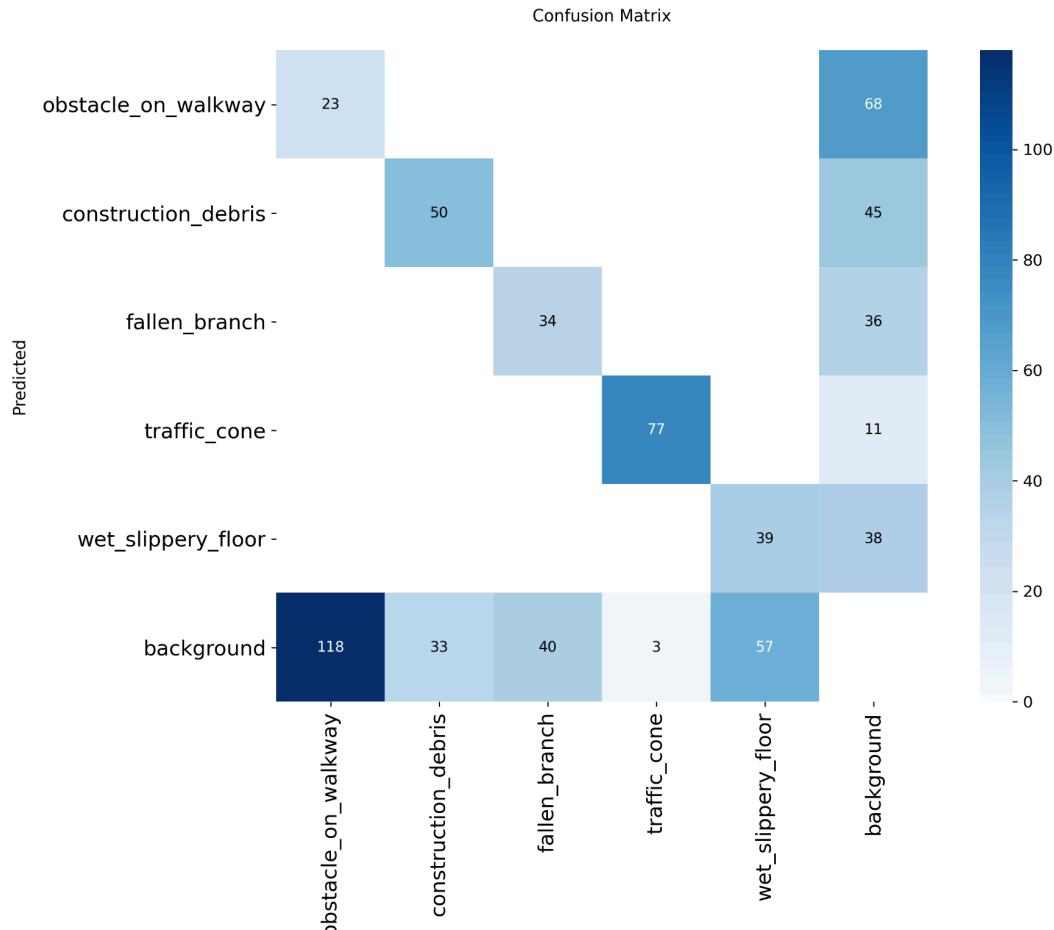
In contrast, **obstacle_on_walkway** achieved the lowest mAP@0.5 of 0.122. This class included a wide variety of objects, backgrounds, sizes, and viewpoints. Therefore, the model had difficulty learning one consistent visual pattern for this general class.

The results represent an initial baseline model. Further improvement can be achieved by training for more epochs, collecting more consistent obstacle images, improving class balance, and using GPU acceleration for higher-resolution training.

5.2.5: Confusion Matrix and Error Analysis

Confusion Matrix and Error Analysis

Figure 3: Confusion matrix of Member 2 YOLOv8n model



The confusion matrix was used to analyse how well the Member 2 model classified the five walkway obstruction hazard classes. The results show that the model performed best on **traffic_cone**, while lower performance was observed for **obstacle_on_walkway**, **fallen_branch**, and **wet_slippery_floor**.

The **traffic_cone** class achieved the strongest detection performance because traffic cones have a relatively consistent visual appearance, including a clear cone shape and bright colour. As a result, the model was able to distinguish this class more effectively from the other hazards.

The **obstacle_on_walkway** class showed the weakest performance. This class included many different types of obstacles, such as bags, boxes, bicycles, rubbish, and other objects placed on pedestrian routes. Because these objects have different shapes, colours, sizes, and backgrounds, the model had difficulty learning one consistent visual pattern. Many obstacle instances were missed or classified as background.

Some confusion also occurred between **construction_debris** and **fallen_branch**. Both classes may appear as irregular objects on outdoor ground surfaces, especially when images contain soil, leaves, broken materials, or construction areas. In addition,

wet_slippery_floor may be difficult to detect in images with reflections, shadows, dark surfaces, or low lighting conditions.

Another limitation is that the model was trained using public datasets collected from different environments. Therefore, the image backgrounds, camera angles, lighting conditions, pavement types, and object appearances may not fully match the actual campus environment. This may affect the model's ability to generalise to real campus walkways.

Future improvement should include collecting more real campus images, especially for **obstacle_on_walkway** and **wet_slippery_floor**. The model can also be improved by using more training epochs, better dataset balancing, higher image resolution, and GPU-based training.

5.2.6: Prediction Testing

Prediction Testing

After training, the **best.pt** model was used to perform prediction testing on validation images. The prediction results show that the model was able to draw bounding boxes around detected hazards and display the predicted class name together with a confidence score. This confirms that the trained YOLOv8n model can perform object detection inference and generate visual outputs for later mobile app integration.

Figure 4: Validation prediction examples generated by Member 2 YOLOv8n model

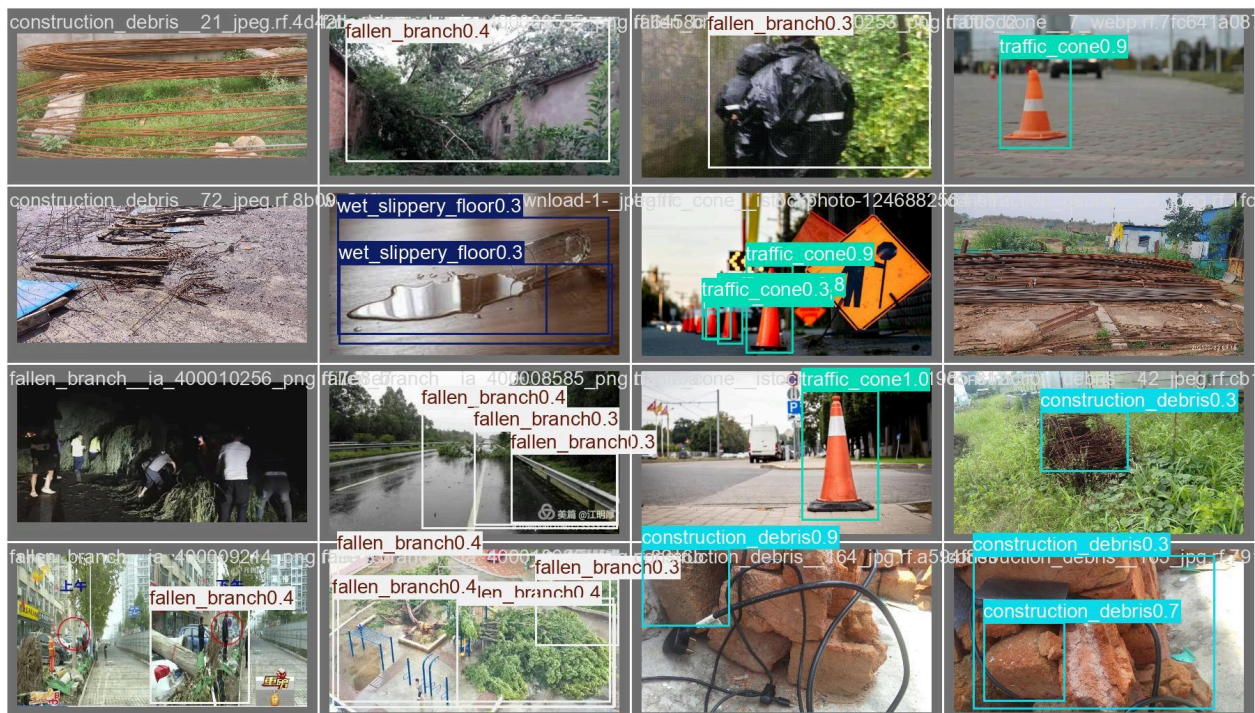


Figure 4 shows prediction examples generated from validation images. The model successfully detected several `traffic_cone`, `construction_debris`, `fallen_branch`, and `wet_slippery_floor` instances. The bounding boxes and predicted labels indicate that the model can identify visible hazard objects and surface conditions in different outdoor and walkway-related environments.

The prediction examples also show that some detections had lower confidence or missed certain objects. This was especially observed for `obstacle_on_walkway`, because this class contains many visually different objects and does not have one consistent shape or appearance. In some images, the model also detected only part of a hazard object or produced bounding boxes that were not perfectly aligned.

The prediction results are consistent with the evaluation metrics. `traffic_cone` was detected with high confidence because of its distinctive shape and colour. `construction_debris` and `fallen_branch` showed moderate performance, while `obstacle_on_walkway` remained more difficult to detect accurately.

Although the model is still an initial baseline model, the prediction testing confirms that it can produce real-time style detection outputs. The trained `best.pt` model can later be integrated into the mobile application and combined with the outputs from the other member YOLO models through the meta-classifier.

5.2.7: Challenges and Limitations

Several challenges were encountered during the development of the Member 2 YOLO model.

First, suitable public datasets were difficult to find for some walkway obstruction hazard classes. The original class `low_hanging_object` was replaced with `traffic_cone` because traffic cone datasets were more available and easier to annotate consistently. This adjustment maintained the walkway safety focus while allowing the model to be trained using sufficient labelled data.

Second, the five public datasets used different class names, label IDs, image sizes, and train-validation-test splits. Therefore, the datasets had to be reorganised into one unified five-class structure. All original labels were remapped into the final classes: `obstacle_on_walkway`, `construction_debris`, `fallen_branch`, `traffic_cone`, and `wet_slippery_floor`.

Third, the original merged dataset contained a large number of training images. Training the full dataset using CPU at an image size of 640×640 required a very long processing time. Therefore, a smaller dataset was created for the initial training experiment, and the image size was reduced to 416×416 . This reduced training time but may also have limited detection performance for small or unclear hazard objects.

Another limitation was the broad definition of `obstacle_on_walkway`. This class included many visually different objects, such as bags, boxes, rubbish, bicycles, and other items. Because these objects do not share a consistent shape or appearance, the model had difficulty detecting them accurately. This caused lower mAP performance compared with more visually consistent classes such as `traffic_cone`.

Finally, the datasets were mainly collected from public sources rather than real campus locations. Differences in lighting, weather, walkway materials, object backgrounds, and camera angles may reduce the model's performance when used in an actual campus environment.

Future improvements should include collecting more campus-specific images, refining the `obstacle_on_walkway` class into more specific categories, increasing training epochs, using GPU acceleration, and training with a larger balanced dataset.

5.2.8: Conclusion for Member 2

In conclusion, Member 2 successfully developed an initial YOLOv8n model for detecting five walkway obstruction hazard classes: `obstacle_on_walkway`, `construction_debris`, `fallen_branch`, `traffic_cone`, and `wet_slippery_floor`.

The dataset was created by combining multiple public object detection datasets and remapping the original labels into a unified five-class format. A reduced dataset containing approximately 1,000 training images and 246 validation images was used for the initial training experiment.

The model was trained using YOLOv8n for 5 epochs with an image size of 416×416 and batch size 8. The model achieved an overall precision of 0.485, recall of 0.508, mAP@0.5 of 0.488, and mAP@0.5:0.95 of 0.299.

The model performed especially well for the `traffic_cone` class because of its distinctive and consistent visual features. However, lower performance was observed for `obstacle_on_walkway` because the class contained many visually different objects. Further improvements can be made through additional training epochs, a larger balanced dataset, more real campus images, clearer class definitions, higher image resolution, and GPU-based training.

The trained `best.pt` model can later be integrated into the final campus hazard detection mobile application. Its detection outputs can also be combined with the other members' YOLO models through the neural-network meta-classifier to produce the final hazard class, confidence score, severity level, and maintenance recommendation.

5.3 Member 3: Drainage and Ground Surface Hazard Detection Model

5.3.1: Member 3 Model Overview

Member 3 was responsible for developing a YOLOv8 object detection model focusing on drainage and ground surface hazards. The purpose of this model is to identify hazards that may create tripping, falling, drainage, or pedestrian safety risks in campus roads, walkways, parking areas, and outdoor public facilities.

The selected hazard classes were open_drain, uncovered_manhole, pothole, uneven_pavement, and cracked_pavement.

These classes were selected because they represent common ground-level hazards that may affect the safety of students, staff, and visitors. Open drains and uncovered manholes may cause severe falling accidents. Potholes may create vehicle damage and tripping risks. Uneven pavement and cracked pavement may affect pedestrian movement and can develop into more serious surface defects if not repaired.

Therefore, this model contributes to the overall campus hazard detection system by identifying drainage-related and ground-surface-related hazards that require maintenance attention.

Item	Description
Member	Wang Qihua
Model Focus	Drainage and Ground Surface Hazards
YOLO Version	YOLOv8n
Number of Classes	5
Classes	open_drain, uncovered_manhole, pothole, uneven_pavement, cracked_pavement

Final Dataset Images	1,922
Epochs	50
Image Size	640 × 640
Batch Size	16
Training Device	Tesla T4 GPU via Google Colab
Model Output	best.pt

5.3.2: Dataset Preparation

The dataset for Member 3 was prepared by combining five public YOLO-format object detection datasets related to drainage hazards, manholes, potholes, rigid pavement damage, and pavement cracks.

Each original dataset was first checked to ensure that it contained image files, YOLO annotation labels, and a data configuration file. Since the downloaded datasets used different class names and class IDs, the labels were reviewed, filtered, and remapped into one unified five-class structure.

Only hazard-related labels were retained. Non-target labels such as closed manholes, fake manholes, normal pavement, and unrelated road features were excluded. This was necessary to ensure that the final model focused only on the selected campus hazard classes.

The final class mapping used for Member 3 was as follows:

Class ID	Final Class Name
----------	------------------

0	open_drain
1	uncovered_manhole
2	pothole
3	uneven_pavement
4	cracked_pavement

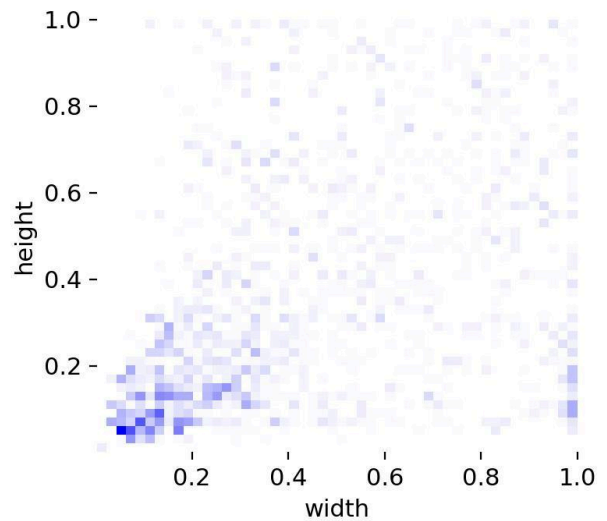
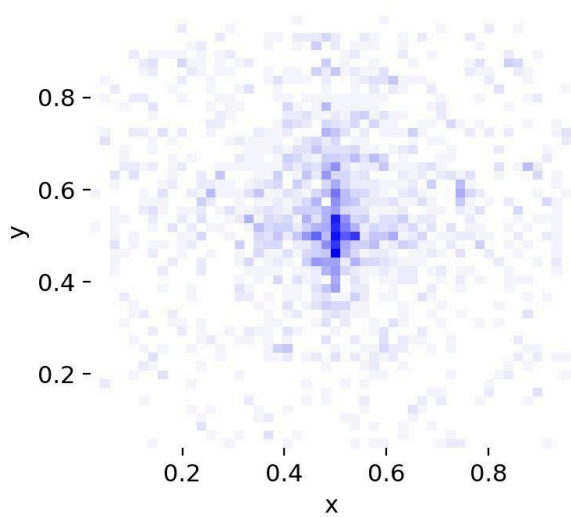
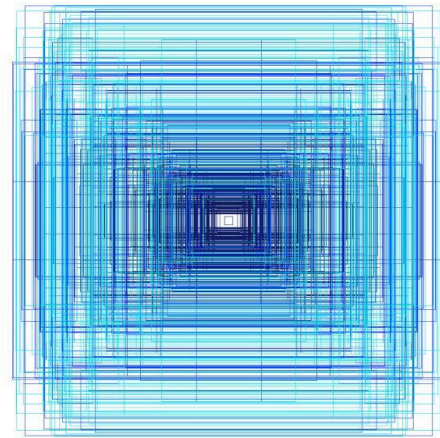
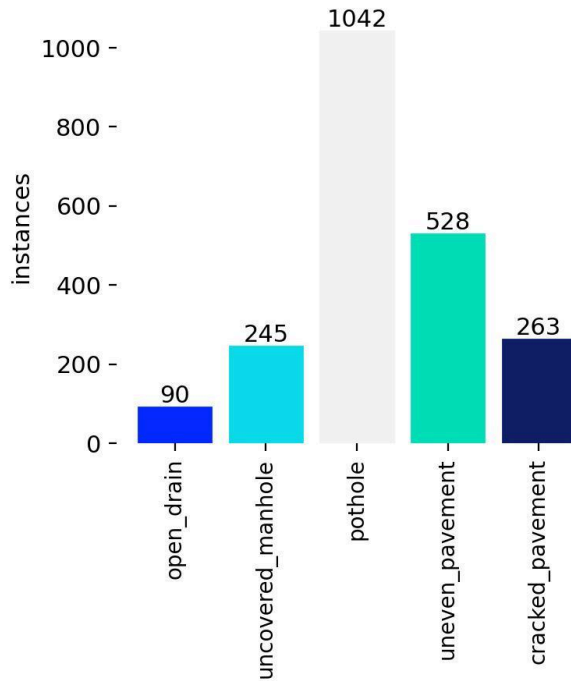
The final merged dataset contained 1,922 images with corresponding YOLO label files. The images and labels were organised into separate training, validation, and test folders.

Dataset Sources and Class Definition

Final Class	Source Dataset Type	Original Labels Used
open_drain	Drainage overflow dataset	drainage overflow, open drainage
uncovered_manhole	Manhole dataset	improperly closed manhole, open manhole
pothole	Pothole dataset	pothole
uneven_pavement	Rigid pavement dataset	hard, medium, low pavement damage

cracked_pavement	Damaged sidewalk dataset	crack
------------------	--------------------------	-------

Class Name	Description	Safety Risk
open_drain	An exposed drainage channel or drainage opening	May cause falling, slipping, or injury
uncovered_manhole	An open or improperly closed manhole	May cause serious falling accidents
pothole	A hole or depression on a road or ground surface	May cause tripping, falling, or vehicle damage
uneven_pavement	Uneven, damaged, or repaired pavement surface	May cause pedestrian instability or tripping
cracked_pavement	Visible cracks on pavement or walkway surfaces	May develop into larger surface damage



5.3.3: Training Configuration

The YOLOv8n model was selected because it is a lightweight object detection model suitable for faster training and possible future mobile or real-time deployment.

The model was trained using the final merged five-class dataset. Training was conducted using a Tesla T4 GPU in Google Colab. GPU acceleration was used to reduce training time and allow a higher image size of 640×640 .

The best-performing model was saved as best.pt, while the final epoch model was saved as last.pt. The best model can later be used for prediction testing, mobile application integration, and meta-classifier processing with the models trained by other group members.

Parameter	Value
YOLO Version	YOLOv8n
Model File	yolov8n.pt
Epochs	50
Image Size	640 × 640
Batch Size	16
Device	Tesla T4 GPU
Framework	Ultralytics YOLOv8
Number of Classes	5
Training Platform	Google Colab
Output Folder	member3_yolov8n_v1-2
Best Model	weights/best.pt

Training Iterations and Improvement Attempts

Several preparation and training steps were completed before producing the final Member 3 model.

First, five public object detection datasets were downloaded and merged into one unified YOLO dataset. During this process, original class IDs were remapped into the final five-class structure.

Second, non-target labels were removed. For example, closed manholes and fake manholes were excluded because they do not represent the selected campus hazards. This helped ensure that the final model learned relevant hazard categories only.

Third, the model was initially prepared for CPU training. However, GPU acceleration was later enabled through Google Colab, allowing the model to be trained using a Tesla T4 GPU with an image size of 640×640 and batch size 16.

Training Iteration	Purpose	Action	Outcome
Initial dataset merge	Combine five datasets	Unified class IDs from 0 to 4	Created final_dataset
Label filtering	Remove irrelevant labels	Retained target hazard annotations only	Reduced non-target samples
GPU training setup	Improve training efficiency	Used Tesla T4 GPU in Google Colab	Enabled faster model training
Final training	Train final model	YOLOv8n, 50 epochs, image size 640, batch size 16	Generated best.pt and evaluation results

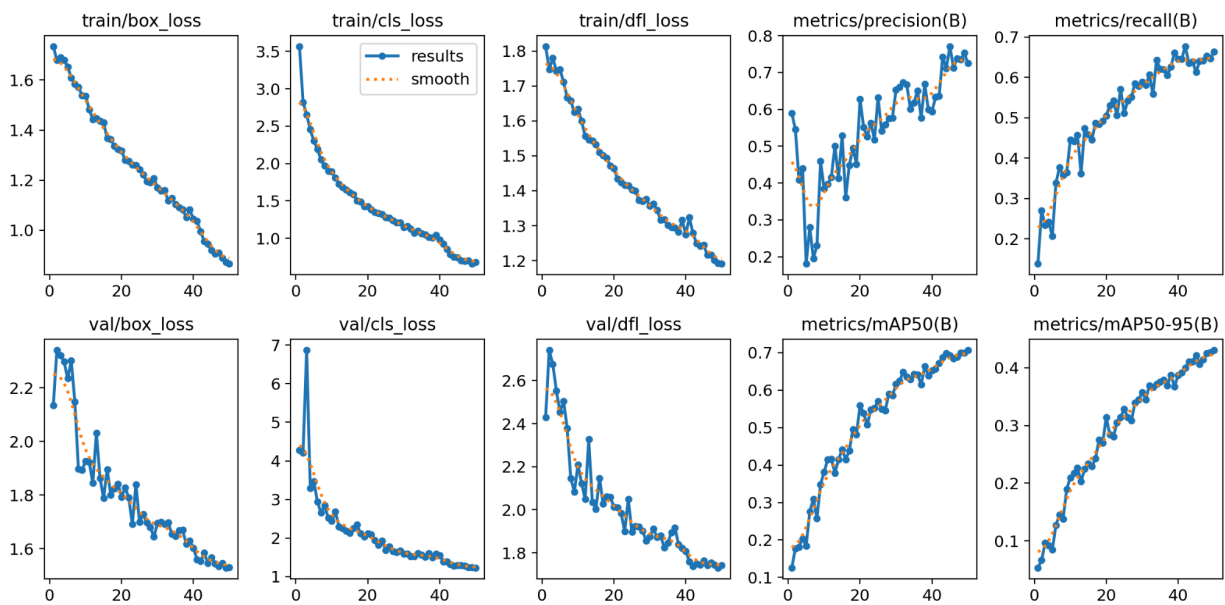
5.3.4: Training Results

Figure 1 shows the training performance curves generated by the Member 3 YOLOv8n model. The curves include training loss, validation loss, precision, recall, mAP@0.5, and mAP@0.5:0.95.

The training and validation losses generally decreased during the training process, showing that the model learned useful visual patterns from the drainage and ground-surface hazard dataset. Precision, recall, and mAP values improved as training progressed.

After 50 epochs of training, the model achieved the following overall validation performance:

Metric	Result
Precision	[Insert Precision]
Recall	[Insert Recall]
mAP@0.5	[Insert mAP@0.5]
mAP@0.5:0.95	[Insert mAP@0.5:0.95]



5.3.5: Confusion Matrix and Error Analysis

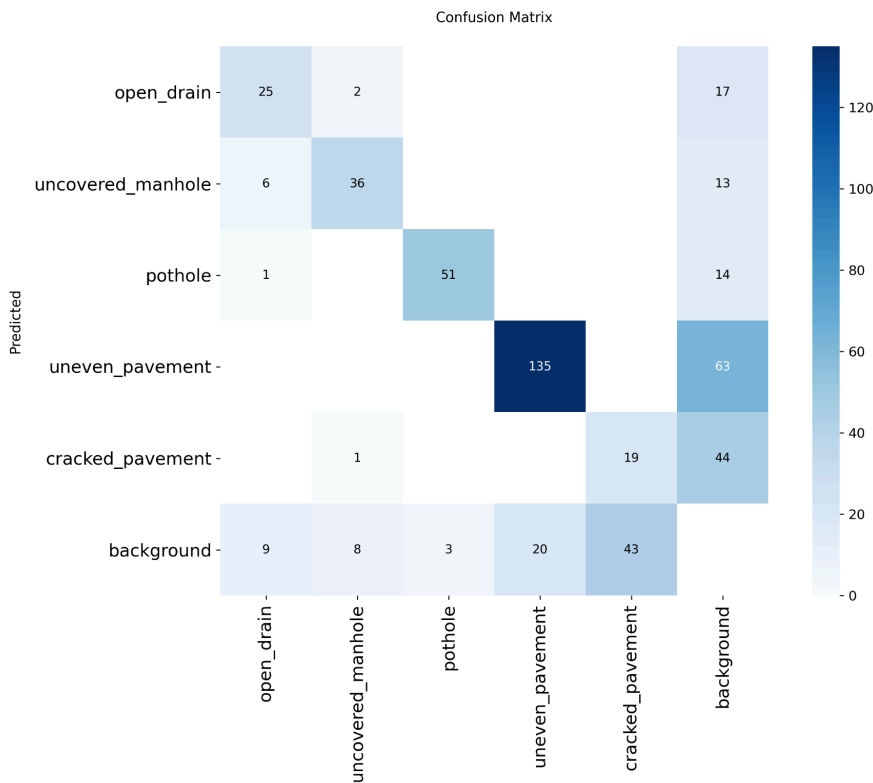
Figure 2 presents the confusion matrix of the Member 3 YOLOv8n model.

The confusion matrix was used to analyse how well the model distinguished between the five drainage and ground surface hazard classes. The model showed clear detection capability for pothole and uncovered_manhole examples during prediction testing.

However, some confusion may occur between visually similar surface-related classes, especially pothole, uneven_pavement, and cracked_pavement. These classes may share similar visual patterns such as broken asphalt, damaged concrete, surface cracks, irregular pavement texture, and depressed ground areas.

For example, a severely damaged pavement surface may contain both cracks and pothole-like areas. Similarly, uneven pavement may appear visually similar to cracked pavement when the camera angle or lighting condition makes the height difference unclear.

Another limitation is that the training datasets were collected from public sources. Therefore, the pavement material, drainage structure, background, weather condition, camera angle, and lighting may not fully match the actual campus environment.



5.3.6: Prediction Testing

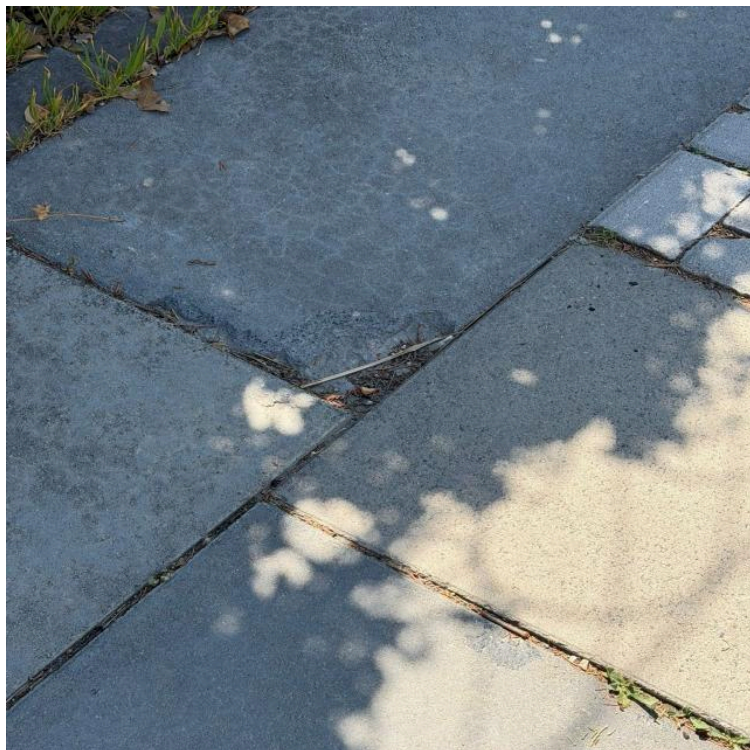
After training, the best.pt model was used to perform prediction testing on unseen test images. The prediction results show that the model can draw bounding boxes around detected hazards and display the predicted class name together with a confidence score.

The model successfully detected an uncovered manhole with a confidence score of 0.88. The bounding box was placed accurately around the manhole cover area. This confirms that the model can recognise manhole-related hazards from test images.

The model also detected multiple potholes in heavily damaged road images. Several pothole detections had confidence scores above 0.80, showing that the model was able to identify multiple separate pothole areas in one image.

However, one damaged walkway image produced no detection. This suggests that the `uneven_pavement` and `cracked_pavement` classes may require more diverse training examples and clearer annotation rules.





uncovered_manhole 0.88



5.3.7: Challenges and Limitations

Several challenges were encountered during the development of the Member 3 YOLO model.

First, suitable public datasets were difficult to find for narrow hazard categories such as open drains and uneven pavement. Some public datasets contained related labels, but their class

definitions were not fully consistent with the project requirements. Therefore, the original labels had to be reviewed and remapped carefully.

Second, the five downloaded datasets used different class names, class IDs, image sizes, and data splits. Therefore, a Python script was created to filter labels, remap original class IDs, and merge the datasets into one unified YOLO-format dataset.

Third, several classes were visually similar. Pothole, uneven pavement, and cracked pavement may all appear as damaged ground surfaces. This can cause classification confusion, especially when images contain multiple types of surface defects.

Fourth, the dataset was mainly collected from public sources rather than actual campus locations. Differences in weather, pavement materials, drain designs, lighting conditions, and camera angles may reduce the model's ability to generalise to real campus environments.

Future improvement should include collecting more campus-specific images, increasing examples for uneven pavement and open drain hazards, improving class balance, refining class definitions, and conducting additional training experiments with augmentation.

5.3.8: Conclusion for Member 3

In conclusion, Member 3 successfully developed a YOLOv8n model for detecting five drainage and ground surface hazard classes: `open_drain`, `uncovered_manhole`, `pothole`, `uneven_pavement`, and `cracked_pavement`.

The final dataset was created by combining multiple public object detection datasets and remapping original labels into a unified five-class structure. After label filtering and dataset merging, 1,922 images were retained for training, validation, and testing.

The model was trained using YOLOv8n for 50 epochs with an image size of 640×640 and batch size 16 using a Tesla T4 GPU in Google Colab. The trained `best.pt` model can identify important drainage and ground-surface hazards, including potholes and uncovered manholes.

Although the model produced useful detection results, further improvement is needed for visually similar surface classes such as `uneven_pavement` and `cracked_pavement`. Future work should include more real campus images, better dataset balancing, more precise annotation guidelines, and additional model training iterations.

The trained `best.pt` model will be used as Member 3's YOLO model output and can later be integrated with the group's mobile application and meta-classifier system.

5.4 Member 4: Access, Boundary and Safety Facility Hazard Detection Model

5.4.1: Member 4 Model Overview

Member 4 was responsible for developing a YOLO object detection model focusing on access, boundary, and safety facility hazards on campus. The purpose of this model is to identify hazards that may affect safe movement, access control, boundary protection, and the visibility or reliability of campus safety facilities.

The selected hazard classes were **missing_barricade**, **damaged_warning_sign**, **broken_handrail**, **blocked_walkway**, and **damaged_sidewalk**.

These classes were selected because they are related to common safety and maintenance issues in campus environments. Missing barricades may expose students, staff, or visitors to unsafe or restricted areas. Damaged warning signs may reduce safety awareness because important hazard information may become unclear or unreliable. Broken handrails may increase the risk of falling, especially on staircases, ramps, and elevated walkways. Blocked walkways may interrupt pedestrian movement and reduce safe access. Damaged sidewalks may cause tripping or falling accidents.

The class **damaged_sidewalk** overlaps with Member 1's Walkway Surface Hazard Detection Model. This overlap was intentionally included to support the ensemble learning and meta-classifier design. During final inference, predictions from both models can be compared using confidence scores, bounding-box IoU agreement, model reliability, and location context before determining the final hazard result.

Therefore, Member 4's model contributes to the overall campus hazard detection system by identifying access-related, boundary-related, and safety facility hazards that may require maintenance attention.

Item	Description
Member	He Jun
Model Focus	Access, Boundary and Safety Facility Hazards

YOLO Version	YOLOv8n
Number of Classes	5
Classes	missing_barricade, damaged_warning_sign, broken_handrail, blocked_walkway, damaged_sidewalk
Training Images	320
Validation Images	80
Total Images	400
Epochs	50
Image Size	640 × 640
Batch Size	16
Training Device	Tesla T4 GPU via Google Colab
Optimizer	AdamW
Model Output	best.pt

5.4.2: Dataset Preparation

The dataset for Member 4 was prepared by collecting and combining public object detection datasets related to barriers, damaged traffic signs, handrails, walkway obstacles, and damaged sidewalks. Since the original datasets came from different sources, their original labels were reviewed and remapped into one unified five-class structure.

The final class mapping used for Member 4 was as follows: **missing_barricade** was assigned as class 0, **damaged_warning_sign** as class 1, **broken_handrail** as class 2, **blocked_walkway** as class 3, and **damaged_sidewalk** as class 4.

Some original labels did not exactly match the final class names. Therefore, label harmonisation was required. For example, the original label **Road Barriers** was mapped into **missing_barricade** because missing barricade is difficult to detect directly as an absence. The model instead learns barrier-related and boundary-protection visual patterns. For damaged warning signs, several original damaged sign labels such as deformation, dirty, graffiti, knocked, occluded, and perforation were mapped into **damaged_warning_sign**, while the normal label **ok** was excluded. The handrail dataset originally used the label **handrail**, which was mapped into **broken_handrail** because public broken handrail datasets were limited. Obstacle-related labels such as obstacles, stalls, and worksite materials were mapped into **blocked_walkway**. Finally, crack and damaged-sidewalk labels were mapped into **damaged_sidewalk**.

After merging the original datasets, the dataset was balanced to reduce class imbalance. Each class contained 80 images, with 64 images used for training and 16 images used for validation. As a result, the final balanced dataset contained 320 training images and 80 validation images, giving a total of 400 images.

The final dataset was saved in YOLO format with separate image and label folders for training and validation. This structure allowed the YOLOv8n model to correctly read the images and corresponding bounding-box annotations during training.

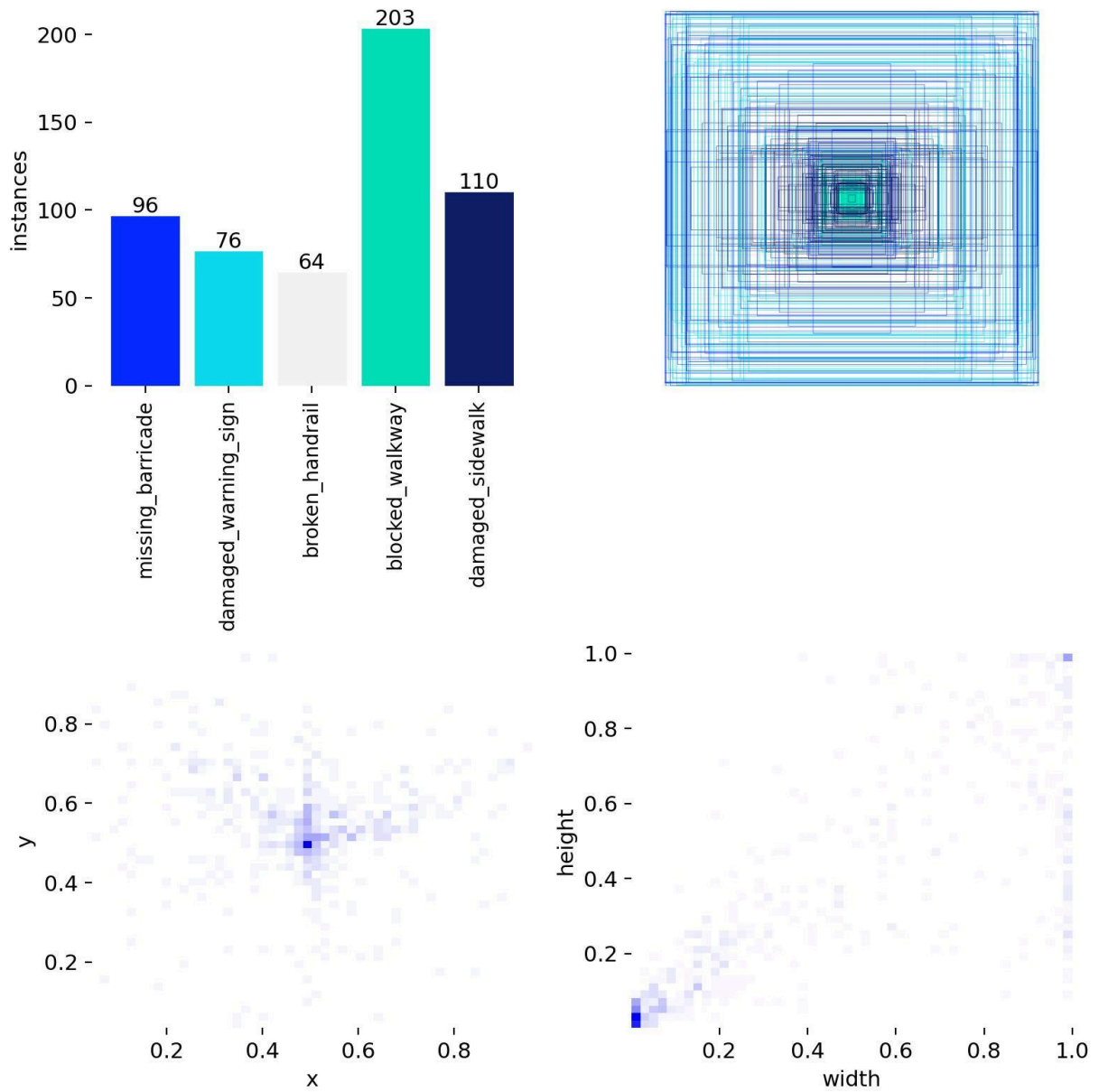


Figure 1: Label distribution and annotation overview for Member 4 dataset

Dataset Class Mapping

Class ID	Class Name	Description	Safety Risk
----------	------------	-------------	-------------

0	missing_barricade	Unsafe or restricted area represented by road barrier or barricade-related visual patterns	May allow people to enter unsafe areas without proper boundary protection
1	damaged_warning_sign	Warning sign that is damaged, dirty, unclear, blocked, deformed, or visually unreliable	May reduce hazard awareness and safety warning effectiveness
2	broken_handrail	Handrail or railing facility that may represent damaged, loose, unsafe, or missing handrail conditions	May increase falling risk on stairs, ramps, or elevated walkways
3	blocked_walkway	Objects, stalls, worksite materials, or obstacles located on access routes or walkways	May interrupt pedestrian movement or emergency access
4	damaged_sidewalk	Cracked or damaged sidewalk surface	May cause tripping, falling, or walking difficulty

Dataset Distribution

Class Name	Training Images	Validation Images	Total Images
missing_barricade	64	16	80
damaged_warning_sign	64	16	80

broken_handrail	64	16	80
blocked_walkway	64	16	80
damaged_sidewalk	64	16	80
Total	320	80	400

Global Label Mapping for Member 4

Member	Model Focus	Class ID	Original Label	Global Label	Relationship Type	Related Model/Class	Handling Rule
Member 4	Access, Boundary and Safety Facility Hazards	0	Road Barriers	missing_barricade	Contextual / Generalization	Boundary and access hazard	Road barrier images were mapped to missing_barricade because missing barricade is difficult to detect directly as an absence. The class represents unsafe or restricted areas that require proper barricade protection.

Member 4	Access, Boundary and Safety Facility Hazards	1	deformation, dirty, graffiti, knocked, occluded, perforation, damaged sign labels	damaged_warning_sign	Synonym / Similar meaning	Safety warning facility hazard	Different damaged traffic sign conditions were normalised into damaged_warning_sign because they all represent warning signs that are damaged, unclear, blocked, or visually unreliable. The normal class "ok" was excluded.
Member 4	Access, Boundary and Safety Facility Hazards	2	handrail	broken_handrail	Generalisation	Safety facility hazard	Public handrail images were mapped to broken_handrail because broken handrail datasets were limited. The class is used to represent unsafe, damaged, loose, or missing handrail conditions in campus areas.

Member 4	Access, Boundary and Safety Facility Hazards	3	obstacles, stalls, worksite materials	blocked_walkway	Contextual overlap / Generalisation	Similar to Member 2 obstacle_on_walkway	Obstacle-related labels were mapped to blocked_walkway because they may block pedestrian movement or reduce safe access. During meta-classification, this class can be compared with Member 2's obstruction-related classes.
Member 4	Access, Boundary and Safety Facility Hazards	4	crack, damaged-sidewalk	damaged_side walk	Exact overlap / Similar meaning	Member 1 damaged_side walk / walkway surface hazards	Crack and damaged-side walk labels were mapped into damaged_side walk because both represent visible sidewalk surface defects. This class overlaps with Member 1 and can be resolved using confidence score, IoU agreement, and model reliability.

5.4.3: Training Configuration

The YOLOv8n model was selected because it is a lightweight object detection model suitable for faster training and potential real-time or mobile application deployment. The model was trained using the balanced Member 4 dataset containing five access, boundary, and safety facility hazard classes.

Training was conducted using Google Colab with a Tesla T4 GPU. GPU acceleration was used to reduce training time and allow training with an image size of 640×640 . The model was trained for 50 epochs using a batch size of 16 and the AdamW optimizer.

The training output was saved in the project directory under the folder **member4_access_safety_hazards_v1**. The best-performing model was saved as **best.pt**, while the final epoch model was saved as **last.pt**. These model files can later be used for prediction testing, mobile application integration, and ensemble processing with the models trained by other group members.

Parameter	Value
YOLO Version	YOLOv8n
Model File	yolov8n.pt
Epochs	50
Image Size	640×640
Batch Size	16
Device	Tesla T4 GPU

Training Platform	Google Colab
Optimizer	AdamW
Training Dataset	Member 4 balanced dataset
Training Images	320
Validation Images	80
Output Folder	member4_access_safety_hazards_v1
Best Model	weights/best.pt
Training Time	0.156 hours

Training Iterations and Improvement Attempts

Several preparation and training steps were completed before producing the final Member 4 model. First, public datasets related to barriers, damaged traffic signs, handrails, walkway obstacles, and damaged sidewalks were downloaded and reviewed. Second, the original labels were remapped into the final five-class structure. Third, labels that were not suitable for hazard detection were removed. For example, the normal warning sign label **ok** and the non-barrier label **no** were excluded.

After merging the datasets, a balanced dataset was created to reduce class imbalance. This was necessary because the original datasets had very different numbers of images. For example, the damaged warning sign dataset contained many more images than the missing barricade and broken handrail datasets. Therefore, each class was limited to 80 images, with 64 training images and 16 validation images per class.

The final training iteration used YOLOv8n with 50 epochs, image size 640 × 640, and batch size 16. The training was performed using a Tesla T4 GPU in Google Colab. The final training output produced the best model file, last model file, training curves, confusion matrix, prediction examples, and performance results.

Training Iteration	Purpose	Action	Outcome
Initial dataset collection	Collect hazard-related data	Downloaded public datasets related to barriers, signs, handrails, obstacles, and sidewalks	Collected raw datasets for five classes
Label remapping	Create unified class structure	Remapped original labels into class IDs 0 to 4	Created final five-class label structure
Label filtering	Remove unsuitable labels	Excluded normal or unrelated labels such as ok and no	Reduced irrelevant training samples
Dataset balancing	Reduce class imbalance	Selected 64 training and 16 validation images per class	Created balanced dataset with 400 images
Final training	Train Member 4 YOLO model	YOLOv8n, 50 epochs, image size 640, batch size 16	Generated best.pt and evaluation results

5.4.4: Training Results

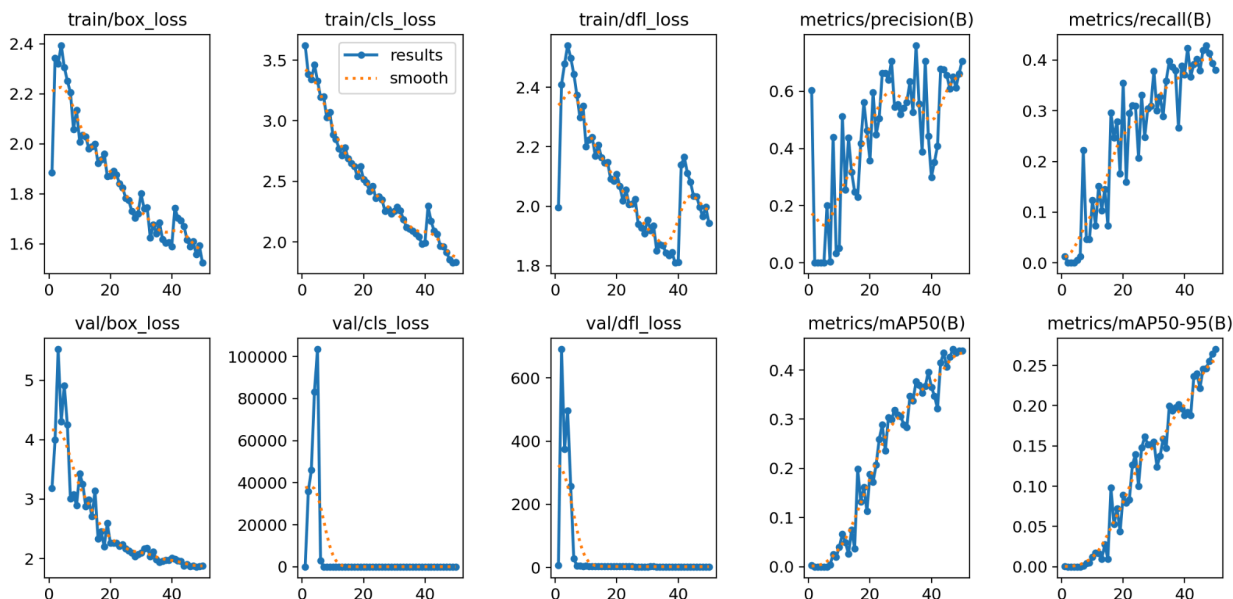


Figure 2: Training performance curves for Member 4 YOLOv8n model

After 50 epochs of training, the Member 4 YOLOv8n model achieved an overall precision of **0.706**, recall of **0.380**, mAP@0.5 of **0.439**, and mAP@0.5:0.95 of **0.270** on the validation dataset.

These results show that the model was able to learn useful visual patterns for some access, boundary, and safety facility hazards. The strongest performance was achieved by the **missing_barricade** class, with a mAP@0.5 of **0.885** and mAP@0.5:0.95 of **0.576**. This indicates that the model was able to recognise barrier-related visual patterns effectively.

The **damaged_warning_sign** and **broken_handrail** classes also showed moderate detection performance. However, the **blocked_walkway** and **damaged_sidewalk** classes achieved lower performance. This suggests that these classes were more difficult for the model to distinguish due to high visual variation, contextual dependency, and similarity with other ground or obstruction-related hazards.

Metric	Result
Precision	0.706

Recall	0.380
mAP@0.5	0.439
mAP@0.5:0.95	0.270

Per-class Performance Results

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
missing_barricade	0.949	0.818	0.885	0.576
damaged_warning_sign	0.808	0.625	0.651	0.369
broken_handrail	0.646	0.438	0.573	0.373
blocked_walkway	0.125	0.0196	0.0591	0.0249
damaged_sidewalk	1.000	0.000	0.0255	0.00789

5.4.5: Confusion Matrix and Error Analysis

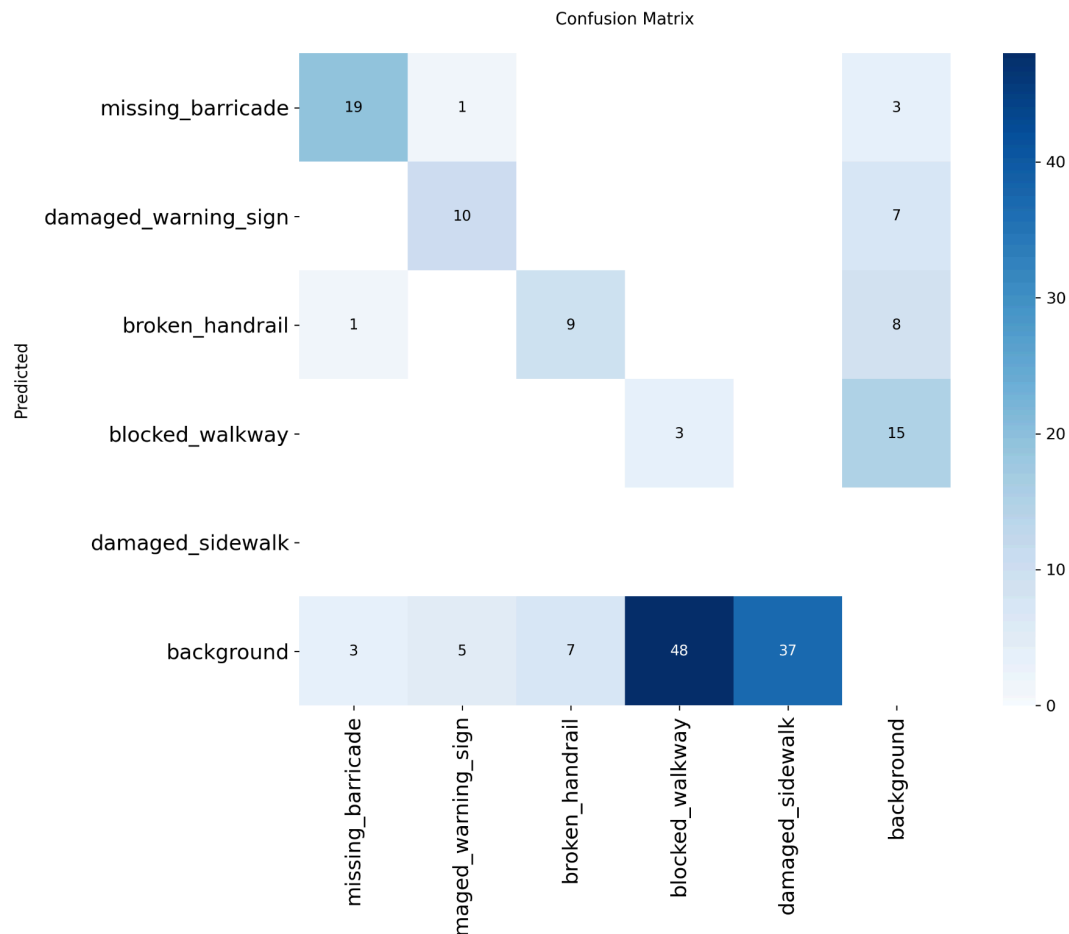


Figure 3: Confusion matrix of Member 4 YOLOv8n model

The confusion matrix was used to analyse how well the Member 4 YOLOv8n model classified the five access, boundary, and safety facility hazard classes. The model performed better on **missing_barricade**, **damaged_warning_sign**, and **broken_handrail** compared with **blocked_walkway** and **damaged_sidewalk**.

The **missing_barricade** class achieved the strongest performance. One possible reason is that road barriers and barricade-related objects have relatively clear shapes and colours, making them easier for the model to detect. Similarly, damaged warning signs and handrails have more structured visual appearances compared with highly variable obstruction and surface damage classes.

The **blocked_walkway** class showed weak performance. This class is highly context-dependent because an object becomes a blocked walkway hazard only when it is located on a pedestrian or access route. The objects in this class may have many different

shapes, sizes, colours, and materials, such as obstacles, stalls, and worksite materials. As a result, the model had difficulty learning one consistent visual pattern.

The **damaged_sidewalk** class also showed low recall and low mAP. This may be because damaged sidewalk images are visually similar to other surface defects such as cracks, broken pavement, and uneven ground. In addition, sidewalk damage may appear differently depending on pavement material, lighting condition, camera angle, and background. Some damaged sidewalk regions may also be small or unclear, making detection more difficult.

Another limitation is that the dataset was mainly collected from public sources rather than real campus environments. Therefore, the model may not fully match actual campus conditions such as local walkway materials, safety sign designs, handrail structures, lighting, and background objects. Future improvement should include more real campus images, clearer annotation rules, and additional training examples for difficult classes.

5.4.6: Prediction Testing



Figure 4: Validation prediction examples generated by Member 4 YOLOv8n model

After training, the best model file **best.pt** was used to perform prediction testing on validation images. The prediction results show that the model was able to draw bounding boxes around detected hazards and display the predicted class name together with a confidence score. This confirms that the trained YOLOv8n model can perform object detection inference and generate visual outputs for later mobile application integration.

The prediction examples included the five Member 4 hazard classes: `missing_barricade`, `damaged_warning_sign`, `broken_handrail`, `blocked_walkway`, and `damaged_sidewalk`. The model was able to detect several visible access and safety facility hazards, especially barrier-related objects and warning signs. However, some predictions had lower confidence or were missed, especially for `blocked_walkway` and `damaged_sidewalk`.

The prediction testing results are consistent with the evaluation metrics. Classes with clearer and more consistent visual patterns, such as `missing_barricade` and `damaged_warning_sign`, were detected more effectively. In contrast, `blocked_walkway` and `damaged_sidewalk` remained more difficult due to high visual variation and contextual dependency.

The trained **best.pt** model can later be integrated with the final campus hazard detection mobile application. Its detection outputs can also be combined with the other members' YOLO models through the neural-network meta-classifier to produce the final hazard class, confidence score, severity level, and recommended maintenance action.

5.4.7: Challenges and Limitations

Several challenges were encountered during the development of the Member 4 YOLO model.

First, suitable datasets for some access and safety facility hazard classes were difficult to find. For example, **`missing_barricade`** is difficult because it represents the absence of a safety facility. Since object detection models are better at detecting visible objects, road barrier images were used to represent barricade-related and boundary-protection hazards. This is a practical limitation of using YOLO for absence-based hazard detection.

Second, **`broken_handrail`** datasets were limited. Most public datasets only contained normal handrail or railing objects. Therefore, handrail images were mapped into the `broken_handrail` class to represent unsafe, damaged, loose, or missing handrail conditions. Future improvement should include collecting more real damaged handrail images from campus areas.

Third, **`blocked_walkway`** was difficult because it is a contextual class. The model needs to understand not only the object, but also whether the object blocks a pedestrian path. Since the dataset contained different object types such as obstacles, stalls, and worksite materials, the class had large visual variation and achieved low performance.

Fourth, **`damaged_sidewalk`** was difficult because it shares visual similarity with other ground surface hazards such as cracks, damaged pavement, and uneven floor. The model may confuse these categories or miss small damaged regions.

Finally, the datasets were mainly collected from public sources rather than actual campus locations. Differences in lighting, camera angle, walkway materials, warning sign designs, handrail structures, and background environments may reduce the model's generalisation to real campus conditions.

Future improvements should include collecting more campus-specific images, increasing the number of examples for difficult classes, using clearer annotation guidelines, improving class definitions, increasing dataset diversity, and conducting additional training experiments.

5.4.8: Conclusion for Member 4

In conclusion, Member 4 successfully developed a YOLOv8n model for detecting five access, boundary, and safety facility hazard classes: **missing_barricade**, **damaged_warning_sign**, **broken_handrail**, **blocked_walkway**, and **damaged_sidewalk**.

The final balanced dataset contained 400 images, with 320 images for training and 80 images for validation. Each class contained 80 images, which helped reduce class imbalance during training. The model was trained using YOLOv8n for 50 epochs with an image size of 640 × 640 and batch size 16 using a Tesla T4 GPU in Google Colab.

The model achieved an overall precision of **0.706**, recall of **0.380**, mAP@0.5 of **0.439**, and mAP@0.5:0.95 of **0.270**. The model performed best on missing_barricade, damaged_warning_sign, and broken_handrail, while blocked_walkway and damaged_sidewalk remained more challenging.

Although the model still has limitations, it produced useful detection outputs and can be used as Member 4's YOLO model in the final campus hazard detection system. The trained **best.pt** model can be integrated with the group's mobile application and combined with the other members' YOLO models through the neural-network meta-classifier. The overlap class **damaged_sidewalk** also supports comparison with Member 1's model during ensemble and meta-classifier processing.

6. Meta-Classifier and Conflict Resolution

6.1 Introduction

The proposed campus hazard detection system combines the outputs of four independent YOLO models. Each YOLO model was trained by a different group member and focuses on different campus hazard categories. Although this approach improves hazard coverage, it may also create inconsistent predictions when multiple models detect the same physical object or area using different labels.

For example, a ground opening may be classified as *pothole* by one model, *open drain* by another model, and *uncovered manhole* by a third model. Similarly, damaged pavement may be detected using labels such as *cracked pavement*, *uneven pavement*, or *damaged sidewalk*. Displaying all these labels directly to users may cause confusion.

Therefore, a meta-classifier was developed as the final decision layer. Its purpose is to combine the YOLO outputs, analyse model agreement and overlap information, and produce one final global hazard label for each detected hazard region.



6.2 Label Harmonisation and IoU Grouping

Before meta-classifier prediction, the outputs from all four YOLO models are converted from local class labels into a shared global label space. This process is known as label harmonisation.

The global label space contains the following hazard categories:

- pothole
- cracked pavement
- wet slippery floor
- surface damage
- damaged sidewalk
- obstacle on walkway
- construction debris

- fallen branch
- traffic cone
- open drain
- uncovered manhole
- uneven pavement
- missing barricade
- damaged warning sign
- broken handrail
- blocked walkway

After label harmonisation, detections from different models are compared using Intersection over Union (IoU). IoU measures the overlap between two bounding boxes. When two detections belong to related hazard categories and have an IoU value of at least 0.30, they are grouped as possible descriptions of the same physical hazard.

The related hazard groups used in this study are shown below.

Related Hazard Group	Labels Included
Ground opening hazards	pothole, open drain, uncovered manhole
Surface damage hazards	surface damage, damaged sidewalk, uneven pavement, cracked pavement
Walkway obstruction hazards	obstacle on walkway, construction debris, fallen branch, blocked walkway

This grouping stage prevents similar or overlapping detections from being treated as completely separate hazards.



6.3 Meta-Classifer Feature Extraction

Each grouped detection is transformed into a feature vector before being sent to the meta-classifier. The feature vector represents the confidence, agreement, overlap, and spatial characteristics of the grouped YOLO predictions.

The extracted features include:

Feature Category	Features Used
------------------	---------------

YOLO confidence features	Best confidence, mean confidence, minimum confidence
Agreement features	Number of models that detected the hazard, number of detections in the group
Overlap features	Maximum IoU and mean IoU
Bounding-box features	Bounding-box position, width, height, and area
Model participation	Whether Member 1, Member 2, Member 3, or Member 4 detected the hazard
Label voting features	Number of votes received by each global hazard label

These features allow the meta-classifier to consider more than the highest confidence score. For example, a detection supported by two or more models may be more reliable than a high-confidence detection produced by only one model.

6.4 Meta-Classifier Architecture

A Multi-Layer Perceptron (MLP) neural network was used as the meta-classifier. The model receives the extracted ensemble features and predicts the final global hazard label.

The MLP architecture contains:

Component	Configuration
Input layer	Ensemble feature vector
Hidden layer 1	32 neurons
Hidden layer 2	16 neurons
Activation function	ReLU
Optimiser	Adam
Feature scaling	StandardScaler
Maximum iterations	1500
Random state	42

The data was divided into 80% training data and 20% testing data using stratified sampling. Stratified sampling was used to preserve the class distribution as much as possible between the training and testing sets.

The meta-classifier supported the following 13 classes:

- broken handrail
- construction debris
- cracked pavement
- damaged sidewalk
- damaged warning sign
- fallen branch
- obstacle on walkway
- open drain
- pothole
- traffic cone
- uncovered manhole
- uneven pavement
- wet slippery floor

6.5 Meta-Classifier Dataset

The meta-classifier dataset was generated from the grouped outputs of the four YOLO models. Each row in the dataset represents one grouped hazard detection from a test image.

A total of 105 grouped ensemble detections were obtained. The class distribution is shown below.

Target label distribution:

```
target_global_label
cracked_pavement      14
traffic_cone          12
pothole                11
damaged_sidewalk       10
uncovered_manhole      10
wet_slippery_floor     10
construction_debris    7
broken_handrail         6
damaged_warning_sign    6
fallen_branch           6
uneven_pavement         5
obstacle_on_walkway     4
open_drain              4
```

The dataset is moderately imbalanced because some hazards were more frequently detected in the available test images. For example, cracked pavement, traffic cone, and pothole had more samples than open drain and obstacle on walkway.

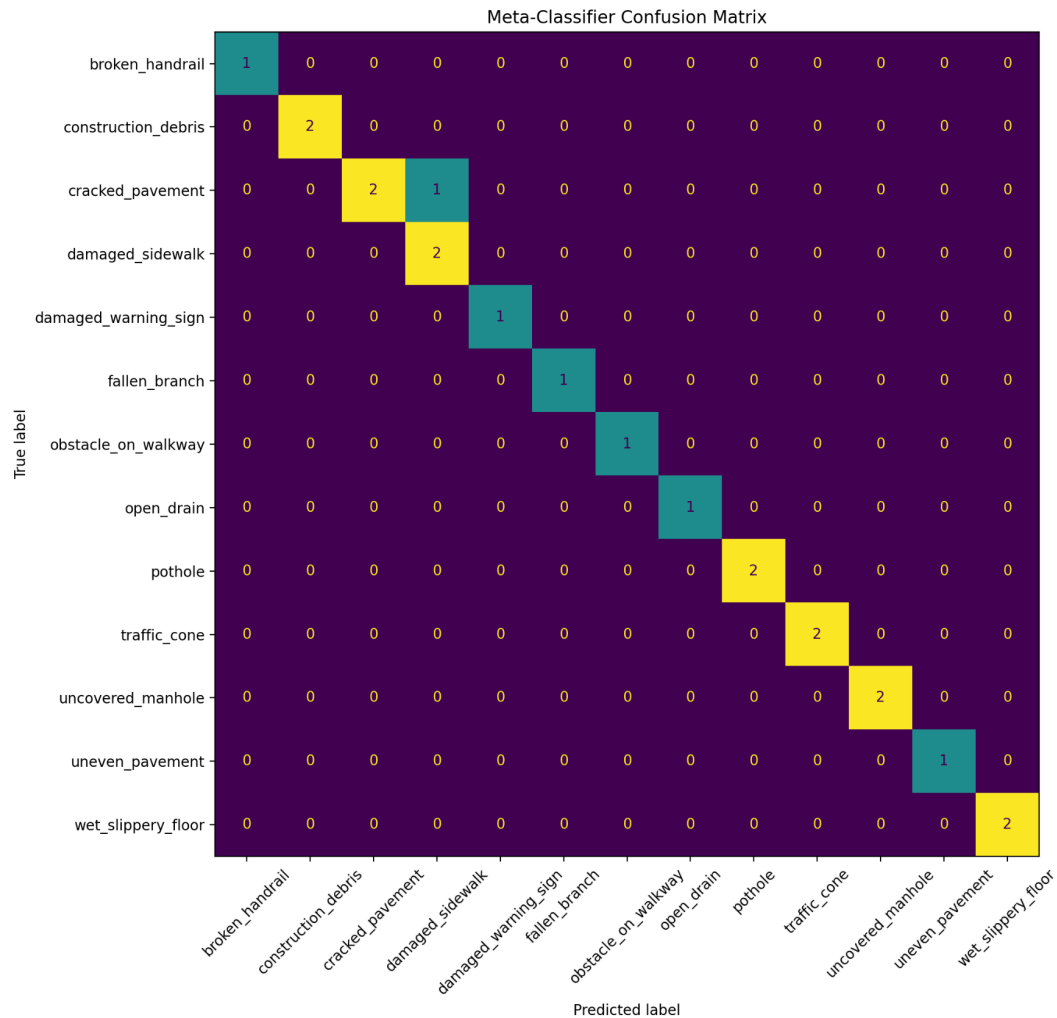
6.6 Training and Evaluation Results

The MLP meta-classifier was trained using 84 samples and evaluated using 21 test samples. The overall performance is shown in Table 4.X.

Evaluation Metric	Result
Training samples	84
Testing samples	21
Accuracy	95.2%
Macro Precision	97.4%
Macro Recall	97.4%
Macro F1-score	96.9%

The results show that the meta-classifier achieved strong performance on the prototype test split. Most test samples were classified correctly.

The confusion matrix shows that only one cracked pavement sample was classified as damaged sidewalk. This confusion is reasonable because both categories belong to surface-damage hazards and may share similar visual characteristics, such as broken pavement edges, cracks, and damaged walkway surfaces.



6.7 Conflict Resolution Example

One important example is the ground opening image. In this case, different YOLO models produced the following predictions for the same region:

YOLO Model	Raw Prediction	Role in Conflict Resolution
Member 1 model	pothole; wet slippery floor	<i>pothole</i> was grouped with the ground-opening conflict. <i>wet slippery floor</i> was treated as a separate detection group.
Member 2 model	fallen branch	Treated as a separate single-model detection group because it did not overlap with the ground-opening region.
Member 3 model	open drain; uncovered manhole	Both labels were grouped with <i>pothole</i> because they referred to the same ground-opening region.
Member 4 model	No detection	Did not contribute a prediction for this image.

For the ground-opening region, Member 1 predicted *pothole*, while Member 3 predicted *open drain* and *uncovered manhole*. These three predictions overlapped spatially and belonged to the same ground-opening hazard group. The meta-classifier selected *uncovered manhole* as the final global label with a meta-classifier confidence of 0.966.

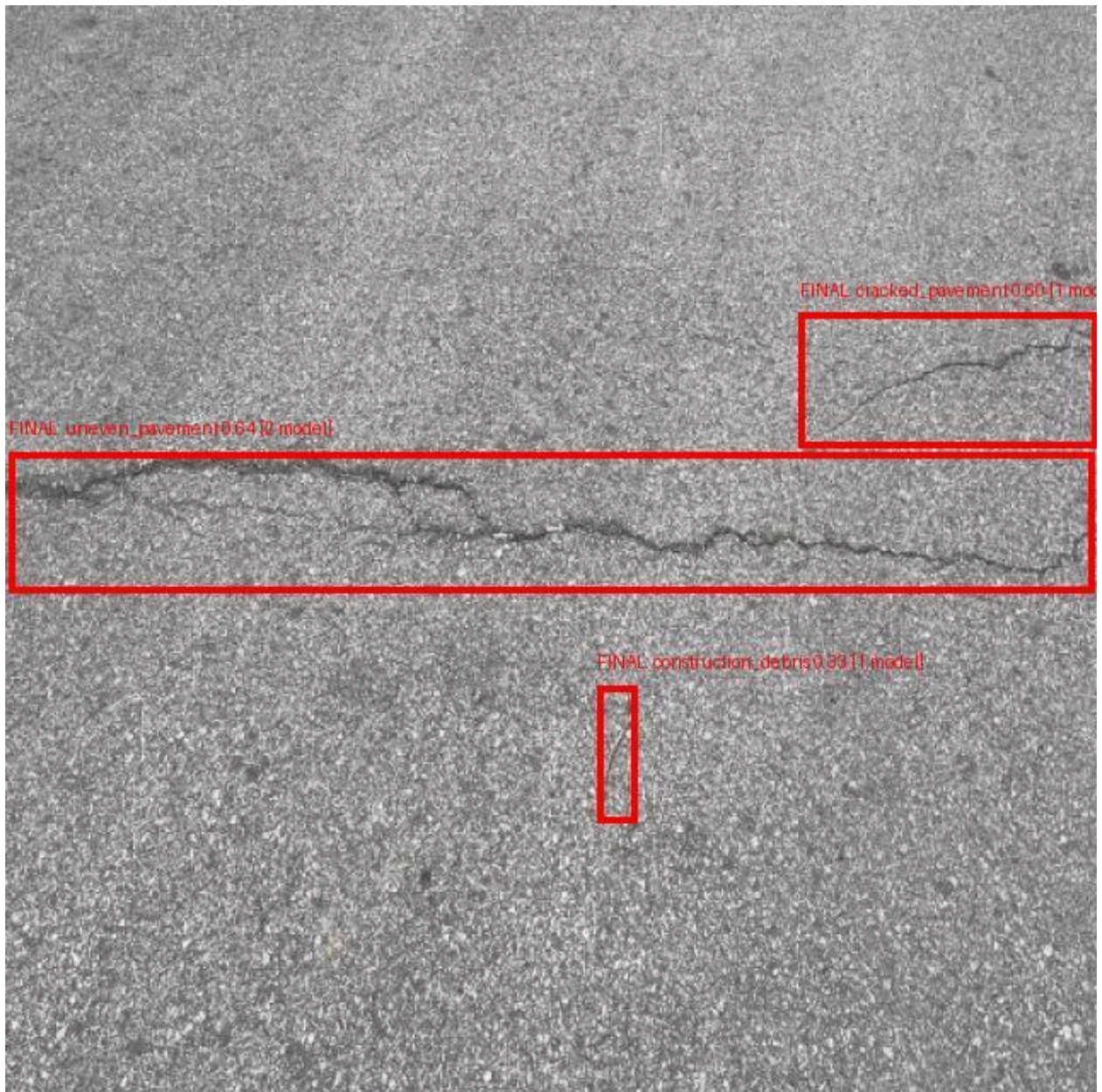
The *wet slippery floor* and *fallen branch* predictions were processed as separate detection groups because they did not sufficiently overlap with the ground-opening bounding box. Member 4 did not produce a detection for this image.

The final predicted label was:

Final Output	Result
Final hazard label	uncovered manhole

Meta-classifier confidence	0.966
Model agreement	2 models
Detection status	High Confidence

This example demonstrates that the system can reduce inconsistent multi-model labels and provide one clearer final hazard result.



6.8 Detection Status and Reliability Handling

To improve the practical usefulness of the system, each final prediction is assigned a detection status based on model agreement and meta-classifier confidence.

Status	Rule	Interpretation
High Confidence	At least two models agree and meta confidence ≥ 0.70	Strong prediction
Medium Confidence	One model detects the hazard and meta confidence ≥ 0.90	Possible hazard; user should inspect
Needs Review	Low agreement or low confidence	Manual verification recommended

This mechanism is important because a single YOLO model may occasionally produce an incorrect label. For example, a construction area may contain visual textures that are mistakenly detected as a broken handrail or uncovered manhole. By displaying a confidence status, the system avoids presenting every prediction as equally reliable.

6.9 Final Inference Workflow

During final system inference, the workflow is as follows:

1. The input image is sent to all four YOLO models.
2. Each model generates bounding boxes, local labels, and confidence scores.
3. Local labels are converted into global labels through label harmonisation.
4. Related bounding boxes are grouped using IoU-based overlap matching.
5. Ensemble features are extracted from every grouped detection.
6. The MLP meta-classifier predicts the final global hazard label.
7. The system generates a final confidence score and detection status.
8. The annotated output image and hazard recommendation are displayed to the user.

This workflow enables the system to combine the specialisation of multiple YOLO models while reducing inconsistent predictions from overlapping hazard categories.

6.10 Limitation and Future Improvement

Although the meta-classifier achieved high prototype evaluation results, several limitations should be considered.

First, the meta-classifier dataset contains only 105 grouped ensemble detections. This is sufficient for a prototype demonstration but is not large enough for strong real-world generalisation.

Second, the dataset is not fully balanced. Some classes, such as open drain and obstacle on walkway, have fewer examples than cracked pavement and traffic cone.

Third, the initial labels used for the meta-classifier were generated from the rule-based conflict-resolution process. Therefore, the current performance mainly demonstrates that the MLP can learn the ensemble decision patterns in the prototype pipeline.

In future work, the system should collect more campus hazard images and create a larger human-verified meta-classifier dataset. Each grouped detection should be manually checked and assigned a ground-truth final global label. This would provide a more reliable evaluation of the meta-classifier in real campus environments.

7. Mobile App Implementation

7. Mobile App Implementation

The mobile application was developed using Flutter and Dart and was named SafeCampus AI. The purpose of the mobile app is to provide an easy-to-use interface for campus hazard detection. Instead of only running the models in a notebook or command-line environment, the app allows a user to select or capture a hazard image, choose the campus zone, send the image to the backend, and view the final detection result in a mobile-friendly format.

The app was designed as the frontend component of the complete hazard detection system. The backend performs the heavy computation, including YOLO inference, ensemble processing, neural-network meta-classification, Gemini-generated recommendation, and evidence saving. The Flutter app focuses on user interaction, image input, result display, bounding box visualisation, and evidence capture.

This design was chosen because YOLO inference and meta-classifier processing are more suitable for the backend, while Flutter is better suited for building a clean mobile interface. This also makes the system easier to test, because the backend can be verified separately through API endpoints and the frontend can be tested on Android emulator or web browser.

7.1 Development Framework and Tools

The frontend was implemented using Flutter and Dart. Flutter was selected because it supports cross-platform mobile development and allows the same codebase to run on Android emulator, web browser, and potentially other platforms. This was useful during development because the app could be tested quickly on Chrome while also being checked on an Android emulator for mobile behaviour.

The main Flutter packages used in the app were ``http`` and ``image_picker``. The ``http`` package was used to send multipart HTTP requests from the Flutter app to the FastAPI backend. The ``image_picker`` package was used to allow the user to select images from the gallery or capture an image using the camera. Flutter Material components were used to build the visual layout, buttons, cards, dropdowns, icons, and result sections.

Visual Studio Code was used as the main development environment for the Flutter app. Android Studio and the Android emulator were used for mobile testing. The backend was developed using Python and FastAPI and was run locally during testing.

7.2 System Architecture of the Mobile App

The mobile app is one part of the full SafeCampus AI system. The complete flow is:

Flutter Mobile App
→ FastAPI Backend
→ Four YOLO Detection Models
→ Feature Vector Construction
→ Neural-Network Meta-Classifer
→ Gemini API Recommendation
→ Detection Result and Evidence Record

The mobile app does not directly run the YOLO models. Instead, it sends the selected image to the backend. This avoids placing large model files inside the app and keeps the mobile app lightweight. It also allows the backend to combine multiple YOLO models and run the meta-classifier before sending the final result back to the app.

The app communicates with the backend through REST API endpoints. During Android emulator testing, the backend address was set to ``http://10.0.2.2:8000``, because Android emulator uses ``10.0.2.2`` to access the host computer. During web testing, the backend address was ``http://127.0.0.1:8000``.

7.3 User Workflow

The app follows a simple and practical workflow. First, the user selects a campus zone from the dropdown menu. The available zones include areas such as Main Walkway, Parking Area, Building Entrance, Sports Area, and Construction Zone. This zone information is important because the final recommendation should mention where the hazard was detected.

Next, the user chooses an image from the gallery or captures an image using the camera button. After the image is selected, the app displays it in the preview area. The user then presses the Detect Hazard button. The app packages the selected image, zone, and confidence threshold into a multipart HTTP request and sends it to the FastAPI backend.

After the backend completes inference, the app receives a JSON response. This response includes the final hazard result, the individual YOLO candidate detections, bounding box coordinates, confidence score, severity level, model source, and recommended action. The app then updates the interface to show the result.

The workflow was designed to be clear for demonstration and testing. It follows a realistic field-use pattern: select location, capture evidence, detect hazard, review result, and save evidence.

7.4 Backend API Integration

The app integrates with the backend using HTTP multipart form requests. The uploaded image is sent as a file, while the selected zone and confidence threshold are sent as form fields. The backend receives the request through the ``/detect`` endpoint.

The ``/detect`` endpoint returns structured JSON data. The most important fields are ``detected``, ``zone``, ``final``, and ``detections``. The ``final`` field contains the final prediction from the meta-classifier, including the final label, category, severity, confidence, bounding box, source model, and recommendation. The ``detections`` field contains the candidate outputs from the individual YOLO models.

The `/health` endpoint was also used during development to verify that the backend was running correctly. It confirmed that all four YOLO models were loaded, the meta-classifier was available, and the Gemini API configuration was detected.

A separate `/evidence` endpoint was implemented for saving evidence records. After detection, the mobile app can send the selected image and detection result to this endpoint so the backend can save them as an evidence package.

7.5 YOLO Ensemble and Meta-Classifier Display

The assignment requires the system to combine multiple YOLO models and use a meta-classifier to make the final decision. Therefore, the app was designed to show both the final meta-classifier result and the individual model candidates.

The Detection Result section displays the final result selected by the meta-classifier. This includes the hazard name, category, severity, confidence score, model name, and campus zone. For example, one test result showed:

Final Hazard: Uncovered Manhole
Category: Hole On Road Or Ground
Confidence: 40.7%
Model: meta_classifier - Neural Meta-Classifier
Zone: Main walkway

The Model Candidates section displays the original detections from the YOLO models. In one example, the candidate outputs included Pothole from member 1 and Uncovered Manhole from member 3. This is useful because it shows how different individual models can produce overlapping or conflicting predictions. The meta-classifier then selects the final class based on the combined feature vector rather than simply taking one YOLO result directly.

This design improves transparency. Users can see not only the final answer, but also the model candidates that contributed to the ensemble decision.

7.6 Bounding Box Visualisation

The app displays the selected image together with a bounding box drawn around the detected hazard. The bounding box coordinates are returned by the backend and then rendered on top of the image in the Flutter interface.

The bounding box label includes the final hazard class and confidence score. For example, the app displayed “Uncovered Manhole 41%” above the detected area. This visual feedback is important because it helps users confirm that the model is focusing on the correct object in the image.

The bounding box display also supports the report and screencast requirements because it provides clear visual evidence of the detection result.

7.7 Gemini API Recommendation

The backend integrates the Gemini API to generate a practical recommended action for the detected hazard. The app does not call Gemini directly. Instead, the backend sends the final detection information to Gemini after the meta-classifier result has been produced.

The prompt sent to Gemini includes the hazard class, category, severity level, confidence score, and selected campus zone. This allows Gemini to generate a recommendation that is specific to the detected hazard and location.

For example, for an uncovered manhole in the Main walkway, the app displayed the following Gemini-generated recommendation:

“Secure the uncovered manhole immediately and place a visible barrier. Report for permanent cover replacement.”

This recommendation is more natural than a fixed template because it is generated based on the detection context. The Gemini API key is stored only in the backend environment variable and is not included in the Flutter app code. This protects the API key from being exposed in the mobile client.

A fallback recommendation function was also implemented in the backend. If Gemini is unavailable or the API request fails, the backend still returns a rule-based recommendation. This makes the app more reliable during testing and demonstration.

7.8 Evidence Capture Feature

The app includes an Evidence Capture section after a successful detection. The user can press the Save Evidence button to save the result. This feature was added to satisfy the assignment requirement for saving detection evidence.

When the Save Evidence button is pressed, the app sends the selected image and the structured detection result to the backend `/evidence` endpoint. The backend creates a new evidence folder and saves two main files: the image file and a JSON record.

The JSON record includes the evidence ID, timestamp, campus zone, final hazard class, confidence score, severity level, recommended action, and candidate detections from the YOLO models. This makes the saved evidence useful for later analysis, report writing, and demonstration.

The evidence capture feature turns the app from a simple prediction interface into a more complete reporting tool. It allows each detection to be documented rather than only displayed temporarily on screen.

7.9 User Interface Design

The interface was designed to be simple and readable on a mobile screen. The app uses a card-based layout with clear sections: Scan Setup, Detection Result, Evidence Capture, Model Candidates, and Image Preview.

The Scan Setup card contains the campus zone dropdown, image selection button, camera button, and detection button. The Detection Result card highlights the final hazard result. The Evidence Capture card appears after a successful detection and allows the user to save the current result. The Model Candidates card shows the individual YOLO detections. The Image Preview card displays the detected image and bounding box.

Color is used to communicate severity. High-severity hazards are highlighted in red, medium hazards use warning colors, and lower-risk results use safer colors. This helps users quickly identify urgent hazards.

The design avoids unnecessary complexity and focuses on the main task: selecting an image, detecting the hazard, reviewing the result, and saving evidence.

7.10 Testing on Android Emulator and Web

The app was tested on both Android emulator and Flutter web. Web testing was useful for quick frontend checks, while Android emulator testing was important for verifying mobile behaviour, gallery image selection, camera access, and communication with the local backend.

Several issues were solved during testing. For Android emulator networking, the backend URL had to be changed from `127.0.0.1` to `10.0.2.2`, because `127.0.0.1` inside the emulator refers

to the emulator itself rather than the host computer. Test images were also pushed into the emulator storage so that the gallery picker could access them.

The backend was tested separately using the `/health` endpoint. This confirmed that the four YOLO models, meta-classifier, and Gemini configuration were correctly loaded. The app was then tested by uploading hazard images and checking whether the returned results were correctly displayed.

7.11 Example Test Result

One successful test case used an image of an uncovered manhole. The backend returned multiple candidate detections. One YOLO model detected Pothole, while another detected Uncovered Manhole. The meta-classifier selected Uncovered Manhole as the final result.

The app displayed the final hazard as Uncovered Manhole, with high severity and a confidence score of 40.7%. It also displayed the candidate detections, the Gemini-generated recommended action, and a bounding box around the manhole in the image.

This test case was useful because it demonstrated the purpose of the ensemble and meta-classifier. The individual YOLO models did not all produce the same label, but the meta-classifier still produced a final decision.

7.12 Limitations and Future Improvements

The current app is a working prototype and depends on a local backend server. The backend must be running before the app can perform detection. This is acceptable for development and demonstration, but a real deployment would require hosting the backend on a cloud server or campus server.

The app currently supports gallery image selection and camera image capture, but it does not yet perform continuous real-time video inference. Real-time camera detection could be added in the future by streaming frames to the backend or by converting the model to a mobile-friendly format.

Another limitation is that the meta-classifier confidence can be relatively low when the image is difficult or when multiple hazards look similar. Future work could improve the meta-classifier training dataset, calibrate confidence scores, and test more examples from different campus zones.

Future improvements could include cloud deployment, user login, a database for saved evidence, report export, map-based hazard location, real-time camera scanning, push notifications for urgent hazards, and a dashboard for campus maintenance staff.

Overall, the Flutter app successfully connects the trained YOLO ensemble and neural-network meta-classifier to a usable mobile interface. It supports image input, backend inference, meta-classifier result display, Gemini-generated recommendations, bounding box visualisation, and evidence capture, which together satisfy the main mobile app requirements of the assignment.

8. LLM-Based Recommended Action

8.1 Purpose

The hazard detection system not only identifies campus hazards but also provides an appropriate recommended action. After the four YOLO models and the meta-classifier determine the final hazard category, the result is passed to a Large Language Model (LLM) recommendation module.

The purpose of this module is to convert the technical classification result into a short, clear, and safety-oriented instruction that can be understood by students, campus staff, and maintenance personnel. For example, instead of displaying only the label `uncovered_manhole`, the system can recommend that the affected area be isolated and reported to campus facilities for urgent repair.

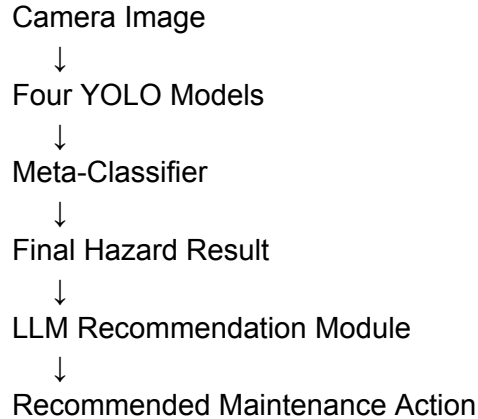
The LLM recommendation is intended to provide initial safety guidance. It does not replace professional inspection, official maintenance procedures, or emergency services.

8.2 Recommendation Workflow

The complete recommendation process contains the following steps:

1. The mobile application captures an image or video frame.
2. The four YOLO models detect possible hazards.
3. The meta-classifier resolves overlapping or conflicting predictions.
4. The final hazard class and confidence score are produced.
5. The system assigns a general category, location, and severity level.
6. The structured hazard information is passed to the LLM module.
7. The LLM generates one concise recommended maintenance action.
8. The mobile application displays the hazard label, confidence, severity, and recommended action to the user.

The overall workflow can be represented as:



8.3 Input Data

The recommendation module receives structured JSON data rather than the original image. This approach reduces the amount of information sent to the LLM and ensures that the generated recommendation is based on the final decision of the meta-classifier.

The input contains the following fields:

- **Hazard class:** The final hazard label selected by the meta-classifier.
- **General category:** The broader category to which the hazard belongs.
- **Location zone:** The campus location where the hazard was detected.
- **Severity:** The estimated risk level, represented as Low, Medium, or High.
- **Confidence:** The confidence score of the final classification result.

An example input is shown below:

```
{
  "hazard_class": "uncovered_manhole",
  "general_category": "Road and walkway hazard",
  "location_zone": "Campus road",
  "severity": "High",
  "confidence": 0.91
}
```

Using structured input makes the recommendation process more consistent and allows the same module to be connected to the mobile application or another user interface in the future.

8.4 LLM Implementation

The recommendation module is implemented in Python using the Google Gemini API. The configured model is `gemini-1.5-flash`, which was selected because it can generate short responses with relatively low latency.

The Gemini API key is read from the `GEMINI_API_KEY` environment variable. The API key is not stored directly in the source code or uploaded to the public GitHub repository. This reduces the risk of exposing authentication credentials.

The main implementation is contained in:

`llm_recommendation/gemini_recommendation.py`

The module performs the following operations:

1. Reads the structured hazard information from a JSON file or application request.
2. Converts the structured fields into a prompt.
3. Sends the prompt to the Gemini API.
4. Extracts the generated text from the API response.
5. Returns the recommendation to the calling application.
6. Uses a rule-based fallback recommendation if the API is unavailable.

A low temperature value of `0.2` is used to reduce randomness. This encourages the model to generate stable and focused safety recommendations rather than creative or unrelated responses. The output token limit is also restricted because the application requires only one short action.

8.5 Prompt Design

A carefully designed prompt is used to control the content and style of the generated recommendation. The system instruction is:

You are a campus safety maintenance assistant. Generate one concise, practical, safety-oriented action for the detected campus hazard. The action must be relevant to facilities maintenance. Do not suggest that students repair dangerous hazards themselves. If the hazard may endanger people, recommend isolating the area and reporting it to the responsible campus maintenance unit.

The hazard information is then inserted into the following user prompt:

Detected hazard:

- Hazard class: `{hazard_class}`
- General category: `{general_category}`

- Location zone: {location_zone}
- Severity: {severity}
- Confidence: {confidence}

Return exactly one short recommended maintenance action.

Keep it practical, safety-oriented, and under 30 words.

This prompt design establishes several constraints:

- The response should contain only one recommended action.
- The recommendation should be concise and easy to display in the mobile application.
- The recommendation must be related to campus maintenance.
- Dangerous hazards should be isolated and reported.
- Students should not be instructed to perform hazardous repairs.
- The recommendation should avoid unnecessary explanations or unrelated information.

8.6 Safety Controls

Safety is an important requirement because an unsuitable recommendation could expose users to additional danger. The module therefore applies the following safety principles:

1. **No dangerous self-repair:**
Students are not instructed to touch, inspect, or repair hazards such as uncovered manholes, electrical hazards, broken handrails, or damaged structures.
2. **Area isolation:**
High-severity hazards should be isolated using barriers, warning signs, or access restrictions where possible.
3. **User warning:**
The generated action should warn pedestrians or other campus users when the hazard presents an immediate risk.
4. **Responsible reporting:**
Hazards should be reported to campus facilities, security, cleaning personnel, or another responsible maintenance unit.
5. **Professional inspection:**
Structural and high-risk hazards should be inspected and repaired by qualified personnel.
6. **Concise output:**
Recommendations are limited to a short action so that important safety information is not hidden inside a long paragraph.

8.7 Rule-Based Fallback Mechanism

The system should still provide useful guidance when the Gemini API cannot be accessed. Therefore, a local rule-based fallback mechanism is included.

The fallback mechanism is activated when:

- `GEMINI_API_KEY` has not been configured;
- the network connection is unavailable;
- the Gemini API request times out;
- the API returns no candidate response;
- the response is empty or invalid; or
- another supported API error occurs.

Predefined recommendations are available for common hazard classes, including:

- uncovered manhole;
- open drain;
- pothole;
- wet or slippery floor;
- broken handrail;
- blocked walkway;
- obstacle on walkway;
- damaged sidewalk;
- cracked pavement;
- missing barricade; and
- damaged warning sign.

For example, the fallback recommendation for `wet_slippery_floor` is:

Place a wet-floor warning sign, restrict access if needed, and request cleaning staff to dry the area promptly.

If the hazard class does not have a predefined recommendation, the module generates a general response based on severity. For an unknown high-severity hazard, the fallback response is:

Isolate the area, warn nearby users, and report the hazard to campus facilities for urgent inspection.

This hybrid approach combines the flexibility of an LLM with the reliability of deterministic rules. It also ensures that the recommendation feature remains functional during demonstrations when the API key or internet connection is unavailable.

8.8 Example Outputs

Example 1: Uncovered Manhole

Input:

```
{
  "hazard_class": "uncovered_manhole",
  "general_category": "Road and walkway hazard",
  "location_zone": "Campus road",
  "severity": "High",
  "confidence": 0.91
}
```

Recommended action:

Place a temporary barricade, keep pedestrians away, and report the uncovered manhole to campus facilities for urgent repair.

Example 2: Wet and Slippery Floor

Input:

```
{
  "hazard_class": "wet_slippery_floor",
  "general_category": "Road and walkway hazard",
  "location_zone": "Lecture hall corridor",
  "severity": "Medium",
  "confidence": 0.86
}
```

Recommended action:

Place a wet-floor warning sign, restrict access if needed, and request cleaning staff to dry the corridor promptly.

Example 3: Broken Handrail

Input:

```
{
  "hazard_class": "broken_handrail",
  "general_category": "Building and structural hazard",
  "location_zone": "Academic block staircase",
  "severity": "High",
}
```

```
"confidence": 0.88  
}
```

Recommended action:

Warn users to avoid the affected side and report the broken handrail to facilities for urgent inspection and repair.

8.9 Integration with the Mobile Application

The generated recommendation is designed to appear together with the final detection result in the mobile application. The result interface should display:

- the detected hazard class;
- the bounding box around the detected hazard;
- the final confidence score;
- the assigned severity level;
- the detected or selected campus location; and
- the recommended maintenance action.

A possible result displayed to the user is:

Hazard: Uncovered Manhole
Confidence: 91%
Severity: High
Location: Campus Road

Recommended Action:

Place a temporary barricade, keep pedestrians away,
and report the uncovered manhole to campus facilities
for urgent repair.

The LLM is not used to determine the hazard category. The classification decision remains the responsibility of the YOLO models and meta-classifier. The LLM is used only after classification to convert the structured result into understandable safety guidance.

8.10 Privacy and Security Considerations

Only structured hazard information is required by the recommendation module. The original image does not need to be transmitted to the LLM API. This reduces unnecessary exposure of images that may contain students, staff, vehicles, or identifiable campus locations.

The following controls are applied:

- API credentials are stored in an environment variable.
- API keys are excluded from the GitHub repository.
- The original camera image is not required by the LLM.
- Only information necessary for generating the recommendation is sent.
- A local fallback is available when external API communication is not possible.

8.11 Limitations

Although the LLM can produce context-aware recommendations, several limitations remain:

- The recommendation depends on the correctness of the final hazard classification.
- An incorrect location or severity value may produce an unsuitable action.
- LLM responses may vary slightly even when similar input is provided.
- The recommendation does not verify whether campus maintenance resources are currently available.
- The module does not replace emergency services or professional assessment.
- Internet access and a valid API key are required for Gemini-generated responses.
- The rule-based fallback contains less contextual flexibility than the LLM.

Future improvements could include validating every LLM response against a safety policy, adding more fallback rules, supporting institution-specific emergency contacts, and allowing campus maintenance staff to approve or update recommended actions.

8.12 Summary

The LLM-based recommendation module connects hazard detection with practical campus safety management. It transforms the final meta-classifier result into a short and actionable maintenance recommendation. Prompt constraints help keep the output concise and safe, while the local fallback mechanism maintains system reliability when the Gemini service is unavailable. This provides users with more useful information than displaying a hazard label alone and supports faster reporting and response to campus safety issues.

9. Mobile App Testing

9.1 Overview of Mobile App Testing

Mobile app testing was conducted to evaluate whether the SafeCampus AI prototype could correctly perform the main functions required by the project. The testing focused on the complete workflow from image input to final hazard result display. This included image selection, backend connection, YOLO model inference, meta-classifier output, LLM-based recommended action, bounding-box display, and evidence saving.

The purpose of testing was not only to check whether the app interface worked, but also to verify whether the integrated system could process campus hazard images and return useful results to the user. Since the application is still a prototype, the testing was performed using selected test images instead of full real-time deployment across the campus.

The testing process covered the following main functions:

Testing Area	Purpose
Image input testing	To check whether the app can select or capture an image
Backend connection testing	To check whether the app can send the image to the FastAPI backend
YOLO model inference testing	To check whether the backend can run the four YOLO models
Meta-classifier testing	To check whether the system can produce a final hazard label
Recommendation testing	To check whether the system can generate a suitable maintenance action
Result display testing	To check whether the app can display hazard class, confidence, severity, and bounding box
Evidence saving testing	To check whether the app can save image evidence and JSON result records

9.2 Testing Environment

The mobile application was tested using Flutter during development. The app was mainly tested on an Android emulator and also checked through browser-based Flutter testing. The backend was implemented using FastAPI and was run locally during testing.

The Android emulator connected to the local backend using <http://10.0.2.2:8000/detect>, while browser-based testing used <http://127.0.0.1:8000/detect>. The backend loaded the trained YOLO model files and the neural-network meta-classifier before processing the uploaded image.

Item	Testing Environment
Mobile framework	Flutter and Dart
Backend framework	FastAPI
Testing platform	Android emulator and Chrome browser
Backend URL for Android emulator	http://10.0.2.2:8000/detect
Backend URL for browser testing	http://127.0.0.1:8000/detect
Input type	Selected test images and camera-captured images
Model integration	Four YOLOv8n models and neural-network meta-classifier
Recommendation module	Gemini API or local fallback recommendation dictionary

9.3 Test Images

The testing images were collected separately from the training datasets. This was done to evaluate whether the system could process unseen images instead of only recognising images used during training. The test images focused on walkway-related campus hazard scenarios

because these hazards are common in university environments and are suitable for mobile app demonstration.

The test image categories included:

Test Category	Example Hazard
Surface hazard	uneven_pavement, cracked_pavement, damaged_sidewalk
Obstruction hazard	obstacle_on_walkway, construction_debris, fallen_branch, traffic_cone
Drainage and ground hazard	uncovered_manhole, open_drain, pothole
Safety facility hazard	broken_handrail, damaged_warning_sign, missing_barricade

The test image folder was organised as follows:

```
test_images_walkway/  
├── uneven_pavement/  
├── obstacle_on_walkway/  
├── fallen_branch/  
├── construction_debris/  
├── traffic_cone/  
├── broken_handrail/  
├── damaged_warning_sign/  
└── uncovered_manhole/
```

Each folder contained several test images that were used for app testing, report evidence, and screencast demonstration.

9.4 Functional Test Cases

The following functional test cases were used to evaluate the mobile app prototype.

Test Case ID	Test Scenario	Input	Expected Result	Actual Result
--------------	---------------	-------	-----------------	---------------

TC01	Launch the mobile app	Open SafeCampus AI app	App opens successfully and displays the scan setup page	Passed
TC02	Select campus zone	Choose Main Walkway / Parking Area / Building Entrance	Selected zone is shown and sent with the detection request	Passed
TC03	Select image from gallery	Upload a test image	Image preview is displayed in the app	Passed
TC04	Send image to backend	Press detect button	Image is uploaded to FastAPI backend	Passed
TC05	Run YOLO models	Backend receives image	Four YOLO models generate candidate detections	Passed
TC06	Run meta-classifier	YOLO candidate outputs	Final hazard class and confidence are returned	Passed
TC07	Display detection result	Backend JSON response	App displays hazard class, severity, confidence, and model source	Passed
TC08	Display bounding box	Detection result with bounding box coordinates	Bounding box is drawn on the image preview	Passed
TC09	Display recommendation	Final hazard class and severity	App displays recommended maintenance action	Passed
TC10	Save evidence	Press Save Evidence button	Backend saves image and JSON result record	Passed

The functional testing showed that the main app workflow was successful. The app was able to accept user input, connect to the backend, process the image, display the detection result, generate a recommended action, and save evidence.

9.5 Detection Testing Scenarios

Several hazard images were used to test the detection and result display. The testing results are summarised below.

Test Scenario	Input Image Type	Expected Output	Observation
Test 1	Uncovered manhole image	uncovered_manhole, High severity	The system detected the hazard and displayed a high-risk maintenance recommendation
Test 2	Traffic cone image	traffic_cone, Low or Medium severity	The system detected the traffic cone and displayed a warning-related result
Test 3	Damaged warning sign image	damaged_warning_sign, Medium severity	The system returned a damaged warning sign result and a recommendation to repair or replace the sign
Test 4	Broken handrail image	broken_handrail, High severity	The system generated a recommendation to warn users and report the handrail for repair
Test 5	Obstacle on walkway image	obstacle_on_walkway or blocked_walkway	The system detected an obstruction-related hazard, but confidence varied depending on object appearance

Test 6	Uneven pavement image	uneven_pavement or damaged_sidewalk	The system sometimes produced lower confidence due to visual similarity with other surface hazards
Test 7	Construction debris image	construction_debris or blocked_walkway	The system detected construction-related obstruction when the object was clearly visible
Test 8	Fallen branch image	fallen_branch or obstacle_on_walkway	The system was able to identify visible branch-like obstruction, but complex backgrounds reduced confidence

These test cases show that the system can detect several clear hazard examples. However, the testing also showed that some visually similar or context-dependent hazards may produce lower confidence or require manual review.

9.6 Example App Output

One example output from the mobile app is shown below:

Hazard: Uncovered Manhole
Confidence: 91%
Severity: High
Location: Campus Road

Recommended Action:
Place a temporary barricade, keep pedestrians away, and report the uncovered manhole to campus facilities for urgent repair.

This output demonstrates that the app can display the final hazard label, confidence score, severity level, location zone, and recommended maintenance action in a user-friendly format.

9.7 Evidence Saving Test

The evidence saving function was tested after a successful detection. When the user pressed the Save Evidence button, the app sent the selected image and structured detection result to the backend `/evidence` endpoint. The backend then saved the image file and a JSON record.

The saved JSON record included:

Field	Description
Timestamp	Time when the evidence was saved
Campus zone	Selected location zone
Final hazard class	Final class selected by the meta-classifier
Confidence score	Final confidence value
Severity level	Low, Medium, or High
Recommended action	LLM-generated or fallback recommendation
Candidate detections	Individual YOLO model outputs before final decision

This function is useful because the saved evidence can be used for technical report screenshots, testing records, and screencast demonstration.

9.8 Testing Observations

The app testing showed that the system could complete the full detection pipeline from image input to result display. The backend was able to load the YOLO models and meta-classifier, while the frontend was able to show the final result clearly.

The strongest results were observed when the input image contained clear and visually distinctive hazards, such as traffic cones, uncovered manholes, and damaged warning signs. These objects have more consistent shapes and features, making them easier for the YOLO models to detect.

Lower confidence or missed detections were observed for visually similar or context-dependent classes. For example, uneven pavement, cracked pavement, and damaged sidewalk may share similar ground textures. Obstacle_on_walkway and blocked_walkway may also be difficult because many different object types can block a walkway.

The testing also showed that lighting, camera angle, object size, and background complexity affected detection performance. Images with clear lighting and a visible hazard object produced better results than dark, blurry, or crowded images.

9.9 Summary

In summary, mobile app testing confirmed that the SafeCampus AI prototype could perform the main required functions. The app successfully allowed image input, sent the image to the backend, displayed YOLO candidate detections, showed the meta-classifier final result, generated a recommended maintenance action, drew bounding boxes, and saved evidence records.

Although the prototype worked for demonstration and testing, some limitations remain. The current app depends on a local backend server, and some hazard classes may still produce low confidence due to visual similarity or limited campus-specific training data. Future testing should include more real campus images, more lighting conditions, more camera angles, and a larger variety of hazard examples.

SafeCampus AI

Web API

Campus hazard scan

Main walkway

Main walkway

Low

2 candidates



Scan setup

Campus zone

Main walkway

Choose Image

Detect Hazard

Detection result

Traffic Cone

Low severity

80%

Hazard

Traffic Cone

Category

Temporary Warning Object

Confidence

80.5%

Model

meta_classifier - Neural Meta-Classifier

Zone

Main walkway

Recommended action

Check the traffic cone's placement on the Main walkway. Relocate it to a more appropriate spot or remove it if no longer needed.

Evidence capture

Save Evidence

Model candidates

Damaged Sidewalk

member_1 - Road Surface Hazards | Walkway Hazard | Medium

33.5%

Traffic Cone

member_2 - Walkway Obstruction Hazards | Temporary Warning Object | Low

77.7%



SafeCampus AI

Web API

Campus hazard scan

Main walkway

Main walkway High 2 candidates



Scan setup

Campus zoneMain walkway

Choose Image

Detect Hazard

Detection result

Uncovered Manhole41%
High severity

HazardUncovered Manhole

CategoryHole On Road Or Ground

Confidence40.6%

Modelmeta_classifier - Neural Meta-Classifier

ZoneMain walkway

Recommended action

Place safety barriers around the uncovered manhole on the Main walkway. Schedule immediate replacement of the manhole cover.

Evidence capture

Save Evidence

Model candidates

Pothole

member_1 - Road Surface Hazards | Road Surface Hazard | High

61.4%

Uncovered Manhole

member_3 - Road and Drainage Hazards | Hole On Road Or Ground | High

84.7%



Campus hazard scan

Main walkway

Main walkway

Medium

1 candidates



Scan setup

Campus zone

Main walkway

Choose Image

Detect Hazard

Detection result



Wet Slippery Floor

45%

Medium severity

Hazard Wet Slippery Floor

Category Slip Hazard

Confidence 44.6%

Model meta_classifier - Neural Meta-Classifer

Zone Main walkway

Recommended action

Clean and dry the wet floor on the Main walkway, placing a 'wet floor' sign until the area is completely dry.

Evidence capture

Save Evidence

Model candidates



Wet Slippery Floor

member_2 - Walkway Obstruction Hazards | Slip Hazard | Medium

88.5%



SafeCampus AI

Web API

Campus hazard scan

Main walkway

Main walkway

Medium

3 candidates

Cracked Pavement 29%



Scan setup

Campus zone

Main walkway

Choose Image

Detect Hazard

Detection result

Cracked Pavement

Medium severity

29%

Hazard

Cracked Pavement

Category

Road Surface Hazard

Confidence

29.4%

Model

meta_classifier - Neural Meta-Classifier

Zone

Main walkway

Recommended action

Repair cracked pavement section on Main walkway to prevent tripping hazards.

Evidence capture

Save Evidence

Model candidates

Damaged Sidewalk

member_1 - Road Surface Hazards | Walkway Hazard | Medium

29.0%

Construction Debris

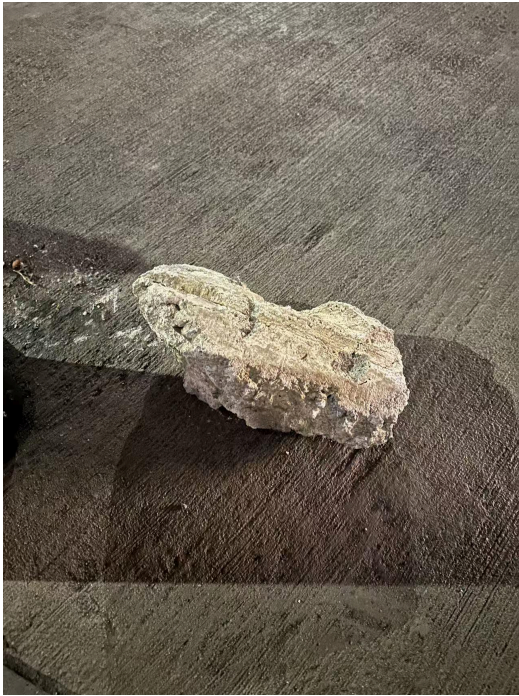
member_2 - Walkway Obstruction Hazards | Obstruction Hazard | Medium

50.8%

Cracked Pavement

member_3 - Road and Drainage Hazards | Road Surface Hazard | Medium

25.7%



Campus hazard scan

Main walkway

Main walkway

Medium

3 candidates



Scan setup

Campus zone

Main walkway

Choose Image

Detect Hazard

Detection result



Uneven Pavement

32%

Medium severity

Hazard Uneven Pavement

Category Road Surface Hazard

Confidence 32.0%

Model meta_classifier - Neural Meta-Classifer

Zone Main walkway

Recommended action

Repair uneven pavement section on Main walkway to prevent tripping hazards.

Evidence capture

Save Evidence

Model candidates

- Cracked Pavement**
member_3 - Road and Drainage Hazards | Road Surface Hazard | Medium 85.9%
- Cracked Pavement**
member_3 - Road and Drainage Hazards | Road Surface Hazard | Medium 81.8%
- Cracked Pavement**
member_3 - Road and Drainage Hazards | Road Surface Hazard | Medium 59.4%



10. Results and Discussion

10.1 Overview of Results

This section discusses the overall results of the campus hazard detection system. The evaluation covers four main parts: individual YOLO model performance, meta-classifier performance, mobile application testing, and LLM-based recommended action generation.

The results show that the proposed system was able to detect different types of campus hazards using four YOLOv8n models trained by different group members. Each model focused on a specific hazard category, including walkway surface hazards, walkway obstruction hazards, drainage and ground surface hazards, and access, boundary and safety facility hazards.

After individual YOLO detection, the meta-classifier was used to combine model outputs and produce the final global hazard class. The mobile application then displayed the final result, including hazard class, confidence score, severity level, bounding box, and recommended maintenance action.

Overall, the system achieved the main project objective by producing a working prototype that connects object detection, ensemble decision-making, mobile app display, and safety-oriented recommendation.

10.2 Individual YOLO Model Results

Each group member trained one YOLOv8n object detection model with five selected campus hazard classes. The models produced different performance results depending on the dataset size, class difficulty, training configuration, and visual consistency of each hazard category.

The overall performance of the individual YOLO models is summarised below.

Member	Model Focus	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Member 1	Walkway Surface Hazards	0.588	0.530	0.556	0.337
Member 2	Walkway Obstruction Hazards	0.485	0.508	0.488	0.299

Member 3	Drainage and Ground Surface Hazards	To be updated	To be updated	To be updated	To be updated
Member 4	Access, Boundary and Safety Facility Hazards	0.706	0.380	0.439	0.270

The results show that each individual model achieved a usable baseline performance for its selected hazard categories. Member 1 achieved an overall mAP@0.5 of 0.556, with wet_slippery_floor performing the best among its classes. Member 2 achieved an overall mAP@0.5 of 0.488, with traffic_cone performing the best because it has a clear and consistent shape. Member 4 achieved an overall precision of 0.706 and mAP@0.5 of 0.439, with missing_barricade showing the strongest performance.

Member 3's final numerical results should be updated after the training metrics are confirmed. However, the prediction examples showed that the model was able to detect important drainage and ground-surface hazards such as potholes and uncovered manholes.

10.3 Discussion of YOLO Model Performance

The individual YOLO results show that the models performed better on classes with clear and consistent visual features. For example, wet_slippery_floor in Member 1, traffic_cone in Member 2, and missing_barricade in Member 4 achieved stronger results compared with more visually complex classes. These classes are easier for the model to learn because they often have more recognisable colours, shapes, textures, or object boundaries.

In contrast, classes with high visual variation or contextual dependency showed lower performance. For example, Member 2's obstacle_on_walkway class achieved lower performance because the class may include many different objects, such as bags, boxes, rubbish, bicycles, and other items. These objects do not share one consistent shape or appearance. Similarly, Member 4's blocked_walkway class was difficult because the hazard depends not only on the object itself, but also on whether it blocks a pedestrian route.

Surface-related hazards were also challenging. Classes such as cracked_pavement, surface_damage, uneven_pavement, and damaged_sidewalk may share similar visual patterns, including cracks, broken pavement, irregular textures, and damaged ground surfaces. This can cause confusion between classes, especially when the images contain more than one type of surface defect.

Another important factor is dataset source. Most datasets were collected from public sources rather than actual campus environments. As a result, the images may have different lighting,

camera angles, pavement materials, weather conditions, object backgrounds, and facility designs compared with the real campus. This may reduce the model's generalisation when tested on real campus images.

10.4 Meta-Classifier Results

The meta-classifier was developed to combine the outputs from the four YOLO models and resolve overlapping or conflicting predictions. It used extracted ensemble features such as confidence values, model participation, bounding-box information, IoU agreement, and label voting information.

The meta-classifier dataset contained 105 grouped ensemble detections. The data was divided into 84 training samples and 21 testing samples. The results are summarised below.

Evaluation Metric	Result
Training Samples	84
Testing Samples	21
Accuracy	95.2%
Macro Precision	97.4%
Macro Recall	97.4%
Macro F1-score	96.9%

The meta-classifier achieved strong performance on the prototype test split. Most test samples were classified correctly. Only one `cracked_pavement` sample was classified as `damaged_sidewalk`, which is reasonable because both categories belong to surface damage hazards and share similar visual characteristics.

These results show that the meta-classifier can learn useful decision patterns from the combined YOLO outputs. Instead of simply selecting the highest-confidence YOLO prediction, the meta-classifier considers multiple features, including model agreement and label relationship. This helps the system produce a clearer final hazard result when different models produce overlapping or related predictions.

However, the meta-classifier results should be interpreted as prototype-level results. The dataset contained only 105 grouped detections, which is relatively small. Therefore, the high accuracy demonstrates that the proposed approach works in the current prototype, but more human-verified testing data is needed for stronger real-world evaluation.

10.5 Conflict Resolution Discussion

One important advantage of the meta-classifier is its ability to handle conflicting predictions from different YOLO models. Since the four models were trained separately, some classes overlap or are contextually related. For example, pothole appears in both Member 1 and Member 3, cracked_pavement appears in both Member 1 and Member 3, damaged_sidewalk appears in both Member 1 and Member 4, and wet_slippery_floor appears in both Member 1 and Member 2.

The system uses global label mapping and IoU-based grouping to identify whether multiple predictions refer to the same physical hazard. If the bounding boxes overlap and the labels belong to a related group, the predictions are grouped and passed to the meta-classifier.

For example, in the ground opening conflict example, one model predicted pothole while another model predicted open_drain and uncovered_manhole for the same region. Instead of displaying all three labels to the user, the meta-classifier selected uncovered_manhole as the final global label with high confidence. This shows that the ensemble approach can reduce confusing multi-label outputs and provide a clearer final result.

This conflict resolution mechanism is useful because campus hazards may be visually similar. A ground opening may look like a pothole, open drain, or uncovered manhole depending on the image angle and dataset label definition. Similarly, a damaged walkway may be classified as cracked pavement, uneven pavement, surface damage, or damaged sidewalk. The meta-classifier helps standardise these outputs into one final decision.

10.6 Mobile App Testing Results

The mobile application was tested using selected campus hazard images. The testing focused on the complete workflow from image input to final result display. The app successfully allowed the user to select or capture an image, choose a campus zone, send the image to the FastAPI backend, receive the detection result, display the final hazard class, show candidate model detections, draw bounding boxes, display recommended actions, and save evidence records.

The main testing results are summarised below.

Testing Function	Result
App launch and user interface display	Passed
Campus zone selection	Passed
Image selection from gallery	Passed

Image upload to backend	Passed
YOLO candidate detection display	Passed
Meta-classifier final result display	Passed
Bounding-box display on image preview	Passed
Recommended action display	Passed
Evidence saving as image and JSON record	Passed

The results show that the mobile app successfully integrated the main components of the system. The frontend was able to communicate with the backend, and the backend was able to process images using the YOLO models and meta-classifier. The app also displayed the recommended maintenance action, making the system more useful than a basic object detection demo.

During testing, clear and visually distinctive hazards such as traffic cones, uncovered manholes, and damaged warning signs produced better results. However, context-dependent or visually similar classes sometimes produced lower confidence. Examples include `obstacle_on_walkway`, `blocked_walkway`, `uneven_pavement`, `cracked_pavement`, and `damaged_sidewalk`.

10.7 LLM Recommendation Results

The LLM-based recommendation module successfully converted the final hazard detection result into a short and practical maintenance action. Instead of displaying only the technical class label, the system provided a safety-oriented instruction that could be understood by users or maintenance staff.

For example, when the final hazard class was `uncovered_manhole` with high severity, the system generated a recommendation to place a temporary barricade, keep pedestrians away, and report the issue to campus facilities for urgent repair. When the hazard was `wet_slippery_floor`, the system recommended placing a wet-floor warning sign and requesting cleaning staff to dry the area.

The LLM recommendation module improves the usefulness of the mobile application because it connects hazard detection with practical action. However, the LLM does not decide the hazard class. The classification decision is made by the YOLO models and meta-classifier. The LLM only uses the structured result, including hazard class, category, location zone, severity, and confidence score, to generate the final recommendation.

A fallback recommendation dictionary was also prepared. This ensures that the app can still display a suitable maintenance action if the Gemini API is unavailable, the network connection fails, or the API response is empty. This makes the recommendation feature more reliable during demonstration and testing.

10.8 Overall System Discussion

The overall system demonstrates that a YOLO ensemble and meta-classifier approach can be used to detect multiple types of campus hazards. Compared with using only one YOLO model, the multi-model approach allows each member model to focus on a smaller and more specific hazard category. This helps cover a wider range of hazards while keeping each model manageable.

The global label mapping and meta-classifier are important because some hazard classes overlap between models. Without this step, the mobile app may display multiple confusing labels for the same object. With the meta-classifier, the system can compare predictions and produce one final hazard class.

The mobile app prototype also shows that the detection result can be presented in a practical way. The app does not only show the class label, but also displays confidence, severity, model candidates, bounding boxes, and recommended maintenance action. This makes the system more useful for campus safety reporting.

However, the current system is still a prototype. The individual YOLO models were trained mainly using public datasets, and some classes had limited or visually diverse data. The meta-classifier dataset was also small. Therefore, the current system is suitable for academic demonstration, but further development is required before real campus deployment.

10.9 Summary

In summary, the project achieved the main expected outcomes. Four YOLOv8n models were trained for different campus hazard categories. The meta-classifier successfully combined YOLO outputs and achieved strong prototype-level evaluation results. The mobile application successfully connected the frontend, backend, YOLO models, meta-classifier, LLM recommendation module, and evidence saving function.

The results show that the system can detect selected campus hazards and provide useful recommended actions. Stronger performance was observed for visually clear classes, while lower performance was observed for visually similar and context-dependent classes. Future improvement should focus on collecting more real campus images, improving difficult classes, expanding the meta-classifier dataset, and testing the app in more real campus scenarios.

11. Challenges and Limitations

11.1 Overview

Although the proposed campus hazard detection system successfully integrates four YOLO models, a neural-network meta-classifier, a mobile application, and an LLM-based recommendation module, several challenges and limitations were identified during the project development.

These limitations are mainly related to dataset collection, class similarity, model performance, meta-classifier dataset size, mobile app deployment, LLM reliability, and real-world campus testing. Since the system is developed as a prototype, the current implementation is suitable for academic demonstration, but further improvement is needed before it can be used in a real campus environment.

11.2 Dataset Collection Challenges

One major challenge was collecting suitable image datasets for all selected campus hazard classes. Some hazards, such as traffic cones, potholes, wet floors, and cracked pavement, were easier to find because many public datasets are available. However, other hazard classes were more difficult to collect, such as missing_barricade, broken_handrail, open_drain, uncovered_manhole, and obstacle_on_walkway.

Some public datasets did not exactly match the project class definitions. For example, missing_barricade represents the absence of a safety facility, but object detection models are better at detecting visible objects. Therefore, road barrier or barricade-related images were used as a practical representation of missing or insufficient boundary protection. Similarly, broken_handrail datasets were limited, so general handrail images were used as a visual basis for safety facility hazard detection.

Another dataset challenge was that the images were mainly collected from public sources rather than actual campus environments. Public datasets may contain different road surfaces, pavement materials, lighting conditions, weather, camera angles, and background objects. As a result, the trained models may not fully generalise to the real university campus environment.

11.3 Annotation and Label Mapping Limitations

Since the datasets were collected from different sources, their original labels were not consistent. Different datasets used different class names, class IDs, annotation styles, and data splits. Therefore, the labels had to be reviewed, filtered, and remapped into each member's final five-class structure.

This process introduced some limitations. Some original labels were directly related to the final class, while others required generalisation or contextual mapping. For example, obstacle-related labels such as obstacles, stalls, and worksite materials were mapped into blocked_walkway. This helped create a usable dataset, but the final class may contain many visually different objects.

Label mapping also created challenges for overlapping classes. Some labels, such as pothole, cracked_pavement, wet_slippery_floor, and damaged_sidewalk, appear in more than one member model. These overlapping labels required global label mapping and conflict resolution rules. Without careful mapping, the system may treat related predictions as separate hazards or display confusing results to the user.

11.4 Visual Similarity Between Hazard Classes

Several hazard classes share similar visual features, making them difficult for YOLO models to distinguish. Surface-related classes such as cracked_pavement, surface_damage, uneven_pavement, damaged_sidewalk, and pothole may all contain cracks, broken pavement, irregular ground texture, or damaged walking surfaces.

This visual similarity may cause classification confusion. For example, a damaged sidewalk may also contain cracked pavement. Uneven pavement may look similar to surface damage when the height difference is not clear in the image. A pothole may also be confused with an open drain or uncovered manhole if the camera angle makes the object appear as a dark ground opening.

Context-dependent classes were also challenging. For example, obstacle_on_walkway and blocked_walkway depend not only on the object itself, but also on whether the object is located on a pedestrian path. A box, branch, traffic cone, or construction material may not always be a hazard unless it blocks movement. YOLO models detect visual objects, but they do not fully understand the walking context of the scene.

11.5 Individual YOLO Model Limitations

Each YOLO model achieved usable prototype-level performance, but several limitations remained.

First, the models performed better on visually distinctive classes and weaker on broad or visually inconsistent classes. For example, traffic_cone performed well because it has a clear shape and colour. Missing_barricade also performed better because barrier-related objects have more consistent visual patterns. However, obstacle_on_walkway, blocked_walkway, damaged_sidewalk, and surface_damage were more difficult because they have high visual variation.

Second, some models were trained using limited hardware or reduced settings. For example, Member 2's model was trained for a smaller number of epochs using a reduced image size because of training resource limitations. This allowed the model to be completed, but it may have reduced detection performance, especially for small or unclear objects.

Third, the YOLO models were trained separately. Each model learned its own local labels and class distributions. This means the raw outputs from different models cannot be directly combined without global label mapping and meta-classifier processing.

Finally, the current YOLO models are not yet fully optimised for real-time campus deployment. Further training with more real campus images, better annotation quality, more balanced datasets, and additional hyperparameter tuning would be needed to improve accuracy and robustness.

11.6 Meta-Classifier Limitations

The meta-classifier improved the final decision-making process by combining outputs from the four YOLO models. However, it also had several limitations.

The main limitation was the small size of the meta-classifier dataset. The prototype meta-classifier dataset contained 105 grouped ensemble detections, with 84 samples used for training and 21 samples used for testing. Although the evaluation results were high, the dataset size was not large enough to prove strong real-world generalisation.

Another limitation was class imbalance. Some hazard classes had more grouped detections than others. For example, cracked pavement, traffic cone, and pothole appeared more frequently than open drain or obstacle on walkway. This may make the meta-classifier more reliable for common classes but less reliable for classes with fewer examples.

In addition, the initial final labels for the meta-classifier were based on the prototype conflict-resolution process. For a more reliable evaluation, future work should use a larger human-verified dataset where each grouped detection is manually reviewed and assigned the correct final global label.

11.7 Mobile App Limitations

The mobile application successfully demonstrated the main workflow, but it still has limitations as a prototype.

First, the current app depends on a local backend server. The backend must be running before the app can perform detection. This means the system is not yet fully deployed for real-world use. If the backend server is closed, unreachable, or incorrectly configured, the app cannot process detection requests.

Second, real-time live camera detection was not fully optimised. The most stable testing method was selecting images from the gallery or capturing still images using the camera. Continuous live video detection would require further optimisation because running four YOLO models and a meta-classifier can be computationally expensive.

Third, the app was tested mainly using an Android emulator and browser-based Flutter testing. More testing is needed on real mobile devices with different screen sizes, camera quality, network conditions, and operating system versions.

Fourth, the current app interface is still designed for prototype demonstration. Future improvements should include better evidence management, user authentication, detection history, report submission, and integration with campus maintenance systems.

11.8 LLM Recommendation Limitations

The LLM-based recommendation module improves the usefulness of the system by generating maintenance actions. However, it also has limitations.

First, the quality of the recommendation depends on the correctness of the final hazard class, severity level, and location zone. If the YOLO models or meta-classifier produce an incorrect hazard label, the LLM may generate a recommendation for the wrong hazard.

Second, LLM-generated responses may vary slightly even when similar input is provided. Although the prompt was designed to produce short and safety-oriented recommendations, the output may still differ depending on the model response.

Third, Gemini API access requires an internet connection and a valid API key. If the API key is missing, the network is unavailable, or the API request fails, the system cannot generate an LLM response. A fallback recommendation dictionary was prepared to reduce this problem, but fallback responses are less flexible than LLM-generated recommendations.

Finally, the LLM recommendation does not replace professional inspection or emergency response. It provides initial guidance only. Serious hazards such as uncovered manholes, broken handrails, open drains, or structural damage should still be handled by qualified maintenance staff.

11.9 Testing Limitations

The testing stage confirmed that the prototype could process images and display results, but the testing scope was still limited.

First, the number of testing images was small compared with real campus conditions. The system was tested using selected images for demonstration, but more diverse real campus images are needed to evaluate actual performance.

Second, the testing images may not fully represent all possible environments. Campus hazards can appear under different lighting, weather, camera angles, distances, and backgrounds. A model that works well on one test image may perform differently on another image with shadows, blur, low light, or crowded surroundings.

Third, some testing images contained multiple possible hazards or visually confusing features. This can affect both YOLO detection and meta-classifier output. For example, a damaged walkway may contain cracks, uneven surfaces, and pothole-like regions at the same time.

Fourth, the current testing mainly focused on functional testing and prototype demonstration. More systematic usability testing, latency testing, stress testing, and real-field testing would be needed for future development.

11.10 Summary of Challenges and Limitations

The main challenges and limitations of the project are summarised below.

Area	Challenge / Limitation	Possible Improvement
Dataset collection	Some hazard classes were difficult to find in public datasets	Collect more real campus images
Label mapping	Original labels from different datasets were inconsistent	Create clearer annotation and mapping guidelines
Visual similarity	Surface hazard classes looked similar	Refine class definitions and increase training examples
Context dependency	Some hazards depend on walkway context	Add location zone or scene context features
YOLO models	Some classes had low confidence or low recall	Train with more data, more epochs, and better balancing
Meta-classifier	Dataset contained only 105 grouped detections	Build a larger human-verified meta-classifier dataset

Mobile app	App depends on a local backend server	Deploy backend to a remote server
Real-time detection	Live camera detection was not fully optimised	Optimise model loading and inference speed
LLM recommendation	Requires API key and internet connection	Improve fallback rules and validate recommendations
Testing	Limited number of real campus test images	Conduct larger field testing on campus

11.11 Future Improvements

Several future improvements can be made to strengthen the system.

First, the group should collect more real campus images from different locations, lighting conditions, weather conditions, and camera angles. This would help the models learn more realistic campus-specific visual patterns.

Second, the annotation process should be improved by using clearer class definitions and more consistent bounding-box guidelines. This is especially important for visually similar classes such as `cracked_pavement`, `damaged_sidewalk`, `uneven_pavement`, and `surface_damage`.

Third, the YOLO models can be improved by increasing the number of training images, training for more epochs, applying better dataset balancing, and testing different YOLO model sizes. Difficult classes such as `obstacle_on_walkway`, `blocked_walkway`, and `damaged_sidewalk` should receive more training examples.

Fourth, the meta-classifier should be trained with a larger and human-verified dataset. Instead of relying only on prototype grouped detections, future work should manually check final labels to improve the reliability of the evaluation.

Fifth, the mobile app should be improved by deploying the backend to a remote server, supporting more stable live camera detection, adding detection history, and connecting the app to an actual campus maintenance reporting workflow.

Finally, the LLM recommendation module should be improved by adding stronger safety validation, institution-specific maintenance contacts, and more detailed fallback recommendations for different hazard severity levels.

11.12 Summary

In summary, the project successfully produced a working prototype, but several limitations remain. The main issues are related to dataset availability, visual similarity between hazard classes, context-dependent hazard definitions, limited meta-classifier data, local backend dependency, and limited real campus testing.

These limitations are acceptable for a prototype project, but they should be addressed in future development. With more real campus data, better annotation quality, improved model training, larger meta-classifier evaluation, and stronger app deployment, the system can become more accurate, reliable, and practical for campus hazard reporting.

12. Conclusion

12.1 Project Summary

This project successfully developed a prototype of a campus hazard detection mobile application using YOLO ensemble models, a neural-network meta-classifier, and an LLM-based recommendation module. The system was designed to detect selected campus maintenance and safety hazards from images and provide useful maintenance recommendations to support faster hazard reporting and response.

The project covered four main hazard categories: walkway surface hazards, walkway obstruction hazards, drainage and ground surface hazards, and access, boundary and safety facility hazards. Each group member trained one YOLOv8n object detection model with five hazard classes. These individual models were then designed to work together as an ensemble system.

The final prototype includes the following main components:

Component	Outcome
Individual YOLO models	Four YOLOv8n models were trained for different campus hazard categories
Global label mapping	Overlapping and related labels were standardised into global hazard labels
Meta-classifier	A neural-network MLP meta-classifier was developed to combine YOLO outputs

Mobile app	A Flutter-based app named SafeCampus AI was implemented
Backend API	A FastAPI backend was used to run model inference and return structured results
LLM recommendation	A Gemini-based or fallback recommendation module was used to generate maintenance actions
Evidence saving	Detection results could be saved as image evidence and JSON records

Overall, the project achieved its main goal of producing a working prototype that connects hazard detection, ensemble decision-making, mobile app display, and safety-oriented recommendation.

12.2 Achievement of Project Objectives

The first objective was to collect and prepare annotated datasets for different campus hazard categories. This objective was achieved because each member prepared a YOLO-format dataset based on their assigned hazard classes. The datasets were cleaned, remapped, and organised into training and validation folders.

The second objective was to train individual YOLO object detection models. This objective was achieved because all four members trained their own YOLOv8n models and produced model outputs such as `best.pt`, training results, confusion matrices, and prediction examples.

The third objective was to design a global label mapping structure. This objective was achieved through the identification of exact overlaps, contextual overlaps, and related hazard labels. Examples include `wet_slippery_floor`, `pothole`, `cracked_pavement`, `damaged_sidewalk`, and `obstacle_on_walkway / blocked_walkway`.

The fourth objective was to develop a neural-network-based meta-classifier. This objective was achieved by creating an MLP meta-classifier that uses ensemble features such as confidence scores, bounding-box information, model participation, IoU agreement, and label voting. The meta-classifier was able to produce final global hazard labels from grouped YOLO detections.

The fifth objective was to integrate the trained models into a mobile application. This objective was achieved through the SafeCampus AI Flutter app and FastAPI backend. The app can send images to the backend, display detection results, show candidate model outputs, draw bounding boxes, and save detection evidence.

The sixth objective was to generate recommended maintenance actions using an LLM-based module. This objective was achieved by using structured hazard information to generate concise and safety-oriented maintenance recommendations. A fallback recommendation dictionary was also prepared to ensure that recommendations remain available when the API is not accessible.

The seventh objective was to evaluate the system. This objective was achieved through individual YOLO model evaluation, meta-classifier testing, mobile app functional testing, and testing with selected hazard images.

12.3 Overall Findings

The results show that the proposed system can detect selected campus hazards and present the results in a user-friendly mobile interface. The YOLO models performed better on hazard classes with clear and consistent visual features, such as `wet_slippery_floor`, `traffic_cone`, `missing_barricade`, and `uncovered_manhole`. These classes are easier to detect because they have more recognisable shapes, colours, or object boundaries.

However, the system had more difficulty with visually similar or context-dependent classes. For example, `cracked_pavement`, `uneven_pavement`, `surface_damage`, and `damaged_sidewalk` may share similar ground surface patterns. Similarly, `obstacle_on_walkway` and `blocked_walkway` are difficult because the hazard depends on whether an object actually blocks a pedestrian path.

The meta-classifier helped address this issue by combining outputs from multiple YOLO models. Instead of directly displaying every YOLO prediction, the system used global label mapping, IoU grouping, and ensemble features to produce one final hazard result. This made the final output clearer and more suitable for the mobile application.

The mobile app testing showed that the prototype could complete the full workflow from image input to final result display. The app successfully displayed the final hazard class, confidence score, severity level, bounding box, candidate detections, and recommended maintenance action. The evidence saving function also supported testing documentation and report evidence.

12.4 System Contribution

The main contribution of this project is the development of an integrated campus hazard detection prototype. Instead of using only one object detection model, the system uses four specialised YOLO models and combines their outputs using a meta-classifier. This design allows the system to cover a wider range of campus hazards while still keeping each individual model focused on a smaller hazard group.

Another contribution is the use of global label mapping and conflict resolution. Since some hazard classes overlap across member models, the system needs a method to standardise

different labels and avoid confusing outputs. The meta-classifier supports this by analysing model agreement, confidence scores, bounding-box overlap, and related hazard groups.

The LLM-based recommendation module also improves the usefulness of the system. A normal object detection app may only display a class label, but this system also provides a practical maintenance action. This makes the result more understandable and useful for campus users and maintenance staff.

12.5 Limitations

Although the project achieved its main objectives, several limitations remain. Most of the datasets were collected from public sources rather than actual campus environments. Therefore, the models may not fully match the lighting, camera angles, pavement materials, safety sign designs, handrail structures, and background conditions of the real campus.

Some hazard classes were also difficult to define and detect. For example, `missing_barricade` represents the absence of a safety facility, which is difficult for an object detection model to detect directly. Context-dependent classes such as `blocked_walkway` and `obstacle_on_walkway` also require scene understanding, not only object recognition.

The meta-classifier dataset was still small and mainly suitable for prototype evaluation. More human-verified grouped detections are needed to evaluate whether the meta-classifier can generalise well in real campus scenarios.

The mobile app currently depends on a local backend server, so it is not yet fully deployed for public use. Real-time live camera detection also requires further optimisation. In addition, the LLM recommendation module depends on the correctness of the final hazard classification and may require internet access when using an external API.

12.6 Future Work

Future work should focus on improving dataset quality, model accuracy, system deployment, and real-world usability.

First, more real campus images should be collected from different locations, weather conditions, lighting conditions, and camera angles. This would help the YOLO models adapt better to the actual campus environment.

Second, difficult classes should be improved with clearer annotation guidelines and more balanced training examples. This is especially important for `cracked_pavement`, `uneven_pavement`, `damaged_sidewalk`, `obstacle_on_walkway`, `blocked_walkway`, `open_drain`, and `broken_handrail`.

Third, the meta-classifier should be trained and tested using a larger human-verified dataset. This would make the final decision layer more reliable and provide stronger evidence for real-world performance.

Fourth, the mobile app should be improved by deploying the backend to a remote server, supporting more stable live camera detection, adding detection history, and connecting the app to a campus maintenance reporting workflow.

Finally, the LLM recommendation module can be improved by adding stronger safety validation, more detailed fallback rules, and institution-specific maintenance contact information.

12.7 Final Conclusion

In conclusion, the Campus Hazard Detection Mobile App Using YOLO Ensemble and Meta-Classifer project successfully produced a functional prototype for detecting and reporting campus hazards. The system combines four YOLOv8n models, global label mapping, a neural-network meta-classifier, a Flutter mobile application, a FastAPI backend, and an LLM-based recommendation module.

The prototype demonstrates that image-based object detection can support campus hazard identification and maintenance reporting. It can detect selected hazards, combine multi-model predictions, display the final result in a mobile interface, and provide a short recommended action. Although the system still has limitations, it provides a strong foundation for future development into a more accurate, reliable, and practical campus safety support tool.

Appendix

Proposed Group Work Distribution

Member	Main Individual Responsibility	Shared System Responsibility
Member 1	Train YOLO model for walkway surface hazards	Flutter home page, image upload or camera input, dataset preparation support

Member 2	Train YOLO model for walkway obstruction hazards	Meta-classifier feature extraction, MLP training, conflict-resolution integration
Member 3	Train YOLO model for ground-opening hazards	Flutter result page, display bounding boxes, confidence and hazard labels
Member 4	Train YOLO model for safety infrastructure hazards	Flutter recommendation page, testing, error logging, report and video preparation

All members contributed to dataset checking, global label mapping discussion, conflict-resolution testing, report review, and screencast preparation.

Development Logbook

Date	Member	Activity	Hours	Result / Issue	Next Action
6/10	Member 1	Collected and organised images for walkway surface hazards.	2.0	Dataset included pavement-related classes with similar visual patterns.	Check labels and prepare YOLO dataset structure.
6/10	Member 2	Collected and merged datasets for walkway obstruction hazards.	2.0	Different source datasets used inconsistent class names and class IDs.	Remap labels into one five-class structure.

6/10	Member 3	Prepared dataset for open drains, uncovered manholes, potholes, uneven pavement, and cracked pavement.	2.0	Ground-opening categories had visual overlap.	Review annotations and class definitions.
6/10	Member 4	Prepared dataset for safety infrastructure and blocked walkway hazards.	2.0	Some images contained multiple hazards in one scene.	Verify bounding boxes and labels.
6/15	Member 1	Trained the first individual YOLO model and reviewed validation output.	2.5	Surface-damage classes showed possible confusion.	Save model results and confusion matrix.
6/15	Member 2	Trained the walkway obstruction YOLO model and analysed validation predictions.	2.5	Traffic cones were easier to detect than irregular obstacles and branches.	Record metrics and error analysis.
6/15	Member 3	Trained the ground-opening YOLO model and tested sample images.	2.5	Pothole, open drain, and uncovered manhole could overlap.	Prepare overlap examples for meta-classifier testing.

6/15	Member 4	Trained the safety infrastructure YOLO model and checked prediction examples.	2.5	Complex construction scenes caused some false detections.	Save validation evidence and improve test cases.
6/20	Member 1	Developed the Flutter home page and image selection interface.	2.0	Basic interface for selecting or capturing an image was created.	Connect image input to prediction workflow.
6/21	Member 3	Developed the Flutter result page for displaying final hazard labels and confidence values.	2.0	Result layout was created.	Add annotated image and status display.
6/21	Member 4	Added recommendation and detection-status display in Flutter.	2.0	High Confidence, Medium Confidence, and Needs Review statuses were prepared.	Connect recommendations to final prediction output.
6/23	Member 2	Created global label mapping and IoU-based grouping logic.	2.5	Similar labels from different YOLO models were mapped into global classes.	Generate grouped ensemble detections.

6/23	Member 2	Generated the meta-classifier dataset from four YOLO model outputs.	2.0	Initial dataset had imbalanced class distribution.	Add more test images for weaker classes.
6/24	Member 1	Added additional images for underrepresented surface and ground hazard cases.	1.5	More diverse test images were collected.	Re-run ensemble prediction.
6/24	Member 3	Added additional images for open drain, uneven pavement, and uncovered manhole cases.	1.5	More grouped detection samples were produced.	Check class distribution.
6/25	Member 4	Added test images for broken handrail, warning sign, and walkway blockage cases.	1.5	More safety infrastructure examples were available.	Test final ensemble output.
6/26	Member 2	Trained and evaluated the MLP meta-classifier.	2.5	Prototype model achieved 95.2% accuracy and 96.9% macro F1-score on the test split.	Save model, metrics, and confusion matrix.

6/26	All Members	Tested the integrated ensemble workflow using controlled hazard images.	2.0 each	Some single-model detections produced possible false positives.	Use confidence status and Needs Review handling.
6/27	Member 1	Integrated image upload flow with Flutter testing interface.	1.5	Input image could be passed to the system workflow.	Perform controlled live-camera demo.
6/27	Member 2	Integrated final result display with annotated output image.	1.5	Bounding boxes, final labels, and confidence values were displayed.	Improve result-page layout.
6/27	Member 4	Tested recommendations, status display, and error handling in Flutter.	1.5	Basic output display was verified.	Record latency and failed cases.
6/27	All Members	Prepared report evidence, screenshots, conflict examples, and screencast materials.	2.0 each	Required evidence was organised.	Complete final review and submission.